

TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN



BÀI GIẢNG **NHẬP MÔN LẬP TRÌNH**

LÊ THỊ BÍCH HẰNG
BỘ MÔN KỸ THUẬT PHẦN MỀM

KHÁNH HÒA – 10/2016

MỤC LỤC

PHẦN 1- LÝ THUYẾT

CHƯƠNG 1 - TỔNG QUAN VỀ LẬP TRÌNH MÁY TÍNH.....	8
1.1. CÁC KHÁI NIỆM CƠ BẢN VỀ LẬP TRÌNH MÁY TÍNH.....	8
1.1.1. Lập trình máy tính	8
1.1.2. Thuật toán	8
1.1.3. Chương trình máy tính.....	8
1.2. BIỂU DIỄN THUẬT TOÁN.....	9
1.2.1. Ngôn ngữ tự nhiên	9
1.2.2. Lưu đồ (sơ đồ khối)	9
1.2.3. Mã giả	10
1.2.4. Một số ví dụ minh họa về biểu diễn thuật toán	11
1.3. NGÔN NGỮ LẬP TRÌNH	12
1.3.1. Khái niệm ngôn ngữ lập trình.....	12
1.3.2. Ngôn ngữ máy (Machine language)	12
1.3.3. Hợp ngữ (Assembly language)	13
1.3.4. Ngôn ngữ lập trình cấp cao (high-level programming language)	13
1.3.5. Chương trình dịch.....	14
BÀI TẬP CHƯƠNG 1	16
CHƯƠNG 2 – TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH C.....	17
2.1. GIỚI THIỆU NGÔN NGỮ LẬP TRÌNH C	17
2.2. CÁC THÀNH PHẦN CƠ BẢN CỦA NGÔN NGỮ C.....	18
2.2.1. Tập ký tự.....	18
2.2.2. Từ khóa.....	18
2.2.3. Tên/Định danh	18
2.2.4. Lời chú thích.....	19
2.3. CÁC KIỂU DỮ LIỆU CƠ BẢN	19
2.3.1. Kiểu dữ liệu	19
2.3.2. Các kiểu dữ liệu cơ sở	19
2.3.3. Hằng	20
2.3.4. Biến.....	23
2.4. BIỂU THỨC VÀ PHÉP TOÁN	24
2.4.1. Biểu thức.....	24
2.4.2. Phép toán	24
2.4.3. Thứ tự ưu tiên của các phép toán.....	28
2.4.4. Chuyển đổi kiểu giá trị	29
2.5. CÂU LỆNH	29

2.5.1. Khái niệm câu lệnh	29
2.5.2. Lệnh gán	30
2.5.3. Các lệnh nhập xuất dữ liệu	32
2.6. CẤU TRÚC CỦA MỘT CHƯƠNG TRÌNH C	36
BÀI TẬP CHƯƠNG 2	39
CHƯƠNG 3 – CÁC CẤU TRÚC ĐIỀU KHIỂN	42
3.1. CẤU TRÚC TUẦN TỰ	42
3.2. CẤU TRÚC Rẽ NHÁNH	42
3.2.1. Câu lệnh if	42
3.2.2. Câu lệnh switch	44
3.3. CẤU TRÚC LẶP	47
3.3.1. Câu lệnh for	47
3.3.2. Câu lệnh while	49
3.3.3. Câu lệnh do...while	50
3.3.4. Sự khác nhau giữa các vòng lặp	52
3.3.5. Các câu lệnh đặc biệt sử dụng trong cấu trúc lặp	53
BÀI TẬP CHƯƠNG 3	55
CHƯƠNG 4 – HÀM	61
4.1. HÀM	61
4.1.1. Khái niệm hàm	61
4.1.2. Cú pháp hàm	62
4.2. THAM SỐ HÌNH THỨC VÀ THAM SỐ THỰC	62
4.3. SỬ DỤNG HÀM	64
4.3.1. Gọi hàm	64
4.3.2. Nguyên tắc hoạt động của hàm	65
4.3.3. Truyền tham số cho hàm	65
4.3.4. Phạm vi của biến	68
BÀI TẬP CHƯƠNG 4	69
CHƯƠNG 5 – LẬP TRÌNH VỚI DỮ LIỆU CÓ CẤU TRÚC	71
5.1. KIỂU MẢNG	71
5.1.1. Giới thiệu kiểu mảng	71
5.1.2. Mảng một chiều	71
5.1.3. Mảng nhiều chiều	75
5.2. CHUỖI KÝ TỰ	78
5.2.1. Khai báo	78
5.2.2. Nhập/xuất chuỗi ký tự	79
5.2.3. Một số hàm thao tác trên chuỗi ký tự	80
5.3. KIỂU STRUCT	82

5.3.1. Khai báo, xử lý dữ liệu kiểu cấu trúc.....	82
5.3.2. Truy xuất đến các thành phần của biến cấu trúc.....	84
5.3.3. Gán dữ liệu kiểu cấu trúc.....	84
5.3.4. Mảng cấu trúc	86
BÀI TẬP CHƯƠNG 5	90
TÀI LIỆU THAM KHẢO.....	92
 <u>PHẦN 2- BÀI TẬP THỰC HÀNH</u>	
LAB 01 – BIỂU THỨC, TOÁN TỬ TRONG C	94
LAB 02 – CÂU LỆNH Rẽ NHÁNH	96
LAB 03 – CÂU LỆNH LẶP.....	98
LAB 04 – HÀM	100
LAB 05 – MẢNG VÀ CẤU TRÚC	102
 <u>PHẦN 3- PHỤ LỤC</u>	
A - MỘT SỐ HÀM CHUẨN THÔNG DỤNG.....	106
B - HƯỚNG DẪN SỬ DỤNG PHẦN MỀM DEV-C++	109
C - MỘT SỐ QUY ƯỚC KHI VIẾT MÃ NGUỒN C/C++	115
D - CÁC LỖI THƯỜNG GẶP KHI VIẾT CHƯƠNG TRÌNH C/C++	117

ĐỀ CƯƠNG HỌC PHẦN

1. Thông tin về học phần:

Tên học phần:

- Tiếng Việt: **NHẬP MÔN LẬP TRÌNH**
- Tiếng Anh: **INTRODUCTION TO PROGRAMMING**

Mã học phần:

Số tín chỉ: 3(2-1)

Đào tạo trình độ: Đại học/Cao đẳng

Học phần tiên quyết: Không

2. Mô tả tóm tắt học phần:

Học phần trang bị cho người học kiến thức, kỹ năng cơ bản về lập trình máy tính: quy trình xây dựng một chương trình máy tính, thuật toán, ngôn ngữ lập trình, phương pháp lập trình cấu trúc với ngôn ngữ lập trình C/C++.

3. Mục tiêu:

Giúp sinh viên có đủ kiến thức và kỹ năng cần thiết giải quyết những bài toán cơ bản về tính toán, xây dựng giải thuật, sử dụng ngôn ngữ C/C++ cài đặt giải thuật thành chương trình máy tính theo phương pháp lập trình cấu trúc.

4. Kết quả học tập mong đợi (KQHT):

Sau khi học xong học phần, sinh viên có thể:

- Trình bày nguyên lý hoạt động của máy tính điện tử, vai trò của lập trình máy tính, vai trò và cơ chế hoạt động của ngôn ngữ lập trình.
- Vận dụng các phương pháp biểu diễn giải thuật: dùng ngôn ngữ tự nhiên, dùng lưu đồ, dùng mã giả.
- Phân biệt được các cấp độ ngôn ngữ lập trình: ngôn ngữ máy, hợp ngữ, ngôn ngữ cấp cao; hiểu được ý nghĩa của chương trình dịch.
- Trình bày các đặc điểm cơ bản của ngôn ngữ lập trình C; vận dụng các kiểu dữ liệu, cách khai báo và sử dụng hằng, biến, các quy tắc cú pháp biểu diễn phép toán, biểu thức để tổ chức cấu trúc dữ liệu phù hợp cho bài toán cụ thể.
- Vận dụng cú pháp của câu lệnh căn bản: nhập xuất dữ liệu, các cấu trúc điều khiển (tuần tự, rẽ nhánh, lặp) để giải quyết bài toán thực tế.
- Trình bày được nguyên lý của phương pháp lập trình cấu trúc, biết cách vận dụng để phân tích bài toán thành các chức năng con (hàm) phù hợp.
- Trình bày được mục đích, ý nghĩa của kiểu mảng và cách tổ chức kiểu mảng trong máy tính bằng ngôn ngữ C; nắm được các giải thuật tìm kiếm, sắp xếp các phần tử trong mảng. Biết vận dụng để tổ chức và xử lý dữ liệu kiểu mảng cho bài toán cụ thể.

- h) Mô tả được cấu trúc và cách tổ chức kiểu chuỗi trong máy tính bằng ngôn ngữ C. Biết vận dụng các hàm của ngôn ngữ C để giải quyết các yêu cầu về xử lý chuỗi.
- i) Trình bày được mục đích, ý nghĩa của kiểu cấu trúc; cú pháp tổ chức kiểu cấu trúc bằng ngôn ngữ C. Biết vận dụng để tổ chức và xử lý dữ liệu kiểu cấu trúc cho bài toán cụ thể.

PHẦN 1

LÝ THUYẾT

CHƯƠNG 1

TỔNG QUAN VỀ LẬP TRÌNH MÁY TÍNH

1.1. CÁC KHÁI NIỆM CƠ BẢN VỀ LẬP TRÌNH MÁY TÍNH

1.1.1. Lập trình máy tính

Máy tính điện tử (electronic computer) là công cụ cho phép xử lý thông tin một cách tự động theo chương trình được xác định trước mà không cần sự tham gia trực tiếp của con người. Khi sử dụng máy tính để giải quyết một vấn đề nào đó, bản thân máy tính không thể tự đưa ra cách giải quyết mà con người phải cung cấp đầy đủ các chỉ thị hướng dẫn máy tính xử lý theo một dãy các thao tác xác định. Tập các chỉ thị như vậy được gọi là chương trình (computer program).

Lập trình máy tính, gọi tắt là lập trình (programming), là kỹ thuật cài đặt một hoặc nhiều thuật toán trừu tượng có liên quan với nhau bằng một ngôn ngữ lập trình để tạo ra một chương trình máy tính.

1.1.2. Thuật toán

Thuật toán (algorithm), còn gọi là giải thuật, là tập hợp (dãy) hữu hạn các chỉ thị (hành động) được định nghĩa rõ ràng nhằm giải quyết một bài toán cụ thể nào đó. Thuật toán gồm một số tính chất sau:

- Tính chính xác: quá trình tính toán hay các thao tác máy tính thực hiện là chính xác.
- Tính rõ ràng: các câu lệnh minh bạch được sắp xếp theo thứ tự xác định.
- Tính khách quan: một thuật toán được thực hiện bởi nhiều người hoặc máy tính khác nhau nhưng kết quả phải như nhau với cùng giá trị đầu vào.
- Tính phổ dụng: thuật toán không chỉ giải quyết một bài toán cụ thể mà phải có khả năng giải quyết một lớp các bài toán có đầu vào tương tự nhau.
- Tính kết thúc: thuật toán phải dừng sau hữu hạn các bước tính toán.

Ví dụ: Thuật toán giải phương trình bậc nhất: $ax + b = 0$ (a, b là các số thực).

Đầu vào: $a, b \in \mathbb{R}$

Đầu ra: nghiệm phương trình $ax + b = 0$

- Nếu $a = 0$
 - $b = 0$ thì phương trình có nghiệm bất kì.
 - $b \neq 0$ thì phương trình vô nghiệm.
- Nếu $a \neq 0$
 - Phương trình có nghiệm duy nhất $x = -b/a$.

1.1.3. Chương trình máy tính

Chương trình máy tính (computer program) là một tập hợp những câu lệnh được viết bằng một hoặc nhiều ngôn ngữ lập trình theo một trật tự xác định nhằm tự động thực hiện một số chức năng hoặc giải quyết một vấn đề cụ thể nào đó.

Chương trình máy tính thực hiện các chức năng của nó bằng cách gửi các chỉ thị trực tiếp đến phần cứng, hoặc bằng cách cung cấp dữ liệu để phục vụ các chương trình hay phần mềm khác.

➤ Quy trình phát triển một chương trình máy tính

Để tạo ra một chương trình máy tính, thực hiện lần lượt các bước sau:

- Bước 1. Xác định vấn đề – bài toán.
- Bước 2. Lựa chọn phương pháp giải.
- Bước 3. Xây dựng thuật toán: sử dụng ngôn ngữ tự nhiên hoặc lưu đồ - sơ đồ khối hoặc mã giả để biểu diễn thuật toán.
- Bước 4. Cài đặt chương trình.
- Bước 5. Hiệu chỉnh chương trình: sửa các lỗi cú pháp và lỗi ngữ nghĩa.
- Bước 6. Thực hiện chương trình.

1.2. BIỂU DIỄN THUẬT TOÁN

Thuật toán là một phương pháp thể hiện lời giải bài toán nên phải tuân theo một số quy tắc nhất định. Để có thể truyền đạt thuật toán cho người khác hay chuyển thuật toán thành chương trình máy tính, ta phải có phương pháp biểu diễn thuật toán. Có 3 phương pháp biểu diễn thuật toán phổ biến:

- Dùng ngôn ngữ tự nhiên.
- Dùng lưu đồ (flowchart).
- Dùng mã giả (pseudo code).

1.2.1. Ngôn ngữ tự nhiên

Trong cách biểu diễn thuật toán theo ngôn ngữ tự nhiên, người ta sử dụng ngôn ngữ thường ngày để liệt kê các bước của thuật toán (Ví dụ về thuật toán trong mục 1.1.2 sử dụng ngôn ngữ tự nhiên). Phương pháp biểu diễn này không yêu cầu người viết thuật toán cũng như người đọc thuật toán phải nắm các quy tắc. Tuy vậy, cách biểu diễn này thường dài dòng, không thể hiện rõ cấu trúc của thuật toán, đôi lúc gây hiểu lầm hoặc khó hiểu cho người đọc. Gần như không có một quy tắc cố định nào trong việc thể hiện thuật toán bằng ngôn ngữ tự nhiên. Tuy vậy, để dễ đọc, ta nên viết các bước con lù vào bên phải và đánh số bước theo quy tắc phân cấp như 1, 1.1, 1.1.1, ...

Ví dụ: Giải thuật giải phương trình bậc nhất $ax + b = 0$ được biểu diễn bằng cách sử dụng ngôn ngữ tự nhiên như sau:

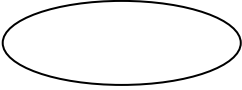
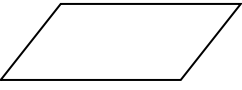
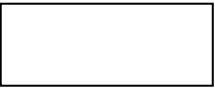
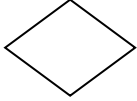
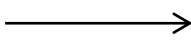
- Bước 1: Nhập 2 số a và b.
- Bước 2: Nếu $a = 0$ thì thực hiện bước 3, ngược lại thực hiện bước 4.
- Bước 3: Nếu $b = 0$ thì thông báo phương trình vô số nghiệm và kết thúc chương trình, ngược lại thông báo phương trình vô nghiệm và kết thúc chương trình.
- Bước 4: Thông báo nghiệm của phương trình là $-b/a$ và kết thúc.

1.2.2. Lưu đồ (sơ đồ khối)

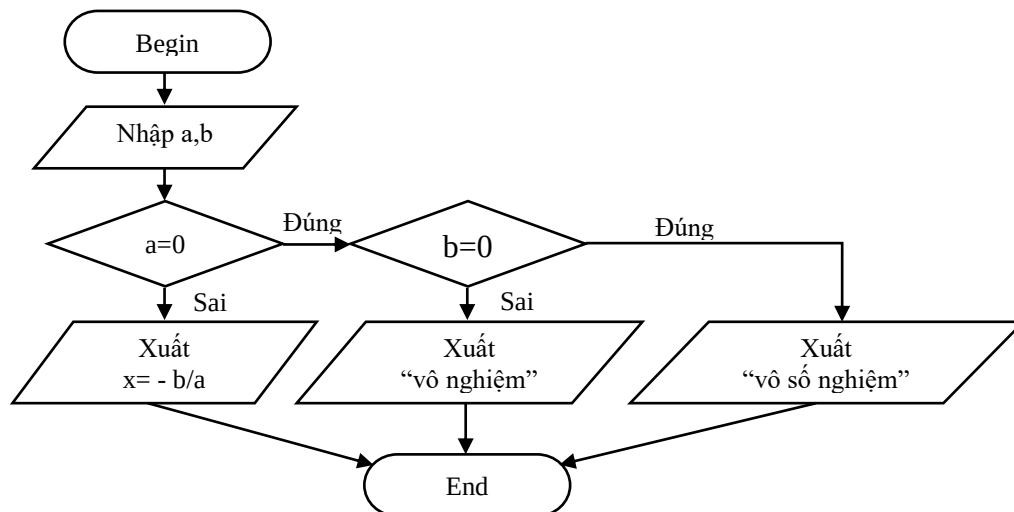
Lưu đồ (flowchart) hay sơ đồ khối là một công cụ trực quan để diễn đạt các thuật toán. Biểu diễn thuật toán bằng lưu đồ sẽ giúp người đọc theo dõi được sự phân nhánh các trường hợp và

quá trình xử lý của thuật toán. Lưu đồ thường được dùng để biểu diễn những thuật toán có tính rắc rối, khó theo dõi được quá trình xử lý.

Lưu đồ sử dụng các sơ đồ hình khối để mô tả thuật toán, mỗi hình khối qui định một hành động. Dưới đây là bảng mô tả các ký hiệu được sử dụng trong lưu đồ:

Ký hiệu	Ý nghĩa
	Khối giới hạn (Chỉ thị bắt đầu/ kết thúc)
	Khối vào ra (Nhập/ Xuất dữ liệu)
	Khối thao tác (Ghi thao tác cần thực hiện)
	Khối lựa chọn (Tùy điều kiện sẽ rẽ nhánh)
	Đường đi (Chỉ hướng thao tác tiếp theo)

Ví dụ: Dùng lưu đồ để biểu diễn giải thuật giải phương trình bậc nhất $ax + b = 0$ như sau:



1.2.3. Mã giả

Mã giả (pseudo code) là một bản mô tả thuật toán ngắn gọn trong đó vay mượn cú pháp của một số ngôn ngữ lập trình nhưng thường bỏ đi những chi tiết không cần thiết để giúp hiểu rõ thuật toán hơn, như bỏ đi chương trình con, khai báo biến và những đoạn mã đặc biệt của hệ thống.

Mục đích của việc sử dụng mã giả là nó có thể con người đọc hiểu dễ dàng hơn những ngôn ngữ lập trình thông thường, và mã giả có thể là một bản mô tả cô đọng và độc lập với môi trường phát triển ghi lại những bước thực hiện của một thuật toán. Dùng mã giả vừa tận dụng được các khái niệm trong ngôn ngữ lập trình, vừa giúp người cài đặt dễ dàng nắm bắt nội dung thuật toán. Không có bất cứ tiêu chuẩn nào cho cú pháp mã giả, vì vậy một chương trình viết bằng mã giả không phải là một chương trình có thể thực thi được.

Ví dụ: Dùng mã giả (bằng ngôn ngữ lập trình Pascal) để biểu diễn thuật toán giải phương trình bậc nhất $ax + b = 0$ như sau:

If $a = 0$ Then

Begin

If $b = 0$ Then Xuất “Phương trình vô số nghiệm”

Else Xuất “Phương trình vô nghiệm”

End

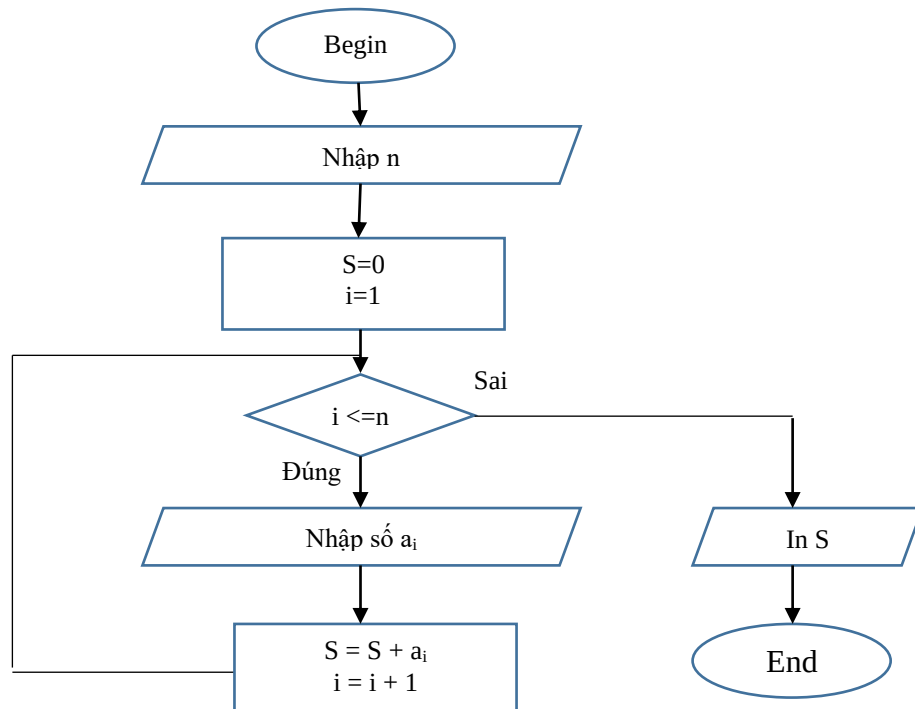
Else Xuất “Phương trình có nghiệm $x = -b/a$ ”

1.2.4. Một số ví dụ minh họa về biểu diễn thuật toán

Ví dụ 1: Viết chương trình tính tổng n số hạng $a_1 + a_2 + \dots + a_n$.

Để tính tổng trên, chúng ta sử dụng phương pháp “cộng tích lũy”, khởi đầu cho giá trị tổng $S = 0$. Sau mỗi lần nhận được một số hạng a_i , ta cộng tích lũy a_i vào S (lấy giá trị được lưu trữ trong S , cộng thêm a_i và lưu trữ lại vào S). Tiếp tục quá trình này đến khi ta tích lũy được a_n vào S thì ta có S là tổng các a_i .

- Giải thuật được mô tả bằng ngôn ngữ tự nhiên:
 - Bước 1: Nhập số các số hạng n (Nhập n).
 - Bước 2: Cho $S = 0$ (lưu trữ số 0 trong S)
 - Bước 3: Cho $i = 1$ (lưu trữ số 1 trong i)
 - Bước 4: Kiểm tra nếu $i \leq n$ thì thực hiện bước 5, ngược lại thực hiện bước 8.
 - Bước 5: Nhập a_i
 - Bước 6: Tính $S = S + a_i$ (lưu trữ giá trị $S + a_i$ trong S)
 - Bước 7: Tăng i lên 1 đơn vị và quay lại bước 4.
 - Bước 8: In S và kết thúc chương trình.
- Giải thuật được mô tả bằng lưu đồ:



Ví dụ 2: Viết chương trình cho phép nhập vào một số n , sau đó lần lượt nhập vào n giá trị $a_1, a_2, \dots, a_n$. Hãy tìm và in ra giá trị lớn nhất trong n số $a_1, a_2, \dots, a_n$.

Để giải quyết bài toán trên, chúng ta áp dụng phương pháp “thử và sửa”. Ban đầu giả sử a_1 là số lớn nhất (được lưu trong giá trị max); sau đó lần lượt xét các a_i còn lại, nếu a_i nào lớn hơn giá trị max thì lúc đó max sẽ nhận giá trị là a_i . Sau khi đã xét hết các a_i thì max chính là giá trị lớn nhất cần tìm.

- Giải thuật được mô tả bằng ngôn ngữ tự nhiên:
 - Bước 1: Nhập số n .
 - Bước 2: Nhập số thứ nhất a_1 .
 - Bước 3: Gán $\text{max} = a_1$.
 - Bước 4: Gán $i = 2$.
 - Bước 5: Nếu $i \leq n$ thì thực hiện bước 6, ngược lại thực hiện bước 9.
 - Bước 6: Nhập a_i .
 - Bước 7: Nếu $\text{max} < a_i$ thì gán $\text{max} = a_i$.
 - Bước 8: Tăng i lên một đơn vị và quay lại bước 5.
 - Bước 9: In max và kết thúc.
- Giải thuật được mô tả bằng lưu đồ: xem như bài tập.

1.3. NGÔN NGỮ LẬP TRÌNH

1.3.1. Khái niệm ngôn ngữ lập trình

Để máy tính có thể thực hiện được thuật toán, cần phải thể hiện thuật toán dưới dạng các dòng lệnh theo các quy ước nào đó mà máy tính có thể thực hiện được một cách trực tiếp, hoặc viết dưới dạng để có thể sinh tự động dạng mà máy tính có thể thực hiện trực tiếp. Tập các kí hiệu và các quy tắc viết các lệnh để thể hiện thuật toán được gọi là một ngôn ngữ lập trình (programming language). Các quy tắc để viết chương trình được gọi là cú pháp (syntax) của ngôn ngữ lập trình. Mỗi chương trình sẽ mang một ý nghĩa nhất định mà ta gọi là ngữ nghĩa (semantic) của chương trình.

Có thể chia ngôn ngữ lập trình thành các loại sau: ngôn ngữ máy, hợp ngữ và ngôn ngữ cấp cao.

1.3.2. Ngôn ngữ máy (Machine language)

Chương trình trong ngôn ngữ máy là dãy các mã lệnh mà CPU có thể thực hiện trực tiếp. Đó là ngôn ngữ lập trình duy nhất mà máy tính "hiểu được". Trong các bậc ngôn ngữ giao tiếp với máy tính, ngôn ngữ máy ở bậc thấp nhất nhưng hiệu quả của chương trình cao nhất vì nó trực tiếp ra lệnh cho máy tính mà không cần qua các khâu trung gian. Tùy theo thiết kế về phần cứng, mỗi loại máy tính có một ngôn ngữ máy khác nhau. Các lệnh viết bằng ngôn ngữ máy nói chung ở dạng nhị phân hoặc biến thể của chúng trong hệ đếm 16.

Chương trình được viết bằng hợp ngữ hay ngôn ngữ cấp cao phải được dịch sang ngôn ngữ máy thì mới có thể thi hành được trên máy tính.

Ví dụ sau đây là một đoạn chương trình viết bằng ngôn ngữ máy của máy tính dùng bộ vi xử lý Intel 8086.

Mã trên hệ nhị phân	Mã hệ 16	Ý nghĩa
1001 0001 0110 0100 0001 0000	A1 64 10	Nạp số 2 byte từ 1064 lên AX
0000 0011 0110 0110 0001 0000	03 65 10	Cộng AX với số 2 byte ở 1066 kết quả để trên AX
1010 0011 0000 0000 0010 1011	A3 00 2B	Chuyển kết quả từ AX về hai byte bắt đầu từ 2B00

Đoạn chương trình trên thực hiện cộng hai số nguyên hai byte ở trong các địa chỉ 1064 và 1066. Kết quả để ở hai byte bắt đầu từ địa chỉ 2B00. Cột đầu là dòng lệnh trong hệ nhị phân, cột giữa là dòng lệnh tương đương trong hệ 16 và cột bên phải là giải thích. AX là tên một thanh ghi 16 bit trong bộ vi xử lý 8086.

Qua ví dụ trên, ta thấy ngôn ngữ máy không thật thích hợp cho số đông người sử dụng máy tính vì để viết hoặc hiểu chương trình, người ta phải nhớ rất máy móc các mã số của lệnh mà các dòng số này không có hàm ý rõ ràng. Mặt khác do tập lệnh của các bộ xử lý có thể khác nhau nên không thể dùng chương trình viết trên bộ xử lý này chạy trên máy tính dùng bộ xử lý khác loại.

1.3.3. Hợp ngữ (Assembly language)

Để khắc phục nhược điểm trên của ngôn ngữ máy, người ta đề xuất một ngôn ngữ giao tiếp với máy ở mức độ hình thức hơn gọi là hợp ngữ. Về cơ bản, hợp ngữ có các cấu trúc rất giống với ngôn ngữ máy. Điều khác là trong hợp ngữ có thể viết lệnh dưới dạng mã chữ. Mã chữ thể hiện mã lệnh hoặc các đối tượng trong lệnh (trong ngôn ngữ máy nó là mã lệnh và địa chỉ của đối tượng). Mã lệnh ở dạng chữ thường chính là những từ trong tiếng Anh có ý nghĩa rõ ràng, còn đối tượng do ta tự đặt tên phù hợp với ý niệm về đối tượng đó. Ví dụ nếu đoạn chương trình trên dùng để cộng chiều dài và chiều rộng của hình chữ nhật để tính nửa chu vi thì trong hợp ngữ ASM ta chỉ cần viết:

MOV	AX	CHIEU_DAI
ADD	AX	CHIEU_RONG
MOV	NUA_CHU_VI	AX

Lệnh thứ nhất có nghĩa là nạp số liệu mà ta đặt tên là CHIEU_DAI lên thanh ghi AX, lệnh thứ hai có nghĩa là cộng số trong thanh ghi AX với số liệu mà ta đặt tên là CHIEU_RONG. Ta thấy mặc dù còn cồng kềnh và còn phụ thuộc vào một loại máy tính cụ thể, hợp ngữ dễ dùng hơn rất nhiều so với ngôn ngữ máy.

Để một chương trình viết bằng hợp ngữ chạy được trên máy tính, nó cần phải được dịch ra ngôn ngữ máy. Mỗi hợp ngữ dùng cho một loại máy nào đó đều cần có trình dịch phù hợp. Chương trình dịch đối với hợp ngữ được gọi là hợp dịch (assembler).

1.3.4. Ngôn ngữ lập trình cấp cao (high-level programming language)

Ngôn ngữ máy và cả hợp ngữ đều phụ thuộc vào hệ thống lệnh của một loại máy tính cụ thể, chúng chưa thật thích hợp cho đông đảo người sử dụng máy tính, trong khi đó người ta muốn thể hiện thuật toán bằng những lệnh với ý nghĩa thực tế và độc lập với bất cứ loại máy tính nào. Chẳng hạn trong ví dụ trên chỉ cần viết $NUA_CHU_VI = CHIEU_DAI + CHIEU_RONG$ là đủ.

Từ đầu những năm 1950, người ta đã xây dựng những ngôn ngữ lập trình vạn năng có các lệnh gần với ngôn ngữ tự nhiên và ngôn ngữ toán học. Các ngôn ngữ lập trình này được gọi là các ngôn ngữ lập trình bậc cao (High level programming language). Mặt khác, vì chúng chỉ nhằm vào thể hiện thuật toán độc lập với các máy tính cụ thể nên người ta còn gọi nó là các ngôn ngữ thuật toán (Algorithmic language).

Cùng với sự phát triển của các thế hệ máy tính, ngôn ngữ lập trình cấp cao cũng được phát triển rất đa dạng và phong phú, việc lập trình cho máy tính vì thế mà cũng có nhiều khuynh hướng khác nhau: lập trình cấu trúc, lập trình hướng đối tượng, lập trình logic... Một chương trình viết bằng ngôn ngữ cấp cao được gọi là chương trình nguồn (source program). Cũng như

đối với hợp ngữ, để máy tính "hiểu" và thực hiện được các lệnh trong chương trình nguồn thì phải có một chương trình dịch để dịch chương trình nguồn thành dạng chương trình có khả năng thực thi trên máy tính.

Chú ý rằng mỗi lệnh của hợp ngữ nói chung được dịch thành một lệnh trong ngôn ngữ máy, còn mỗi lệnh của ngôn ngữ bậc cao thường tương đương với nhiều lệnh máy. Ví dụ lệnh `NUA_CHU_VI = CHIEU_DAI + CHIEU_RONG` sẽ dịch thành 3 lệnh máy.

➤ Lập trình cấu trúc

Phương pháp **lập trình thủ tục** (procedural programming), còn gọi là phương pháp hướng chức năng, chia một chương trình (chức năng) lớn thành các khối chức năng hay hàm (thủ tục) đủ nhỏ để dễ lập trình và kiểm tra. Mỗi hàm có một điểm bắt đầu và một điểm kết thúc và có dữ liệu và logic riêng. Trong một hệ thống chương trình, các biến có các phạm vi hoạt động nhất định. Trong chương trình, các hàm làm việc độc lập với nhau. Dữ liệu được chuyển đổi qua lại thông qua các tham số gọi hàm. Việc chia chương trình thành các hàm cho phép nhiều người có thể tham gia vào việc xây dựng chương trình, mỗi người xây dựng một hay một số các hàm độc lập với nhau. Việc chia hàm sẽ có nhiều lợi thế như: sử dụng lại mã (*reuse code*), dễ dàng chia công việc cho nhiều người, các chương trình dễ viết, dễ đọc, dễ hiểu, dễ kiểm lỗi và dễ hiệu chỉnh.

Lập trình cấu trúc là một tập con của lập trình thủ tục. Trong một chương trình máy tính, các khối chức năng có thể được thực hiện không chỉ tuần tự mà còn có thể theo các tình huống và lặp lại nhiều lần. Theo phương pháp lập trình này, một chương trình máy tính có thể được viết dựa trên ba cấu trúc: tuần tự, rẽ nhánh và vòng lặp.

➤ Lập trình logic

Đây là một phương pháp tiếp cận việc biểu diễn tri thức và giải các bài toán logic từ một cơ sở tri thức cho trước trên máy tính. Một cơ sở tri thức là một tập các sự kiện và các luật biểu diễn quan hệ logic giữa các sự kiện đó. Lập trình logic xuất phát từ một cơ sở tri thức và một câu hỏi, tiến hành các lập luận logic để tìm ra lời giải cho câu hỏi đó.

➤ Lập trình hướng đối tượng

Lập trình hướng đối tượng (Object-Oriented Programming), là phương pháp lập trình xây dựng trên mô hình thế giới thực, khi mà mọi sự vật được xem như các đối tượng. Các đối tượng sẽ chứa đựng trong nó các thuộc tính, các hành động của riêng nó. Ví dụ như mỗi nhân viên sẽ có các thuộc tính họ tên, năm sinh, giới tính và có các hành động: chấm công, tính lương tháng...

Phương pháp lập trình hướng đối tượng gắn liền với các thuật ngữ Lớp (class), Đối tượng (object), Thuộc tính (attribute), Phương thức (method). Lập trình hướng đối tượng đem lại cho người lập trình một phương pháp tư duy xây dựng bài toán ở mức gần gũi với thế giới thực tế nhất. Phương pháp lập trình hướng đối tượng ngoài ưu điểm cho phép tái sử dụng mã nguồn, nó còn có các ưu điểm như tính bảo mật cao, tính nhất quán dữ liệu.

1.3.5. Chương trình dịch

Như trên đã trình bày, muốn chuyển từ chương trình nguồn sang mã máy phải có chương trình dịch. Thông thường mỗi một ngôn ngữ cấp cao đều có một chương trình dịch riêng nhưng chung quy có hai phương pháp dịch: thông dịch và biên dịch.

- Thông dịch (interpreter): Là cách dịch từng lệnh một, dịch tới đâu thực hiện tới đó. Chẳng hạn ngôn ngữ LISP sử dụng trình thông dịch.

- Biên dịch (compiler): Dịch toàn bộ chương trình nguồn thành chương trình đích rồi sau đó mới thực hiện. Các ngôn ngữ sử dụng trình biên dịch như Pascal, C...

Giữa thông dịch và biên dịch có khác nhau ở chỗ: Do thông dịch là vừa dịch vừa thực thi chương trình còn biên dịch là dịch xong toàn bộ chương trình rồi mới thực thi nên chương trình viết bằng ngôn ngữ biên dịch thực hiện nhanh hơn chương trình viết bằng ngôn ngữ thông dịch.

Một số ngôn ngữ sử dụng kết hợp giữa thông dịch và biên dịch chẳng hạn như Java. Chương trình nguồn của Java được biên dịch tạo thành một chương trình đối tượng (một dạng mã trung gian) và khi thực hiện thì từng lệnh trong chương trình đối tượng được thông dịch thành mã máy.

Để trình bày rõ hơn cách sử dụng một ngôn ngữ lập trình nhằm giải quyết một bài toán/vấn đề cụ thể, bài giảng này chủ yếu đề cập đến ngôn ngữ lập trình C - là ngôn ngữ sử dụng phương pháp lập trình có cấu trúc - để minh họa.

BÀI TẬP CHƯƠNG 1

1. Thuật toán là gì? Trình bày các tính chất quan trọng của một thuật toán?
2. Các bước xây dựng chương trình?
3. Các cách biểu diễn thuật toán? Ưu và khuyết điểm của từng phương pháp? Cho ví dụ minh họa.

Bằng ngôn ngữ tự nhiên và lưu đồ, hãy mô tả giải thuật cho các bài toán sau:

4. Nhập năm sinh của một người. Tính tuổi người đó.
5. Nhập 2 số a và b. Tính tổng, hiệu, tích và thương của hai số đó.
6. Nhập vào 2 số nguyên. Tính min và max của hai số đó.
7. Nhập vào 3 số nguyên. Tính min và max của ba số đó.
8. Nhập tên sản phẩm, số lượng và đơn giá. Tính tiền và thuế giá trị gia tăng phải trả, biết:
 - a. tiền = số lượng * đơn giá.
 - b. thuế giá trị gia tăng = 10% tiền.
9. Nhập vào số xe N gồm 4 chữ số. Cho biết số xe được mấy nút? Sau đó phát triển bài toán với N có n chữ số bất kỳ.
10. Giải phương trình bậc 2 dạng $ax^2 + bx + c = 0$ với a, b, c là các số sẽ nhập từ bàn phím.
11. Tính tổng bình phương của n số nguyên có dạng sau: $S = a_1^2 + a_2^2 + \dots + a_n^2$, với n và a_i ($i=1\dots n$) là các số sẽ nhập từ bàn phím.
12. Tính tổng có dạng sau: $S = 1 - a_1^2 + a_2^2 - a_3^2 + \dots + (-1)^n a_n^2$, với n và a_i ($i=1..n$) là các số sẽ nhập từ bàn phím.
13. Tính $S(n) = 1^2 + 2^2 + 3^2 + \dots + n^2$ (^ là dấu mũ).
14. Cho hai số nguyên dương a và b. Tìm ước chung lớn nhất của hai giá trị này.
15. Tìm ước lẻ lớn nhất của số nguyên dương n. Ví dụ n=100 ước lẻ lớn nhất của 100 là 25.

CHƯƠNG 2

TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH C

2.1. GIỚI THIỆU NGÔN NGỮ LẬP TRÌNH C

C là ngôn ngữ lập trình cấp cao, được sử dụng rất phổ biến để lập trình hệ thống cùng với Assembler và phát triển các ứng dụng.

Vào những năm cuối thập kỷ 60 đầu thập kỷ 70 của thế kỷ XX, Dennis Ritchie (làm việc tại phòng thí nghiệm Bell) đã phát triển ngôn ngữ lập trình C dựa trên ngôn ngữ BCPL (do Martin Richards đưa ra vào năm 1967) và ngôn ngữ B (do Ken Thompson phát triển từ ngôn ngữ BCPL vào năm 1970 khi viết hệ điều hành UNIX đầu tiên trên máy PDP-7) và được cài đặt lần đầu tiên trên hệ điều hành UNIX của máy DEC PDP-11.

Năm 1978, Dennis Ritchie và B.W Kernighan đã cho xuất bản quyển “Ngôn ngữ lập trình C” và được phổ biến rộng rãi đến nay.

Lúc ban đầu, C được thiết kế nhằm lập trình trong môi trường của hệ điều hành Unix nhằm mục đích hỗ trợ cho các công việc lập trình phức tạp. Nhưng về sau, với những nhu cầu phát triển ngày một tăng của công việc lập trình, C đã vượt qua khuôn khổ của phòng thí nghiệm Bell và nhanh chóng hội nhập vào thế giới lập trình để rồi các công ty lập trình sử dụng một cách rộng rãi. Sau đó, các công ty sản xuất phần mềm lần lượt đưa ra các phiên bản hỗ trợ cho việc lập trình bằng ngôn ngữ C và chuẩn ANSI C cũng được khai sinh từ đó.

Ngôn ngữ lập trình C là một ngôn ngữ lập trình hệ thống rất mạnh và rất “mềm dẻo”, có một thư viện gồm rất nhiều các hàm (function) đã được tạo sẵn. Người lập trình có thể tận dụng các hàm này để giải quyết các bài toán mà không cần phải tạo mới. Hơn thế nữa, ngôn ngữ C hỗ trợ rất nhiều phép toán nên phù hợp cho việc giải quyết các bài toán kỹ thuật có nhiều công thức phức tạp. Ngoài ra, C cũng cho phép người lập trình tự định nghĩa thêm các kiểu dữ liệu trừu tượng khác.

Ngôn ngữ C có những đặc điểm cơ bản sau:

- o *Tính cô đọng (compact)*: C chỉ có 32 từ khóa chuẩn và 40 toán tử chuẩn, nhưng hầu hết đều được biểu diễn bằng những chuỗi ký tự ngắn gọn.

- o *Tính cấu trúc (structured)*: C có một tập hợp những chỉ thị của lập trình như cấu trúc lựa chọn, lặp... Từ đó các chương trình viết bằng C được tổ chức rõ ràng, dễ hiểu.

- o *Tính tương thích (compatible)*: C có bộ tiền xử lý và một thư viện chuẩn vô cùng phong phú nên khi chuyển từ máy tính này sang máy tính khác các chương trình viết bằng C vẫn hoàn toàn tương thích.

- o *Tính linh động (flexible)*: C là một ngôn ngữ rất uyển chuyển về cú pháp, chấp nhận nhiều cách thể hiện, có thể thu gọn kích thước của các mã lệnh làm chương trình chạy nhanh hơn.

- o *Biên dịch (compile)*: C cho phép biên dịch nhiều tập tin chương trình riêng rẽ thành các tập tin đối tượng (object) và liên kết (link) các đối tượng đó lại với nhau thành một chương trình có thể thực thi được (executable) thống nhất.

2.2. CÁC THÀNH PHẦN CƠ BẢN CỦA NGÔN NGỮ C

2.2.1. Tập ký tự

Mọi ngôn ngữ lập trình đều được xây dựng từ một bộ ký tự nào đó. Các ký tự được nhóm lại theo nhiều cách khác nhau để tạo nên các từ. Các từ lại được liên kết với nhau theo một qui tắc nào đó để tạo nên các câu lệnh. Một chương trình bao gồm nhiều câu lệnh và thể hiện một thuật toán để giải một bài toán nào đó.

Tập ký tự trong ngôn ngữ C bao gồm những ký tự, ký hiệu sau (*phân biệt chữ hoa và chữ thường*):

- 26 chữ cái hoa A, B, C...Z và 26 chữ cái thường a, b, c...z của bộ chữ cái latin.
- 10 chữ số thập phân 0, 1, 2...9.
- Các ký hiệu toán học: + - * / = ()
- Ký tự gạch nối _
- Các ký hiệu đặc biệt: : . , : ; [] { } ! \ & % # \$
- Dấu cách (hay khoảng trống) được dùng như ký tự phân cách giữa các từ trong chương trình.

2.2.2. Từ khóa

Từ khóa (keyword) là các từ dành riêng của C được định nghĩa trước với những ý nghĩa hoàn toàn xác định. Từ khóa thường được dùng để khai báo, định nghĩa các kiểu dữ liệu, định nghĩa các toán tử, các hàm và viết các câu lệnh.

Một số từ khóa thông dụng trong C:

- const, enum, signed, struct, typedef, unsigned...
- char, double, float, int, long, short, void
- case, default, else, if, switch
- do, for, while
- break, continue, goto, return

Lưu ý:

- Khi viết chương trình ta không thể dùng từ khóa để đặt tên cho hằng, biến, mảng hay hàm...
- Từ khóa trong ngôn ngữ C bao giờ cũng được viết bằng chữ thường. Ví dụ, để khai báo biến kiểu số nguyên ta không được viết INT mà phải viết int.

2.2.3. Tên/Định danh

Tên (Identifier) là một khái niệm rất quan trọng dùng để xác định các đại lượng khác nhau trong một chương trình, hay nói cách khác tên được dùng để đặt cho hằng, biến, mảng, hàm, con trỏ...

Tên được đặt theo quy tắc sau:

- Tên là một dãy các ký tự bao gồm chữ cái, số và gạch nối. Ký tự đầu tiên của tên phải là chữ cái hoặc dấu gạch nối.
- Tên không được trùng với khoá.

- Độ dài cực đại của tên theo mặc định là 32, tuy nhiên không nên đặt tên quá dài.
- C phân biệt chữ hoa và chữ thường. Thông thường, ta dùng chữ hoa để đặt tên cho các hằng và chữ thường để đặt tên cho các biến, hàm, cấu trúc.

Ví dụ:

Các tên hợp lệ:	<code>x_1</code>	<code>delta</code>	<code>chu_vi</code>	<code>GAMA</code>
Các tên không hợp lệ:	Giải thích			
	<code>3x</code>	Ký tự đầu tiên là số		
	<code>r#3</code>	Sử dụng dấu #		
	<code>f(x)</code>	Sử dụng ký tự (và)		
	<code>del ta</code>	Có khoảng cách ở giữa		
	<code>del-ta</code>	Sử dụng dấu gạch ngang '-'		
	<code>do</code>	Trùng với từ khóa		

2.2.4. Lời chú thích

Khi viết chương trình đôi lúc ta cần phải có vài lời ghi chú về một đoạn chương trình nào đó để dễ nhớ và dễ điều chỉnh sau này, nhất là phần nội dung ghi chú phải không thuộc về chương trình (khi biên dịch phần này bị bỏ qua). Trong ngôn ngữ lập trình C, nội dung chú thích phải được viết trong cặp dấu `/*` và `*/`, hoặc được đặt sau cặp dấu `//`.

Ví dụ 1: `/* Ho ten SV: Nguyen Van Binh`

`MSSV: 5512345 */`

Ví dụ 2: `// Day la chuong trinh tinh tong 2 so`

2.3. CÁC KIỂU DỮ LIỆU CƠ BẢN

2.3.1. Kiểu dữ liệu

Có thể xem những gì đưa vào cho máy tính xử lý là dữ liệu (data). Dữ liệu trong thực tế rất đa dạng, tồn tại dưới nhiều dạng khác nhau về bản chất, ý nghĩa, bao gồm các số liệu, ký tự, mệnh đề logic... Tuy nhiên, máy tính điện tử chỉ hiểu các thông tin biểu diễn dưới dạng mã nhị phân (tức là dãy các bit 0 và 1).

Ở các ngôn ngữ lập trình bậc cao như C, dữ liệu được khái quát hóa thành kiểu dữ liệu. Kiểu dữ liệu dùng để mô tả một lớp dữ liệu tương tự nhau, một kiểu dữ liệu (data type) được định nghĩa với hai đặc điểm chính là:

- Miền giá trị mà kiểu dữ liệu đó có thể nhận được.
- Các phép toán thực hiện được trên kiểu dữ liệu đó.

2.3.2. Các kiểu dữ liệu cơ sở

2.3.2.1. Kiểu ký tự

Mỗi giá trị kiểu char chiếm một byte và biểu diễn được một ký tự thông qua bảng mã ASCII.

Ví dụ:

Ký tự	Mã ASCII
0	048

1	049
A	065
B	066
A	097

Các kiểu ký tự trong C:

Kiểu dữ liệu	Kích thước	Miền giá trị
unsigned char	1 byte	Từ 0 đến 255
char	1 byte	Từ -128 đến 127

2.3.2.2. Kiểu số nguyên

Dùng để lưu các giá trị nguyên hay còn gọi là kiểu đếm được. Trong C cho phép sử dụng: số nguyên (int), số nguyên dài (long) và số nguyên không dấu (unsigned)

Các kiểu dữ liệu số nguyên:

Kiểu dữ liệu	Kích thước	Miền giá trị
int	2 byte	Từ -32,768 đến 32,767
unsigned int	2 byte	Từ 0 đến 65,535
short	2 byte	Từ -32,768 đến 32,767
long	4 byte	Từ -2,147,483,648 đến 2,147,483,647
unsigned long	4 byte	Từ 0 đến 4,294,967,295

Nhận xét: Kiểu ký tự cũng có thể xem là một dạng của kiểu nguyên.

2.3.2.3. Kiểu số thực

Dùng để lưu các số thực hay các số có dấu chấm thập phân. Các kiểu dữ liệu số thực:

Kiểu dữ liệu	Kích thước	Miền giá trị
float	4 byte	Từ $3.4 * 10^{-38}$ đến $3.4 * 10^{38}$
double	8 byte	Từ $1.7 * 10^{-308}$ đến $1.7 * 10^{308}$
long double	10 byte	Từ $3.4 * 10^{-4932}$ đến $1.1 * 10^{4932}$

2.3.2.4. Kiểu void

C có một kiểu dữ liệu đặc biệt gọi là void. Kiểu dữ liệu này chỉ cho trình biên dịch C biết rằng không có dữ liệu của bất cứ kiểu nào. Trong C, các hàm số thường trả về dữ liệu thuộc một kiểu nào đó. Tuy nhiên, khi một hàm không có gì để trả về, kiểu dữ liệu void được sử dụng để chỉ ra điều này.

2.3.3. Hằng

Hằng là các đại lượng mà giá trị của nó không thay đổi trong suốt quá trình thực thi của chương trình. Hằng có thể là một chuỗi ký tự, một ký tự, một con số xác định. Chúng có thể được biểu diễn hay định dạng với nhiều dạng thức khác nhau.

Đề đặt tên cho hằng ta dùng lệnh sau:

```
#define tên_hằng giá_trị
```

Ví dụ: #define MAX 1000

Lúc này tất cả các tên MAX trong chương trình xuất hiện sau này đều có giá trị 1000.

Hằng cũng có thể được khai báo theo cú pháp sau:

```
const Kiểu_dữ_liệu Tên_hằng = giá_trị;
```

Ví dụ:

```
const int MAX = 1000;
const char NEWLINE = '\n';
const char a[20] = "Hello World";
```

2.3.3.1. Hằng số thực (float và double)

Số thực được thể hiện theo 2 cách sau:

- **Cách 1 (dạng thập phân):** số thực gồm: phần nguyên, dấu chấm thập phân (.) và phần thập phân.

Ví dụ: 123.34 -223.333 3.00 -56.0

☞ **Lưu ý:** phần nguyên hay phần phân có thể vắng mặt nhưng dấu chấm không thể thiếu, ví dụ: .34 .15

- **Cách 2 (dạng mũ):** Một số thực được tách làm 2 phần, cách nhau bằng ký tự e hay E:
 + *Phần giá trị:* là một số nguyên hay số thực dạng thập phân.
 + *Phần mũ:* là một số nguyên

Giá trị của số thực là: Phần giá trị nhân với 10 mũ phần mũ.

Ví dụ: 1234.56e-3 = 1.23456 (là số 1234.56 * 10⁻³)
 -123.45E4 = -1234500 (là số -123.45 * 10⁴)

2.3.3.2. Hằng số nguyên (int)

Là đại lượng có giá trị từ -32768 đến +32767.

Ví dụ: 123 -45

Có thể biểu diễn một hằng nguyên dưới dạng bát phân (hệ 8) hay thập lục phân (hệ 16):

- **Hằng nguyên hệ 8:** được viết theo cách: 0c1c2c3...

Trong đó ci là một số nguyên từ 0 đến 7. Hằng nguyên hệ 8 luôn nhận giá trị dương.

Ví dụ: 0345 là một hằng nguyên hệ 8. Giá trị của nó trong hệ 10 là : 3*8*8+4*8+5=229

- **Hằng nguyên hệ 16:** có dạng : 0xc1c2c3... hoặc 0Xc1c2c3...

Trong đó ci là một chữ số hệ 16.

Ví dụ: 0xa9, 0Xa9, 0xA9, 0XA9 là 4 hằng nguyên hệ 16 như nhau. Giá trị của chúng trong hệ 10 là: 10*16+9=169

2.3.3.3. Hằng long

Giống như hằng nguyên, nhưng khác ở chỗ có thêm chữ *L* hoặc *l* ở đằng sau. Ví dụ: 198l, 123L

Một số nguyên vượt ra ngoài miền xác định của int cũng được xem là hằng long. Ví dụ: 456395L và 456395 là hai hằng long có cùng giá trị.

2.3.3.4. Hằng ký tự

Hằng ký tự là một ký tự riêng biệt được viết trong cặp dấu nháy đơn ('). Giá trị của hằng ký tự chính là mã ASCII của ký tự đó.

Ví dụ: Ký tự 'A' có giá trị là 65

Hằng ký tự về thực chất có thể coi nó như một giá trị nguyên. Do đó nó có thể tham gia vào các phép toán số học như mọi số nguyên khác.

Ví dụ: '9' - '0' = 57 - 48 = 9

Hằng ký tự còn có thể được viết là '\x1x2x3', trong đó x1x2x3 là một số của hệ đếm cơ số 8 mà giá trị của nó bằng mã ASCII của ký tự cần biểu diễn.

Ví dụ: chữ 'a' có hệ mã 10 là 97, đổi ra hệ bát phân là 0141. Vậy hằng ký tự 'a' có thể viết dưới dạng '\0141'

Có một số hằng ký tự đặc biệt được viết theo quy ước trong bảng sau:

Cách viết	Ký tự	Diễn giải
'\ '	'	Dấu nháy đơn
'\,'	,	Dấu phẩy
'\"'	''	Dấu nháy kép
'\\'	\	Dấu gạch chéo ngược
'\n'	\n	Ký tự xuống dòng
'\0'	\0	Ký tự NULL
'\t'	Tab	Ký tự Tab
'\b'	Backspace	Ký tự Backspace
'\r'	CR	Ký tự trở về đầu dòng
'\f'	LF	Ký tự sang trang

2.3.3.5. Hằng chuỗi

Hằng chuỗi ký tự là một chuỗi ký tự được đặt trong cặp dấu nháy kép ("").

Ví dụ: "Nhập môn lập trình", "Khoa CNTT"

Chuỗi được lưu trữ trong máy dưới dạng một mảng có các phần tử là các ký tự riêng biệt. Trình biên dịch sẽ tự động thêm ký tự NULL (ký tự '\0') vào cuối mỗi chuỗi để đánh dấu sự kết thúc của một chuỗi.

Lưu ý:

- Một chuỗi không có nội dung "" được gọi là chuỗi rỗng.

- Cần phân biệt ký tự 'a' và chuỗi "a". 'a' là hằng ký tự được lưu trữ trong một byte còn "a" là một chuỗi ký tự được lưu trữ trong một mảng gồm hai phần tử là 'a' và '\0'
- Để biểu diễn ký tự đặc biệt bên trong chuỗi ta phải thêm dấu \ phía trước.

Ví dụ: "I'm a student" cần được viết là "I\'m a student"

2.3.4. Biến

2.3.4.1. Khai báo biến

Biến là một đại lượng được người lập trình định nghĩa và được đặt tên thông qua việc khai báo biến. Biến dùng để chứa dữ liệu trong quá trình thực hiện chương trình và giá trị của biến có thể bị thay đổi trong quá trình này.

Mỗi biến thuộc về một kiểu dữ liệu xác định và có giá trị thuộc kiểu đó.

Cú pháp khai báo biến:

Kiểu_dữ_liệu Danh_sách_tên_biến;

Trong đó:

- + *Kiểu dữ liệu*: là một trong các kiểu dữ liệu cho phép trong C.
- + *Danh sách tên biến*: gồm các tên biến có cùng kiểu dữ liệu, mỗi tên biến cách nhau bởi dấu phẩy (,).

Ví dụ:

```
int    a, b, c;           /*Ba biến a, b, c có kiểu int*/
long   chu_vi;           /*Biến chu_vi có kiểu long*/
float   nua_chu_vi;      /*Biến nua_chu_vi có kiểu float*/
double dien_tich;       /*Biến dien_tich có kiểu double*/
```

2.3.4.2. Gán giá trị cho biến

Có thể kết hợp việc khai báo với toán tử gán để biến nhận ngay giá trị cùng lúc với khai báo.

Cú pháp:

Kiểu_dữ_liệu tên_biến = giá_trị;

Ví dụ:

▪ **Khai báo trước, gán giá trị sau:**

```
int main()
{   int a, b, c;
    a = 1;
    b = 2;
    ...
}
```

▪ **Vừa khai báo vừa gán giá trị:**

```
int main()
{   int a = 1, b = 2, c = 5;
    ...
}
```

}

2.4. BIỂU THỨC VÀ PHÉP TOÁN

2.4.1. Biểu thức

Biểu thức là một sự kết hợp giữa các toán tử (operator) và các toán hạng (operand) để diễn đạt một công thức toán học nào đó.

Mỗi toán hạng có thể là một hằng, một biến hoặc một biểu thức khác. Toán tử là các thao tác xử lý trên các toán hạng theo các quy tắc nhất định để đạt được mục đích mong muốn.

Trong trường hợp biểu thức có nhiều toán tử, ta dùng cặp dấu ngoặc đơn () để chỉ định toán tử nào được thực hiện trước.

Biểu thức thường được dùng trong các trường hợp sau:

- Ở vế phải của câu lệnh gán.
- Làm tham số thực của hàm.
- Trong các toán tử if, for, while, do while.

Ví dụ: `b = 1 + 5 * 2 / i;`

`a = 6 % (7 + 1);`

`x++ * 2 / 4 + 5 - power(i, 2)`

2.4.2. Phép toán

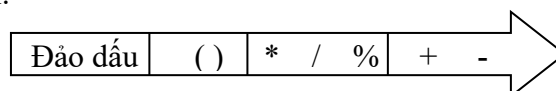
Trong C có các nhóm phép toán (toán tử) chủ yếu sau:

2.4.2.1. Phép toán số học

Toán tử	Ý nghĩa	Ví dụ	Ghi chú
+	Cộng	$100 + 25 = 125$	Áp dụng trên tất cả các toán hạng có kiểu dữ liệu <i>char</i> , <i>int</i> , <i>float</i> , <i>double</i> (kể cả <i>long</i> , <i>short</i> , <i>unsigned</i>)
-	Trừ	$100 - 25 = 75$	
*	Nhân	$11 * 3 = 33$	
/	Chia	$11 / 3 = 3$ $15.6 / 3 = 5.2$	
%	Chia lấy phần dư	$10 \% 4 = 2$	Áp dụng trên các toán hạng có kiểu dữ liệu <i>char</i> , <i>int</i> , <i>long</i>
-	Phép đảo dấu	$-(a + b)$	

✓ Các phép toán số học được dùng để viết các công thức toán học.

✓ Thứ tự ưu tiên:



2.4.2.2. Phép toán quan hệ

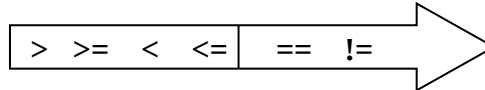
Phép toán quan hệ được dùng để so sánh 2 biểu thức với nhau, bao gồm các phép toán sau:

Toán tử	Ý nghĩa	Ví dụ
>	Lớn hơn	$4 > 10$ có giá trị 0 (sai)

$\geq$	Lớn hơn hoặc bằng	$4 \geq 4$ có giá trị 1 (đúng)
$<$	Nhỏ hơn	$3 < 6$ có giá trị 1 (đúng)
$\leq$	Nhỏ hơn hoặc bằng	$10 \leq 9$ có giá trị 0 (sai)
$==$	Bằng	$3 == 5$ có giá trị 0 (sai)
$!=$	Khác	$6 != 4$ có giá trị 1 (đúng)

Kết quả của phép toán quan hệ bằng 1 nếu đúng, bằng 0 nếu sai. Phép toán quan hệ ngoài toán hạng được sử dụng là số còn được sử dụng với kiểu dữ liệu char.

✓ **Thứ tự ưu tiên:**



✓ Các phép toán quan hệ có độ ưu tiên thấp hơn các phép toán số học.

Ví dụ: $6 - 3 < 4 \Leftrightarrow (6 - 3) < 4 \rightarrow$ có giá trị 1 (đúng)
 $-2 * -4 < 3 + 2 \Leftrightarrow (-2 * -4) < (3 + 2) \rightarrow$ có giá trị 0 (sai)
 $s1 = (1 == 2); \rightarrow s1=0;$

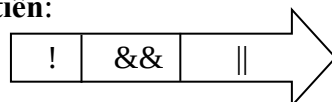
2.4.2.3. Phép toán luận lý

Phép toán	Ký hiệu
NOT (Phủ định)	!
AND (và)	&&
OR (hoặc)	

Hai toán hạng của phép toán này phải có kiểu logic, nghĩa là chỉ nhận một trong hai giá trị: đúng (được thể hiện bởi các số nguyên khác 0) hoặc sai (thể hiện bởi 0). Khi đó giá trị trả về của phép toán là 1 hoặc 0 và được cho trong bảng sau:

Toán hạng a	Toán hạng b	!a	a && b	a b
Khác 0	Khác 0	0	1	1
Khác 0	0	0	0	1
0	Khác 0	1	0	1
0	0	1	0	0

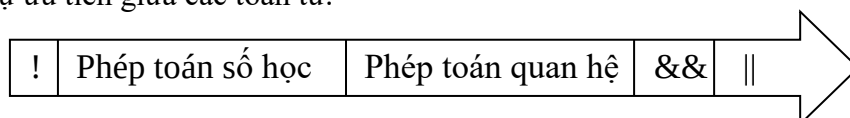
✓ **Thứ tự ưu tiên:**



Ví dụ: $!(2 \leq 1) \rightarrow$ có giá trị 1 (đúng)
 $(1 > 2) \&\& (3 < 4) \rightarrow$ có giá trị 0 (sai)
 $(1 > 2) || (3 < 4) \rightarrow$ có giá trị 1 (đúng)

✓ Các phép toán quan hệ và luận lý được dùng để thiết lập điều kiện rẽ nhánh trong câu lệnh if và điều kiện kết thúc chu trình trong câu lệnh lặp.

✓ Thứ tự ưu tiên giữa các toán tử:



Ví dụ: $2 + 6 \ \&\& \ 8 \Leftrightarrow (2 + 6) \ \&\& \ 8$

→ có giá trị 1 (đúng)

2.4.2.4. Phép toán trên bit

Toán tử trên bit tác động lên các bit của toán hạng, gồm các phép toán sau:

Phép toán	Ký hiệu
AND (Và)	&
OR (Hoặc)	
XOR (Hoặc loại trừ)	^
Dịch trái	<<
Dịch phải	>>
NOT (Phủ định)	~

Ý nghĩa của chúng được cho trong bảng sau:

Bit a	Bit b	~a	a & b	a b	a ^ b
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Ví dụ: $a \ll n \rightarrow a * (2^n)$

$a \gg n \rightarrow a / (2^n)$

$a = 13 \rightarrow$ đổi ra hệ nhị phân $\rightarrow 1101$

$b = 10 \rightarrow$ đổi ra hệ nhị phân $\rightarrow 1010$

1101	1101	1101	
& 1010	 1010	^ 1010	
= 1000	= 1111	= 0111	
= 8	= 15	= 7	(dạng thập phân)

2.4.2.5. Phép toán tăng, giảm

Phép toán tăng (giảm) là các phép toán một ngôi trong ngôn ngữ lập trình C dùng để tăng (giảm) các biến (nguyên hoặc thực) lên (xuống) 1 giá trị, bao gồm các phép toán sau (giả sử i là biến):

Tên phép toán	Biểu diễn	Chức năng
Phép tăng trước	++i	i được tăng lên 1 trước, sau đó sẽ lấy kết quả để thực hiện biểu thức
Phép tăng sau	i++	i được đưa vào thực hiện biểu thức trước, sau đó mới tăng i lên 1
Phép giảm trước	--i	i được giảm đi 1 trước, sau đó sẽ lấy kết quả để thực hiện biểu thức

Phép giảm sau	i--	i được đưa vào thực hiện biểu thức trước, sau đó mới giảm i đi 1
---------------	-----	--

Ví dụ: Xét hai đoạn chương trình sau:

```
int i=10, x;
x=i++;
⇒ x có giá trị là 10
⇒ i có giá trị là 11
```

```
int i=10, x;
x=++i;
⇒ x có giá trị là 11
⇒ i có giá trị là 11
```

- Phép toán tăng, giảm thường được sử dụng trong các toán tử for, while để tăng hay giảm giá trị của biến điều khiển.

- Không nên sử dụng các phép tăng giảm quá tùy tiện trong các biểu thức vì có thể sẽ dẫn đến các kết quả sai không mong muốn.

2.4.2.6. Toán tử điều kiện

Toán tử điều kiện yêu cầu 3 toán hạng, dạng tổng quát như sau:

toán hạng 1 ? toán hạng 2: toán hạng 3

Toán hạng 1 được ước lượng và được xem như là một điều kiện. Nếu kết quả không là 0 thì toán hạng 2 được ước lượng và giá trị của nó là kết quả sau cùng. Ngược lại, toán hạng 3 được ước lượng và giá trị của nó là kết quả sau cùng.

Ví dụ: `int m = 1, n = 2;`
`int min = (m < n ? m : n); // min nhận giá trị 1`

☞ **Lưu ý:**

- Trong các toán hạng thứ 2 và toán hạng thứ 3 của toán tử điều kiện thì chỉ có một toán hạng được thực hiện. Điều này là quan trọng khi một hoặc cả hai chứa hiệu ứng phụ (nghĩa là, việc ước lượng của chúng làm chuyển đổi giá trị của biến). Ví dụ, với `m = 1` và `n = 2` thì trong `int min = (m < n ? m++ : n++);` `m` được tăng lên bởi vì `m++` được ước lượng nhưng `n` không tăng vì `n++` không được ước lượng.

- Vì phép toán điều kiện cũng là một biểu thức nên nó có thể được sử dụng như một toán hạng của phép toán điều kiện khác, có nghĩa là các biểu thức điều kiện có thể được lồng nhau.

Ví dụ: `int m = 1, n = 2, p = 3;`
`int min = (m < n ? (m < p ? m : p) : (n < p ? n : p)); //min nhận giá trị 1`

2.4.2.7. Toán tử phẩy

Nhiều biểu thức có thể được kết nối vào cùng một biểu thức sử dụng toán tử phẩy. Toán tử phẩy yêu cầu 2 toán hạng. Đầu tiên nó ước lượng toán hạng trái sau đó là toán hạng phải, và trả về giá trị của toán hạng phải như là kết quả sau cùng.

Ví dụ: `int m, t;`
`m = (t = 2, t*5 + 10);`

Khi đó `t = 2` và `m = 2*5+10 = 20`.

2.4.3. Thứ tự ưu tiên của các phép toán

Thứ tự ưu tiên của các phép toán là một vấn đề hết sức quan trọng, trong biểu thức bao giờ các phép toán có độ ưu tiên cao hơn cũng được thực hiện trước và ngược lại. Thứ tự ưu tiên của các phép toán trong ngôn ngữ C được tổng kết trong bảng sau:

Độ ưu tiên	Các toán tử	Thứ tự kết hợp
1	() [] -> .	Trái sang phải
2	! ~ & * - ++ -- (type) sizeof	Phải sang trái
3	* (phép nhân) / %	Trái sang phải
4	+ - (phép trừ)	Trái sang phải
5	<< >>	Trái sang phải
6	< <= > >=	Trái sang phải
7	= = !=	Trái sang phải
8	& (phép thao tác trên bit AND)	Trái sang phải
9	^	Trái sang phải
10		Trái sang phải
11	&& (phép toán logic AND)	Trái sang phải
12		Trái sang phải
13	? :	Phải sang trái
14	= += -= *= /= %= <<= >>= &= ^= =	Phải sang trái
15	,	Trái sang phải

Giải thích:

Dòng	Toán tử	Diễn giải
1	[]	Dùng để biểu diễn phần tử mảng, ví dụ: a[i]
	.	Dùng để biểu diễn thành phần của cấu trúc, ví dụ: ht.ten
	->	Để biểu diễn thành phần cấu trúc thông qua con trỏ, ví dụ: ht->ten
2	*	Dùng để khai báo con trỏ, ví dụ: int *a;
	&	Phép toán lấy địa chỉ, ví dụ: &x
	type	Phép ép kiểu, ví dụ: (float) x+y

- Các phép toán trên cùng một hàng có cùng thứ tự ưu tiên, các phép toán ở hàng trên có số ưu tiên cao hơn các phép toán ở hàng dưới.
- Đối với các phép toán có cùng mức ưu tiên thì trình tự tính toán có thể từ trái qua phải hay ngược lại.
- Nếu không nhớ thứ tự ưu tiên của các phép toán thì ta có thể dùng dấu ngoặc đơn () để viết các biểu thức. Cặp ngoặc đơn được thực hiện theo thứ tự ưu tiên từ trong ra ngoài.

Ví dụ: $3 * 5 + 4 = 19$

$6 + 2 / 2 - 3 = 4$

$$-7 + 2 * ((4 + 3) * 4 + 8) = 65$$

$$6 - 3 < 4 \rightarrow \text{có giá trị } 1 \text{ (đúng), tương đương } (6 - 3) < 4$$

$$-2 * -4 < 3 + 2 \rightarrow \text{có giá trị } 0 \text{ (sai), tương đương } (-2 * -4) < (3 + 2)$$

2.4.4. Chuyển đổi kiểu giá trị

Việc chuyển đổi kiểu giá trị xảy ra tự động trong hai trường hợp sau:

- Khi trong biểu thức có các toán hạng khác kiểu.
- Khi gán giá trị kiểu này cho biến kiểu kia.

Ngoài ra ta có thể chuyển từ một kiểu giá trị sang một kiểu bất kỳ mà ta muốn bằng phép ép kiểu: **(type) biểu thức**

Ví dụ: **(int) (a)** sẽ cho một giá trị kiểu int. Nếu a là biến kiểu float thì ở đây sẽ có sự chuyển đổi từ float sang int. Tuy nhiên, bản thân kiểu của a vẫn không bị thay đổi: a vẫn có kiểu float nhưng **(int)a** có kiểu int.

☞ **Lưu ý:**

- Muốn có giá trị chính xác trong phép chia 2 biến nguyên cần dùng phép ép kiểu:

(float)a / b

- Để đổi giá trị thực r sang số nguyên, nên dùng: **(int)(r+0.5)**
- Chú ý đến thứ tự ưu tiên:

$$(int) 1.4 * 10 = 1 * 10 = 10$$

$$(int) (1.4 * 10) = (int) 14.0 = 14$$

- Đối với các hàm toán học của thư viện chuẩn thì giá trị của đối số và của hàm đều có kiểu double, vì vậy để tính căn bậc hai của một biến nguyên n, ta phải dùng phép ép kiểu int sang kiểu double: **sqrt ((double) n)**.

✓ **Chuyển đổi kiểu trong biểu thức:** Khi hai toán hạng trong một biểu thức có kiểu khác nhau thì kiểu thấp hơn sẽ được nâng thành kiểu cao hơn trước khi thực hiện phép toán. Kết quả thu được là một giá trị có kiểu cao hơn:

$$\text{char} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$$

Khi chuyển đổi theo chiều ngược lại sẽ mất mát thông tin, có thể có cảnh báo nhưng không có lỗi.

$$\text{Ví dụ: } 1.5 * (11/3) = 4.5$$

$$1.5 * 11/3 = 5.5$$

$$(11/3) * 1.5 = 4.5$$

2.5. CÂU LỆNH

2.5.1. Khái niệm câu lệnh

Câu lệnh (Statement) là đơn vị nhỏ nhất của một chương trình máy tính, nghĩa là tất cả các chương trình đều phải được xây dựng từ tập hợp các câu lệnh theo một thứ tự logic nào đấy. Một câu lệnh xác định một công việc mà chương trình phải thực hiện để xử lý dữ liệu đã được

mô tả và khai báo. Trong C, câu lệnh được kết thúc bằng dấu chấm phẩy (;) và trình biên dịch bỏ qua các khoảng trắng, tab hoặc dấu xuống dòng (ký tự '\n') chen giữa lệnh.

Ví dụ: Các cách viết cho câu lệnh gán $a=123$; như sau là tương đương nhau:

```
a = 123;
⇔ a = 123;
⇔ a
   =
   123;
```

Quy tắc viết câu lệnh: Giữa các từ trong một câu lệnh có thể đặt một hoặc một số khoảng trắng, tab hoặc dấu xuống dòng (ký tự '\n'), nhưng không được viết một từ trên nhiều dòng hoặc làm gián đoạn một từ bằng khoảng trắng.

Phân loại câu lệnh: Có hai loại câu lệnh: lệnh đơn và lệnh phức (khối lệnh).

- *Lệnh đơn:* là một lệnh không chứa các lệnh khác. Các lệnh đơn gồm: lệnh gán, các câu lệnh nhập xuất dữ liệu...

Ví dụ: $a=2$;

- *Khối lệnh:* là một dãy các câu lệnh nằm trong khối bao bởi dấu '{' và '}'.

Cú pháp: { **Câu lệnh 1;**
 Câu lệnh 2;
 ...
 Câu lệnh n;
 }

Ví dụ: { $a=2$;
 $b=3$;
 $\text{printf}(\text{"\n\%6d\%6d", a, b});$
 }

C xem một khối lệnh cũng như một câu lệnh riêng lẻ. Nói cách khác, những nơi có thể đặt lệnh đơn thì cũng có thể đặt khối lệnh. Bên trong một khối lệnh có thể chứa các khối lệnh khác.

2.5.2. Lệnh gán

Phép gán là phép dùng để đưa giá trị của một biểu thức nào đó vào trong một biến tương thích theo cú pháp sau:

biến = Biểu_thức;

Trước tiên, máy sẽ tiến hành tính giá trị của biểu thức, sau đó mới gán giá trị tính được cho biến.

Ví dụ: $\text{float } a, b=3.5$;

$$a = b * (b - 1.5) + 10;$$

Kết quả tính toán của biểu thức $b*(b-1.5) + 10$ sẽ được gán cho biến a ($a=17$).

☞ **Lưu ý:**

- Khi một biểu thức được gán cho một biến thì giá trị của biến sẽ thay thế giá trị cũ mà biến đã lưu giữ trước đó.
- Kiểu của biểu thức ở vế phải của phép gán phải tương thích với kiểu của biến. Nếu phép gán không tương thích thì chương trình sẽ phát sinh một lỗi. Chẳng hạn ví dụ sau cho thấy một sự không tương thích về kiểu:

Ví dụ: `int main()`

```
{    int x, y;
    x = 10;           /*Gán hằng số 10 cho biến x*/
    y = "Xin chao";   /*y có kiểu int, còn "Xin chao" có kiểu char* */
}
```

Khi biên dịch chương trình này, C sẽ báo lỗi "*invalid conversion from 'constant char *' to 'int'*", nghĩa là C không thể tự động chuyển đổi kiểu từ chuỗi ký tự sang số nguyên `int`.

- Cần phân biệt phép gán (`=`) và phép so sánh (`==`). Phép gán vừa gán giá trị cho biến vừa trả lại giá trị bất kỳ (là giá trị của toán hạng bên phải), trong khi phép so sánh luôn trả lại giá trị 1 hoặc 0.

➤ **Chuyển đổi kiểu trong lệnh gán**

Trong biểu thức số học, C tự động hiệu chỉnh kiểu `char` và kiểu `short` thành kiểu `int`.

Ví dụ:

```
char ch;
ch = 'A' + 1;           // 'A'=65 nên 'A'+1=66 do đó ch='B'
```

Trong nhiều trường hợp để tạo ra sự tương thích về kiểu, ta phải sử dụng đến cách thức chuyển đổi kiểu một cách tường minh. Cú pháp của phép toán này như sau:

(Tên kiểu) Biểu_thức

Sẽ chuyển đổi kiểu của `Biểu_thức` thành kiểu mới `Tên_kiểu`.

Ví dụ:

```
float x, f;
x=10/4;           /* x = 2.0*/
f = (float) 10 / 4;   /* f= 2.5*/
```

- Trong câu lệnh gán, dấu `=` là một toán tử; do đó nó có thể được sử dụng là một thành phần của biểu thức. Trong trường hợp này giá trị của biểu thức gán chính là giá trị của biến.

Ví dụ:

```
int x, y, z;
z = y = x = 3;
```

Các biến `x`, `y`, `z` đều có giá trị là 3, và câu lệnh trên tương đương với các lệnh sau:

```
x = 3;
y = x;
z = y;
```

- Đối với một số phép gán đặc biệt mà biến được gán có mặt cả trong hai vế của biểu thức gán, các phép gán đó là: +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=.

Ví dụ: Ta có giá trị $i = 3$

```
i = i + 3;           → i = 6
i += 3;             → i = 6 (i = i + 3)
i *= 3;             → i = 9 (i = i * 3)
```

2.5.3. Các lệnh nhập xuất dữ liệu

2.5.3.1. Hàm printf

Hàm printf (nằm trong thư viện **stdio.h**) dùng để xuất giá trị của các biểu thức lên màn hình.

Cú pháp: **printf ("chuỗi định dạng", đối mục 1, đối mục 2,...);**

Trong đó:

- *đối mục 1, ...*: là các mục dữ kiện cần in ra màn hình. Các đối mục này có thể là biến, hằng hoặc biểu thức phải được định trị trước khi in ra.
- *Chuỗi định dạng*: được đặt trong cặp nháy kép (" "), dùng để qui định kiểu dữ liệu, cách biểu diễn, độ rộng, số chữ số thập phân... Một số định dạng đối với số nguyên, số thực, ký tự:

Định dạng	Ý nghĩa
%d	Xuất số nguyên thập phân có dấu
%[.số chữ số thập phân] f	Xuất số thực có <số chữ số thập phân> theo quy tắc làm tròn số.
%o	Xuất số nguyên hệ bát phân không dấu
%x	Xuất số nguyên hệ thập lục phân không dấu
%c	Xuất một ký tự
%s	Xuất chuỗi ký tự
%e (hoặc %E hoặc %g hoặc %G)	Xuất số nguyên dạng khoa học (nhân 10 mũ x)
L	Tiền tố dùng kèm với %d, %u, %x, %o để chỉ số nguyên dài (ví dụ %ld)

Ví dụ:

```
%d      In ra số nguyên
%4d     In số nguyên tối đa 4 ký số, nếu số cần in nhiều hơn 4 ký số thì in hết
%f      In số thực
%6f     In số thực tối đa 6 ký số (tính luôn dấu chấm), nếu số cần in nhiều hơn 6 ký số thì in hết
%.3f    In số thực có 3 số lẻ, nếu số cần in có nhiều hơn 3 số lẻ thì làm tròn
```

Các ký tự điều khiển và ký tự đặc biệt:

Ký tự	Ý nghĩa
\n	Nhảy xuống dòng kế tiếp canh về cột đầu tiên
\t	Canh cột tab ngang
\r	Nhảy về đầu hàng, không xuống hàng
\a	Tiếng kêu bip
\\	In ra dấu \
\"	In ra dấu "
%%	In ra dấu %

Ví dụ 1: giả sử biến a có giá trị = 7 và b có giá trị = 4

```
printf("Tong cua 2 so %5d va %3d la %1d . \n", a, b, a+b);
```

Kết quả in ra màn hình:

Tong cua 2 so	7	va	4	la	11.
—					
					2 kí tự (mặc dù định dạng là 1)
					3 kí tự
					5 kí tự

Ví dụ 2: giả sử biến a có giá trị = 7 và b có giá trị = 4

```
printf("Tong cua 2 so %02d va %02d la %04d . \n", a, b, a+b);
```

Kết quả in ra màn hình:

Tong cua 2 so	07	va	04	la	0011.
—					
					thêm 2 số 0 trước -> đủ 4 kí tự
					thêm 1 số 0 trước -> đủ 2 kí tự
					thêm 1 số 0 trước -> đủ 2 kí tự

Ví dụ 3: giả sử float a = 6.4, b = 1234.56, c = 62.3

```
printf("%7.2d%7.2d%7.2d.\n", a, b, c);
```

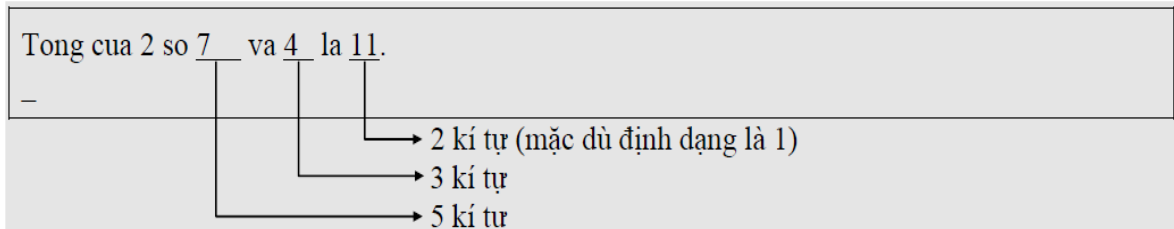
6.40	1234.56	62.30	Số canh về bên phải bề rộng trường.
—			
			7 kí tự

Dấu trừ trước bề rộng trường sẽ hiển thị kết quả về bên trái

Ví dụ 4: giả sử biến a có giá trị = 7 và b có giá trị = 4

```
printf("Tong cua 2 so %-5d va %-3d la %-1d . \n", a, b, a+b);
```

Kết quả in ra màn hình:



2.5.3.2. Hàm scanf

Hàm scanf (nằm trong thư viện **stdio.h**) cho phép đọc dữ liệu từ bàn phím và gán cho các biến trong chương trình khi chương trình thực thi.

Cú pháp: **scanf ("chuỗi định dạng", địa chỉ của các biến);**

Trong đó:

- **Chuỗi định dạng:** được đặt trong cặp nháy kép (" ") dùng để qui định kiểu dữ liệu, cách biểu diễn, độ rộng, số chữ số thập phân... Một số định dạng khi nhập kiểu số nguyên, số thực, ký tự:

Định dạng	Ý nghĩa
%[số ký số]d	Nhập số nguyên có tối đa <số ký số>
%[số ký số]f	Nhập số thực có tối đa <số ký số> tính cả dấu chấm
%c	Nhập một ký tự

- **Địa chỉ của các biến:** là địa chỉ của các biến mà chúng ta cần nhập giá trị cho nó, được viết như sau: **&tên_biến**.

Ví dụ:

```
scanf ("%d%d", &a, &b);
/*Nhập vào 2 số nguyên a, b phải cách nhau bằng khoảng trắng hoặc enter*/
scanf ("%f", &x);          /*Doc gia tri cho bien x co kieu thuc*/
```

☞ **Lưu ý:**

- Các địa chỉ của biến phải cách nhau bằng dấu phẩy (,).
- Có bao nhiêu biến thì phải có bấy nhiêu định dạng.
- Thứ tự của các định dạng phải phù hợp với thứ tự của các biến.
- Để nhập giá trị kiểu char được chính xác, nên dùng hàm **fflush(stdin)** để loại bỏ các ký tự còn nằm trong vùng đệm bàn phím trước câu lệnh nhập.

Ví dụ: `#include<stdio.h>`
`#include<conio.h>`
`int main()`
`{` `char ht[25], gt;`
`int t;`

```

printf("\nTuoi");
scanf("%d",&t);
fflush(stdin); /*xóa stdin*/
printf("\nGT(Y/N -Y:Nam, N:Nu):");
scanf("%c",&gt);
fflush(stdin); /*xóa stdin*/
printf("Ho ten:");
gets(ht);
printf("\nTuoi=%d\tHoten=%s\tGioi tinh=%c",t,ht,gt);
getch();
return 0;
}

```

2.5.3.3. Một số lệnh nhập xuất khác

a. Các hàm dùng để trình bày dữ liệu ra màn hình

✓ Hàm putchar()

- Được khai báo trong thư viện chuẩn **stdio.h**
- Dùng để đưa một ký tự ra màn hình tại vị trí hiện tại của con trỏ.
- Cú pháp: **putchar(c);**

Ví dụ:

```

int c;

c = getchar();

putchar(c);

```

✓ Hàm puts()

- Được khai báo trong thư viện chuẩn **stdio.h**
- Dùng để đưa một chuỗi ký tự s và đưa thêm ký tự ‘\n’ ra màn hình. Nếu thành công, hàm trả về ký tự cuối cùng được xuất (chính là ‘\n’), khi có lỗi, hàm trả về EOF.
- Cú pháp: **puts(s);**

Ví dụ: Câu lệnh `puts("Ha Noi");` sẽ đưa dòng chữ *Ha Noi* lên một dòng mới sau đó chuyển con trỏ xuống đầu dòng tiếp theo.

b. Các hàm dùng để nhập dữ liệu từ bàn phím

✓ Hàm getchar()

- Được khai báo trong thư viện chuẩn **stdio.h**
- Nhận một ký tự từ bàn phím và không đưa ra màn hình. Hàm sẽ trả về ký tự nhận được và lưu vào biến.
- Cú pháp: `biến = getchar();`

Ví dụ: để nhập một ký tự từ bàn phím vào biến c ta có thể viết như sau:

```
char c;

c=getchar();
```

Nếu nhấn phím A ↵ (Enter), thì c= 'A', ký tự '\n' vẫn còn lại trong stdin (vùng đệm bàn phím) và nó sẽ làm trôi hàm getchar hoặc gets sau đó. Còn nếu ta chỉ nhấn phím Enter thì c='\n' và '\n' bị loại khỏi stdin.

✓ Hàm getch()

- Được khai báo trong thư viện chuẩn **conio.h**
- Cú pháp: **getch();**
- Nếu có sẵn ký tự trong bộ đệm bàn phím thì hàm sẽ nhận một ký tự trong đó. Nếu bộ đệm rỗng, máy sẽ tạm dừng. Khi gõ một ký tự thì hàm nhận ngay ký tự đó (không cần bấm thêm phím Enter như trong các hàm nhập khác). Ký tự vừa gõ không hiện lên màn hình.

Nếu dùng: *biến=getch();* thì biến sẽ chứa ký tự đọc vào.

Ví dụ: *c = getch();*

✓ Hàm getche()

- Được khai báo trong thư viện chuẩn **conio.h**
- Cú pháp: **getche();**
- Hàm này công dụng như hàm getch, chỉ có điểm khác là cho hiện ký tự được gõ lên màn hình.

✓ Hàm gets()

- Được khai báo trong thư viện chuẩn **stdio.h**
- Cú pháp: **gets(s);**
- Nhận một chuỗi từ bàn phím cho đến khi gặp ký tự '\n'. Ký tự '\n' bị loại khỏi stdin nhưng không được đặt vào chuỗi. Chuỗi được bổ sung thêm ký tự kết thúc '\0' và đặt vào vùng nhớ do s trỏ tới.
- Hàm trả về địa chỉ của chuỗi nhận được.

2.6. CẤU TRÚC CỦA MỘT CHƯƠNG TRÌNH C

Một chương trình C bao gồm các phần: Các chỉ thị tiền xử lý, khai báo biến toàn cục, các hàm tự tạo, chương trình chính (hàm main).

Các chỉ thị tiền xử lý (Preprocessor directives)
#include <Tên tập tin thư viện>
Định nghĩa kiểu dữ liệu: phần này tùy chọn, dùng để đặt tên lại cho một kiểu dữ liệu nào đó để gọi nhớ hay đặt 1 kiểu dữ liệu cho riêng mình dựa trên các kiểu dữ liệu đã có. Cú pháp: typedef <Tên kiểu cũ> <Tên kiểu mới> Ví dụ: typedef int SoNguyen; // Kiểu SoNguyen là kiểu int
Khai báo các prototype (tên hàm, các tham số, kiểu kết quả trả về,... của các hàm): phần này tùy chọn, và chỉ là các khai báo đầu hàm, không phải là phần định nghĩa hàm.
Khai báo các biến ngoài: phần này tùy chọn, dùng để khai báo các biến toàn cục được sử dụng trong cả chương trình.

Chương trình chính (phần này bắt buộc phải có)

<Kiểu dữ liệu trả về> main()

{

Các khai báo cục bộ trong hàm main: Các khai báo này chỉ tồn tại trong hàm mà thôi, có thể là khai báo biến hay khai báo kiểu.

Các câu lệnh dùng để định nghĩa hàm

return <kết quả trả về>; Hàm phải trả về kết quả

}

Cài đặt các hàm

<Kiểu dữ liệu trả về> function1(các tham số)

{

Các khai báo cục bộ trong hàm.

Các câu lệnh dùng để định nghĩa hàm

return <kết quả trả về>;

}

Ví dụ: Tính diện tích hình chữ nhật biết chiều dài và chiều rộng

```
#include <stdio.h>                                /*các chỉ thị tiền xử lý*/
#include <conio.h>
int  tinh(int x,int y);                            /* khai báo hàm*/
int main()                                         /* chương trình chính*/
{
    int x,y;
    printf("\nNhap chieu dai:");
    scanf("%d",&x);
    printf("\nNhap chieu rong:");
    scanf("%d",&y);
    printf("\nDien tich hinh chu nhat: %d", tinh(x,y));
    getch();
    return 0;
}
int  tinh(int x,int y)                            /* định nghĩa hàm*/
{
    return (x*y);
}
```

Tuy nhiên, chương trình trên có thể được viết lại như sau, với hàm *tinh* được khai báo và được định nghĩa ở trước hàm *main()*:

```
#include <stdio.h>                                /*các chỉ thị tiền xử lý*/
#include <conio.h>
int  tinh(int x,int y);                            /* khai báo và định nghĩa hàm*/
{
    return (x*y);
}
int main()                                         /* chương trình chính*/
{
    int x,y;
    printf("\nNhap chieu dai:");
    scanf("%d",&x);
```

```
printf("\nNhap chieu rong:");
scanf("%d",&y);
printf("\nDien tich hinh chu nhat: %d", tich(x,y));
getch();
return 0;
}
```

➤ Một số qui tắc khi viết chương trình

Khi viết chương trình nên tuân theo một số qui tắc sau:

- Mỗi câu lệnh có thể viết trên một hoặc nhiều dòng nhưng phải kết thúc bởi dấu ;
- Các lời chú thích được đặt giữa /* và */ và có thể nằm trên một dòng, hoặc nhiều dòng.
- Trong chương trình khi sử dụng các hàm chuẩn như scanf(), gets()...mà các hàm này lại chức trong thư viện **stdio.h**, thì ở đầu chương trình ta phải khai báo:

```
#include <stdio.h>
```

- Một chương trình có thể có một hàm chính (hàm main()) và một hoặc nhiều hàm khác.

BÀI TẬP CHƯƠNG 2

1. Cho biết kết quả màn hình khi chạy chương trình sau:

```
#include <stdio.h>
int main()
{
    float a=1.2 ;
    int b=a ;
    printf ("%d",b) ;
}
```

- a) Chương trình có lỗi khi biên dịch
b) Kết quả không xác định được
c) Màn hình hiển thị giá trị (điền vào ô trống bên)

2. Cho biết kết quả màn hình khi chạy chương trình sau:

```
# include <stdio.h>
int main()
{
    int a = 5, b = 10, c = 15, d = 0;
    printf ("%d", a+b || c==d);
    printf ("%d", (c >= 6) || (a+c <= 20));
    printf ("%d", ! (b <= 12) && a % 2 == 0);
    printf ("%d", ! (a > 5) || c < a+b);    getch();
}
```

- a) Chương trình có lỗi khi biên dịch.
b) Kết quả không xác định được.
c) Màn hình hiển thị giá trị (điền vào ô trống bên)

3. Cho biết kết quả màn hình khi chạy chương trình sau:

```
int main()
{
    int a = 2, b = 3;
    int c = a++, d = ++b;
    printf ("%d", (c-b || d-b));
    printf ("%d", (a > c) || (d < b)) ;
    printf ("%d", !(c == 2) && a % 2 == 0) ;
}
```

- A. 001 B. 010 C. 011 D.100
E. 101 F. 110 G. 111 H. Kết quả khác

4. Cho biết kết quả màn hình khi chạy chương trình sau:

```
int main()
{
    int a=3, b=4, c=5, d;
    d = a * b + c/2;
    printf ("%d",d);
    d = a % b;
    printf ("%d",d);
    d = a / b + c;
```

```
printf("%d",d);
return 0;
}
```

a) Chương trình có lỗi khi biên dịch.

b) Kết quả không xác định được.

c) Màn hình hiển thị giá trị (điền vào ô trống bên)

5. Test chương trình sau, sửa lỗi nếu có? Cho biết kết quả màn hình khi chạy?

```
#include <stdio.h>
# define MAX 4.0
int main()
{
    int a = 10, b= 5
    double c = 4.0, f = 0.0;
    f = MAX + c * (b / a);
    printf("%f", f);
    getch();
    return 0;
}
```

6. Xét chương trình tính tổng, hiệu, tích, thương 2 số nhập vào từ bàn phím:

```
#include <stdio.h>
int main()
{
    short x , y, tong, hieu, tich;
    float thuong;
    printf("Nhap so thu nhat: ");
    scanf("%d", x);
    printf("Nhap so thu hai:");
    scanf("%d", y);
    tong = x + y ;
    hieu = x - y ;
    tich = x * y ;
    thuong = x / y ;
    printf("Tong hai so la %d\n", tong);
    printf("Hiệu hai so la %d\n", hieu);
    printf("Tich hai so la %d\n", tich);
    printf("Thuong hai so la %d\n", thuong);
    getch();
    return 0;
}
```

a) Chạy thử chương trình trên, nhập giá trị x, y như bảng sau, cho biết kết quả và nhận xét? Sửa lại chương trình để chương trình chạy đúng?

b) Giới hạn của x, y để kết quả của các phép tính vẫn còn đúng?

x	y	x+y	x-y	x*y	x/y
100	100				
2000	2000				

7. Xác định giá trị của những biểu thức sau, viết chương trình kiểm chứng lại kết quả:

- a) $x = 7 + 3 * 6 / 2 - 1$
- b) $x = 2 \% 2 + 2 * 2 - 2 / 2$
- c) $x = (3 * 2 * (3 + (4 * (3 / 3))))$
- d) $x = (7 \% 2) * 2 + 4 / 2 * 3 - 8$
- e) $x = 7 \% 2 * 2 + 4 / 2 * 3 - 8$
- f) $x = (7 \% 2 * (2 + 4) / 2 * 3) - 8$

8. Xác định giá trị của những biểu thức sau, viết chương trình kiểm chứng lại kết quả. Giả sử $a = 6, b = 5$.

- | | |
|--|--------------------|
| a) $(a > 5) \ \&\& \ (b \leq 6)$ | g) $y = ++a + b++$ |
| b) $(a \neq b) \ \ (b == 4)$ | h) $y = a++ + b$ |
| c) $! \ (b > 6)$ | i) $y = --a + b$ |
| d) $! \ (a == 4) \ \&\& \ (b < 3)$ | j) $y = a-- + b$ |
| e) $(a > 2) \ \&\& \ ((b < 3) \ \ (a > 4))$ | k) $y = ++a * 3$ |
| f) $y = a + b$ | |

9. Xác định giá trị của những biểu thức sau, viết chương trình kiểm chứng lại kết quả. Giả sử $i = 3, j = 4$.

- | | |
|-------------------------------------|--|
| a) $i = ++j;$ | i) $! \ ((i / 6 == 3) \ \&\& \ (j < 8));$ |
| b) $i = j++;$ | j) $((i > 11) \ \&\& \ ((j < 15) \ \ (i > 4))));$ |
| c) $i++;$ | k) $k = i + j \% 8;$ |
| d) $i = ++i + 5;$ | l) $k = i \% 3 + 4 - 2 * j / 5;$ |
| e) $i = i + ++5;$ | m) $k = (i + j) / 3 * 2 \% 8 * 100;$ |
| f) $((i > 5) \ \&\& \ (j \leq 6));$ | n) $k = ++i + j++;$ |
| g) $((i \neq i) \ \ (j == 6));$ | o) $k = i * 8 * 3 * (j \% 7);$ |
| h) $(! \ (i > 10));$ | p) $k = i + j / 3 - 2 * ++j / 19;$ |
| | q) $k = ++(i++ + j++);$ |

CHƯƠNG 3

CÁC CẤU TRÚC ĐIỀU KHIỂN

3.1. CẤU TRÚC TUẦN TỰ

Trong cấu trúc tuần tự (sequence structure), các lệnh trong chương trình được thực hiện một cách tuần tự theo thứ tự xuất hiện của các lệnh trong chương trình.

Tuy nhiên, một chương trình thường không chỉ bao gồm các lệnh tuần tự nối tiếp nhau. Trong quá trình thực hiện nó có thể rẽ nhánh hay lặp lại một đoạn lệnh nào đó. Để thực hiện chương trình như vậy, chúng ta sử dụng cấu trúc rẽ nhánh và cấu trúc lặp.

3.2. CẤU TRÚC RẼ NHÁNH

Cấu trúc rẽ nhánh (selection structure), hay còn gọi là cấu trúc *nếu...thì...ngược lại* (*if....then...else*), cho phép các lập trình viên diễn đạt được các logic điều kiện trong chương trình. Ý nghĩa của cấu trúc này là nếu điều kiện là đúng thì thực hiện khối lệnh ứng với điều kiện đúng, ngược lại, thực hiện khối lệnh ứng với điều kiện sai.

Một dạng khác của cấu trúc rẽ nhánh là cấu trúc switch. Cấu trúc switch được dùng khi một điều kiện được kiểm tra có thể dẫn đến 2 hay nhiều quá trình xử lý khác nhau. Trong một chương trình, menu là một ví dụ về cấu trúc switch vì nó cung cấp nhiều chọn lựa xử lý khác nhau.

3.2.1. Câu lệnh if

Toán tử if cho phép lựa chọn theo một trong hai nhánh tùy thuộc vào giá trị bằng 0 (*sai*) hay khác 0 (*đúng*) của biểu thức.

➤ Cú pháp:

Câu lệnh if có 2 dạng:

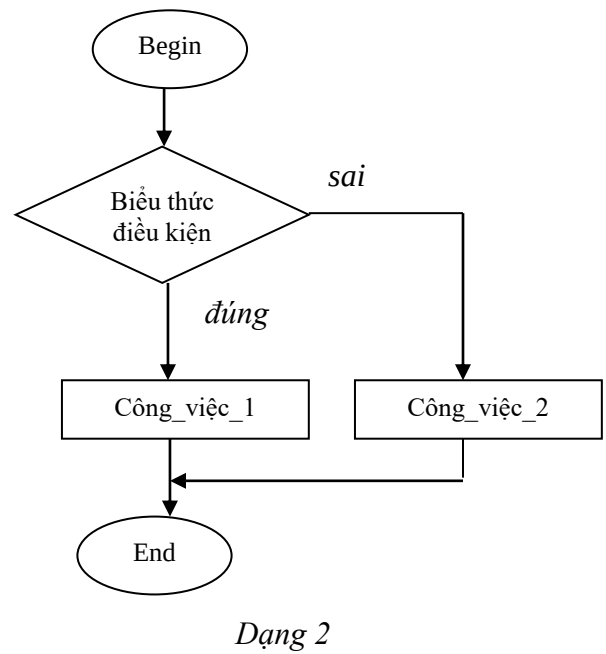
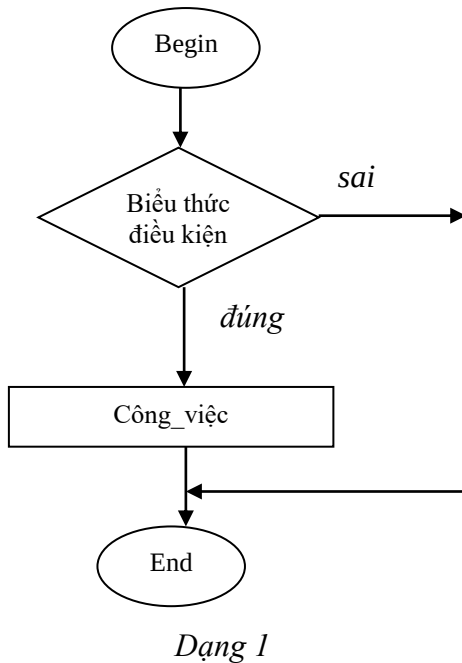
Dạng 1

if (Biểu thức điều kiện)
 Công_việc;

Dạng 2

if (Biểu thức điều kiện)
 Công_việc_1;
else
 Công_việc_2;

➤ Lưu đồ cú pháp:



Thứ tự thực hiện của câu lệnh if theo dạng 1 như sau:

B1: Tính giá trị của biểu thức điều kiện trước.

B2: Nếu giá trị của biểu thức điều kiện khác 0 (*đúng*) thì thực hiện công việc rồi kết thúc.

- Nếu giá trị của biểu thức điều kiện bằng 0 (*sai*) thì kết thúc.

Ví dụ 1: Viết chương trình nhập vào 2 số a, b. Tìm số lớn nhất trong 2 số.

```

#include <stdio.h>
int main()
{
    float a,b,max;
    printf("\n Cho a=");
    scanf("%f",&a);
    printf("\n Cho b=");
    scanf("%f",&b);
    if (a>=b) max=a;
    if (a<b) max=b;
    printf(" \n Max của hai số a=%8.2f và b=%8.2f là Max=%8.2f",a,b,max);
    getch();
    return 0;
}
    
```

Thứ tự thực hiện của câu lệnh if theo dạng 2 như sau:

B1: Tính giá trị của biểu thức điều kiện trước.

B2: Nếu giá trị của biểu thức điều kiện khác 0 (*đúng*) thì thực hiện công việc 1 rồi kết thúc.

- Nếu giá trị của biểu thức điều kiện bằng 0 (*sai*) thì thực hiện công việc 2 rồi kết thúc.

Ví dụ 2: Chương trình “Nhập vào 2 số a, b. Tìm số lớn nhất trong 2 số” có câu lệnh if được viết theo dạng 2:

```

#include <stdio.h>
    
```

```
int main()
{
    float a,b,max;
    printf("\n Cho a=");
    scanf("%f",&a);
    printf("\n Cho b=");
    scanf("%f",&b);
    if (a>=b) max=a;
    else max=b;
    printf(" \n Max của hai số a=%8.2f  và b=%8.2f là Max=%8.2f",a,b,max);
    getch();
    return 0;
}
```

☞ Lưu ý:

- Biểu thức điều kiện có thể là biểu thức bất kỳ (biểu thức nguyên, thực, quan hệ, logic...) miễn sao trả về hoặc giá trị bằng 0 (*ứng với trường hợp sai*) hoặc giá trị khác 0 (*ứng với trường hợp đúng*).
- Các **Công_việc** có thể là câu lệnh đơn hoặc là khối lệnh.
- Biểu thức điều kiện được đặt trong cặp dấu (và).
- Ta có thể sử dụng các câu lệnh if...else lồng nhau. Trong trường hợp if...else lồng nhau thì *else sẽ kết hợp với if gần nhất chưa có else*.
- Trong trường hợp câu lệnh if “bên trong” không có else thì phải viết nó trong cặp dấu {} (coi như là khối lệnh) để tránh sự kết hợp else if sai.

Ví dụ 2:

```
if ( so1>0)
if (so2 > so3)
    a=so2;
else          /*else của if(so2>so3) */
    a=so3;
```

Ví dụ 3:

```
if (so1>0)
{ if (so2>so3) /*lệnh if này không có else*/
    a=so2;
} else          /*else của if(so1>0)*/
    a=so3;
```

3.2.2. Câu lệnh switch

Lệnh switch cũng giống cấu trúc else if, nhưng nó mềm dẻo hơn và linh động hơn nhiều so với sử dụng if. Tuy nhiên, nó cũng có mặt hạn chế là kết quả của biểu thức phải là **giá trị hằng nguyên**. Một bài toán sử dụng lệnh switch thì cũng có thể sử dụng if, nhưng ngược lại còn tùy thuộc vào giải thuật của bài toán.

➤ **Cú pháp:**

```

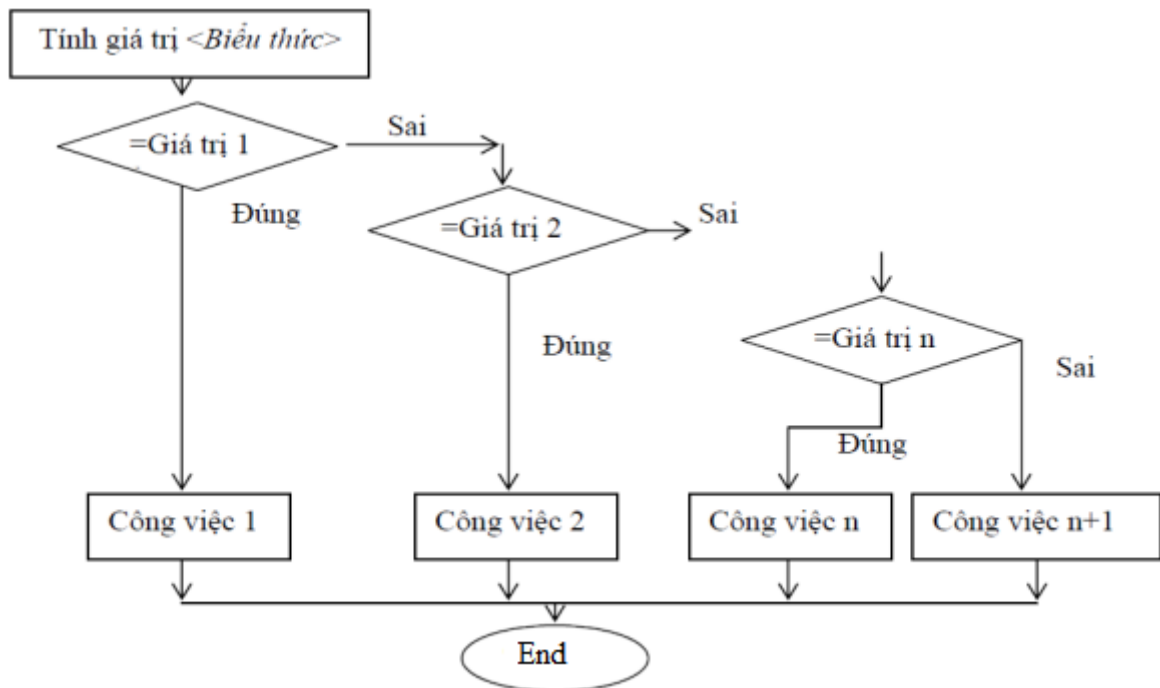
switch (Biểu thức)
{
    case giá_trị_1:
        Khởi lệnh thực hiện công việc 1;
        break;

    ...

    case giá_trị_n:
        Khởi lệnh thực hiện công việc n;
        break;

    [default:
        Khởi lệnh thực hiện công việc mặc định;
        break;]
}
    
```

➤ Lưu đồ cú pháp:



Thứ tự thực hiện của câu lệnh case như sau:

B1: Tính giá trị của biểu thức trước.

B2: Nếu giá trị của biểu thức bằng giá trị 1 thì thực hiện công việc 1 rồi thoát.

- Nếu giá trị của biểu thức khác giá trị 1 thì so sánh với giá trị 2, nếu bằng giá trị 2 thì thực hiện công việc 2 rồi thoát.
- Cứ như thế, so sánh tới giá trị n.

B3: Nếu tất cả các phép so sánh trên đều sai thì thực hiện công việc mặc định của trường hợp default.

☞ **Lưu ý:**

- Biểu thức trong switch() phải có kết quả là giá trị **kiểu số nguyên** (int, char, long, ...).
- Các giá trị sau case cũng phải là kiểu số nguyên.
- Không bắt buộc phải có default.

Ví dụ 1: Chương trình nhập vào các số 1, 2, 3. In ra số sao tương ứng.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int i;
    printf("Nhập vào số 1, 2 hoặc 3: ");
    scanf("%d", &i);
    switch(i)
    {
        case 3: printf("***");
                break;
        case 2: printf("**");
                break;
        case 1: printf("*");
                break;
        default: printf("Bạn phải nhập vào số 1, 2 hoặc 3.\n");
    }
    getch();
    return 0;
}
```

Ví dụ 2: Chương trình nhập vào các tháng. In ra tháng đó thuộc quý nào trong năm.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int thang;
    printf("Nhập vào tháng: ");
    scanf("%d", &thang);
    if (thang > 0 && thang <= 12)
    {
        switch(thang)
        {
            case 1:
            case 2:
            case 3: printf("Tháng %d thuộc Quý 1.", thang);
                    break;
            case 4:
            case 5:
            case 6: printf("Tháng %d thuộc Quý 2.", thang);
                    break;
            case 7:
            case 8:
```

```

        case 9: printf("Thang %d thuoc Quy 3.", thang);
                break;
        case 10:
        case 11:
        case 12: printf("Thang %d thuoc Quy 4.", thang);
                break;
    }
    else printf("Thang khong hop le.");
    getch();
    return 0;
}

```

3.3. CẤU TRÚC LẶP

Cấu trúc lặp (iteration structure/looping structure) cho phép thực hiện nhiều lần một khối lệnh dựa trên một điều kiện cho trước. Có 2 dạng cơ bản của cấu trúc lặp là cấu trúc for và while.

- Cấu trúc lặp for cho phép lặp lại một quá trình xử lý một số lần xác định trước.
- Cấu trúc lặp while thì lặp lại quá trình xử lý cho đến lúc điều kiện kiểm tra không còn đúng nữa.

3.3.1. Câu lệnh for

Lệnh for cho phép lặp lại công việc cho đến khi điều kiện sai.

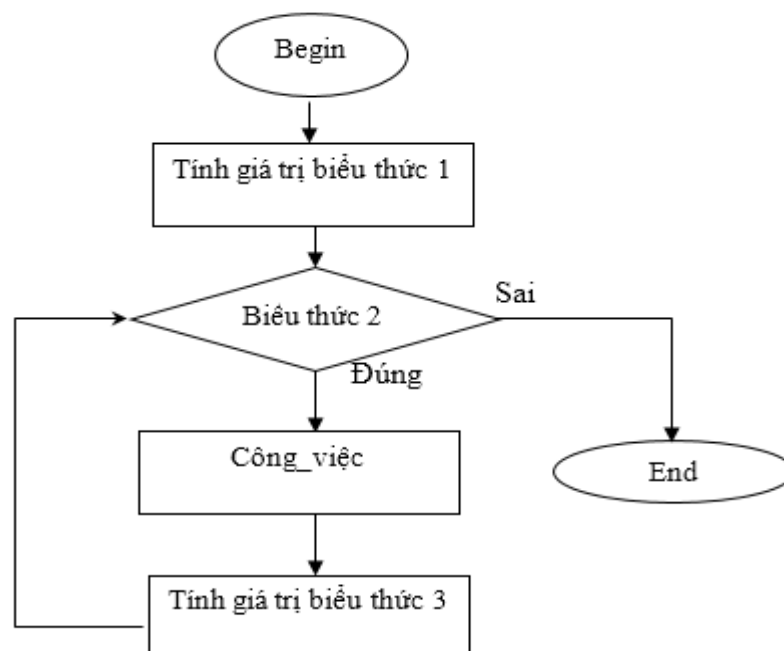
➤ Cú pháp:

```

for (biểu thức 1; biểu thức 2; biểu thức 3)
{
    <Công_việc>;
}

```

➤ Lưu đồ cú pháp:



Giải thích:

- <Công_việc>: được thể hiện là 1 câu lệnh hay 1 khối lệnh.
- Biểu thức 1: là một phép gán để khởi tạo giá trị ban đầu cho biến điều khiển.
- Biểu thức 2: là một biểu thức kiểm tra điều kiện đúng sai để dừng vòng lặp.
- Biểu thức 3: phép gán để thay đổi giá trị của biến điều khiển.

Thứ tự thực hiện của câu lệnh for như sau:

B1: Tính giá trị của biểu thức 1.

B2: Tính giá trị của biểu thức 2.

- Nếu giá trị của biểu thức 2 là sai (=0): thoát khỏi câu lệnh for.
- Nếu giá trị của biểu thức 2 là đúng (!=0): thực hiện Công_việc trong thân vòng **for**

B3: Tính giá trị của biểu thức 3 và quay lại **B2**.

Ví dụ 1: Viết chương trình nhập vào một số nguyên n. Tính tổng của các số nguyên từ 1 đến n.

```
#include <stdio.h>
#include<conio.h>
void main ()
{
    unsigned int n,i,tong;
    clrscr();
    printf("\n Nhập vào số nguyên dương n:"); scanf("%d",&n);
    tong = 0;
    for (i = 1; i <= n; i++)
        tong += i;
    printf("\n Tổng từ 1 đến %d =%d ", n, tong);
    getch();
    return 0;
}
```

Nếu chúng ta nhập vào số 9 thì kết quả như sau:

Nhap vào số nguyên dương n: 9
 Tổng từ 1 đến 9 = 45 _

☞ **Lưu ý:**

- Trong phần ngoặc tròn () của toán tử for được chia thành 3 phần phân cách nhau bởi dấu ‘;’. Các dấu ‘;’ này bắt buộc phải có mặt, kể cả trường hợp một hoặc một số biểu thức bị vắng mặt.
- Với mỗi biểu thức có thể viết thành một dãy biểu thức con phân cách nhau bởi dấu phẩy (,). Khi đó các biểu thức con được xác định từ trái sang phải. Tính đúng sai của dãy biểu thức con trong biểu thức thứ 2 được xác định bởi biểu thức con cuối cùng.
- Trong thân for (khối lệnh) có thể chứa một hoặc nhiều cấu trúc điều khiển khác.
- Khi gặp lệnh **break**, cấu trúc lặp sâu nhất sẽ thoát ra.
- Trong vòng lặp for có thể:

- Dùng lệnh **goto** để thoát khỏi vòng lặp đến vị trí mong muốn.
- Sử dụng **return** để trở về một hàm nào đó.
- Sử dụng lệnh **continue** để chuyển đến đầu vòng lặp (bỏ qua các câu lệnh còn lại trong thân).
- Khi biểu thức 2 vắng mặt thì nó được coi là luôn luôn đúng. Muốn thoát khỏi vòng lặp for trong trường hợp này ta phải dùng một trong 3 lệnh **break**, **goto** hoặc **return**.

Ví dụ 2: Đọc vào một loạt kí tự trên bàn phím, đếm số kí tự nhập vào. Kết thúc khi gặp dấu chấm '.'

```
#include <stdio.h>
#include <conio.h>
#define DAU_CHAM  '.'
int main()
{
    char c;
    int idem = 0;
    for(;;)
    {
        c = getchar();
        if (c == DAU_CHAM) //nhap vao dau cham
            break;        //thoat vong lap
        idem++;
    }
    printf("So ki tu: %d.\n", idem);
    getch();
    return 0;
}
```

3.3.2. Câu lệnh while

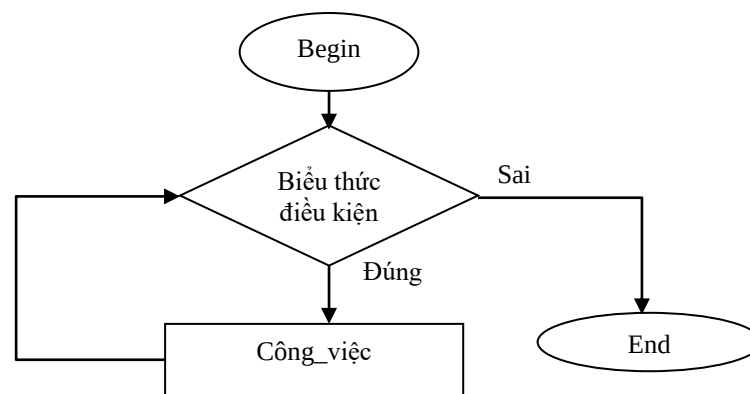
Vòng lặp while giống như vòng lặp for, dùng để lặp lại một công việc nào đó cho đến khi điều kiện sai.

➤ Cú pháp:

while (Biểu thức điều kiện)

<Công việc>

➤ Lưu đồ cú pháp:



Giải thích:

- <Công_việc>: được thể hiện bằng 1 câu lệnh hay 1 khối lệnh.

Thứ tự thực hiện câu lệnh while như sau:

B1: Kiểm tra *Biểu thức điều kiện*.

- Nếu điều kiện sai ($=0$) thì chuyển sang B2.
- Nếu điều kiện đúng ($\neq 0$) thì thực hiện Công_việc rồi quay lại kiểm tra điều kiện tiếp.

B2: Thoát khỏi lệnh while.

Ví dụ: Viết chương trình nhập vào một số nguyên n. Tính tổng của các số nguyên từ 1 đến n.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    unsigned int n,i,tong;
    clrscr();
    printf("\n Nhập vào số nguyên dương n:");
    scanf("%d",&n);
    tong = 0;
    i = 1;
    while (i <= n)
    {
        tong += i;
        i++;
    }
    printf("\n Tổng từ 1 đến %d =%d ", n, tong);
    getch();
    return 0;
}
```

☞ **Lưu ý:**

- Thân vòng lặp có thể được thực hiện một hoặc nhiều lần và cũng có thể không được thực hiện lần nào nếu ngay từ ban đầu *Biểu thức điều kiện* đã sai ($=0$).
- Biểu thức: có thể là một biểu thức hoặc nhiều biểu thức con. Nếu là nhiều biểu thức con thì cách nhau bởi dấu phẩy (,) và tính đúng sai của biểu thức được quyết định bởi biểu thức con cuối cùng.
- Trong thân while (khối lệnh) có thể chứa một hoặc nhiều cấu trúc điều khiển khác.
- Trong thân while có thể sử dụng lệnh **continue** để chuyển đến đầu vòng lặp (bỏ qua các câu lệnh còn lại trong thân).
- Muốn thoát khỏi vòng lặp while tùy ý có thể dùng các lệnh **break**, **goto**, **return** như lệnh **for**.

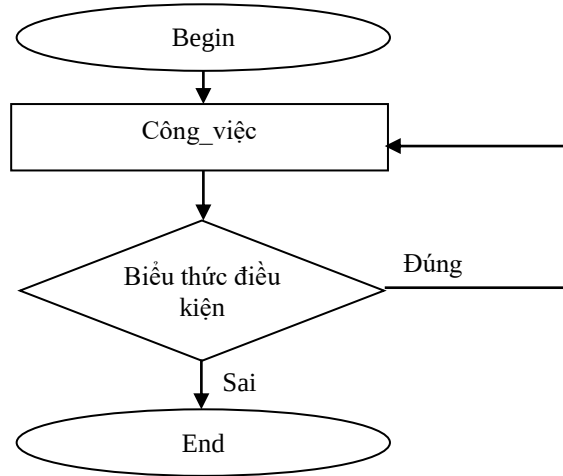
3.3.3. Câu lệnh do...while

Vòng lặp do ... while giống như vòng lặp for, while, dùng để lặp lại một công việc nào đó khi điều kiện còn đúng.

➤ **Cú pháp:**

```
do    <Công_việc>
while (<Biểu thức điều kiện>)
```

➤ Lưu đồ cú pháp:



Giải thích:

- <Công_việc>: được thể hiện bằng 1 câu lệnh hay 1 khối lệnh.

Thứ tự thực hiện câu lệnh do while như sau:

B1: Thực hiện Công_việc.

B2: Kiểm tra *Biểu thức điều kiện*.

- Nếu điều kiện sai ($\neq 0$) thì chuyển sang B3.
- Nếu điều kiện còn đúng ($\neq 0$) thì thực hiện Công_việc rồi quay lại kiểm tra điều kiện tiếp.

B3: Kết thúc lệnh do...while.

Ví dụ: Viết chương trình nhập vào một số nguyên n. Tính tổng của các số nguyên từ 1 đến n.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    unsigned int n,i,tong;
    clrscr();
    printf("\n Nhập vào số nguyên dương n:");
    scanf("%d",&n);
    tong=0;
    i=1;
    do
    {
        tong+=i;
        i++;
    } while (i<=n);
    printf("\n Tổng từ 1 đến %d =%d ", n, tong);
    getch();
    return 0;
}
```

☞ **Lưu ý:**

- Lệnh do...while thực hiện công việc ít nhất 1 lần.
- Biểu thức: có thể là một biểu thức hoặc nhiều biểu thức con. Nếu là nhiều biểu thức con thì cách nhau bởi dấu phẩy (,) và tính đúng sai của biểu thức được quyết định bởi biểu thức con cuối cùng.
- Trong thân do...while (khối lệnh) có thể chứa một hoặc nhiều cấu trúc điều khiển khác.
- Trong thân do...while có thể sử dụng lệnh **continue** để chuyển đến đầu vòng lặp (bỏ qua các câu lệnh còn lại trong thân).
- Muốn thoát khỏi vòng lặp do...while tùy ý có thể dùng các lệnh **break, goto, return**.

3.3.4. Sự khác nhau giữa các vòng lặp

- Vòng lặp for thường sử dụng khi biết được số lần lặp xác định.
- Vòng lặp while, do...while thường sử dụng khi không biết rõ số lần lặp.
- Khi gọi vòng lặp while, do...while, nếu biểu thức sai thì vòng lặp while sẽ không được thực hiện lần nào nhưng vòng lặp do...while thực hiện được 1 lần.
- Các lệnh lặp for, while, do...while có thể lồng vào chính nó, hoặc lồng vào lẫn nhau. Nếu không cần thiết thì không nên lồng vào nhiều cấp vì dễ gây nhầm lẫn khi lập trình cũng như kiểm soát chương trình.

Ví dụ: Vẽ hình chữ nhật đặc bằng các dấu “*”

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int i, j, dai, rong;
    printf("Nhap vao chieu dai: ");
    scanf("%d", &dai);
    printf("Nhap vao chieu rong: ");
    scanf("%d", &rong);
    for (i = 1; i <= rong; i++)
    {
        for (j = 1; j <= dai; j++) //in mot hang voi chieu dai dau *
            printf("*");
        printf("\n"); //xuong dong khi in xong 1 hang
    }
    getch();
    return 0;
}
```

Kết quả hiển thị như sau:

```
Nhap vao chieu dai: 10
Nhap vao chieu rong: 5
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

3.3.5. Các câu lệnh đặc biệt sử dụng trong cấu trúc lặp

➤ Câu lệnh break

Cú pháp: **break;**

Dùng để thoát khỏi vòng lặp **for**, **while**, **do while** hoặc toán tử **switch**. Khi gặp câu lệnh **break** trong vòng lặp hoặc cấu trúc switch, chương trình sẽ thoát ra khỏi vòng lặp và chuyển đến câu lệnh liền sau nó.

Nếu có nhiều vòng lặp hoặc switch lồng nhau thì câu lệnh break sẽ thoát ra khỏi vòng lặp sâu nhất (trong cùng) chứa lệnh break đó mà thôi.

Ví dụ: Chương trình kiểm tra một số nguyên nhập vào có phải là số nguyên tố hay không?

Số nguyên tố là một số nguyên có hai ước số, đó là 1 và chính số đó. Chương trình được minh họa như sau:

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int i, n, ng_to=1;
    printf("Nhap vao mot so nguyen: ");
    scanf("%d", &n);
    for (i=2; i<= sqrt(n); i++)
        if (n mod i ==0)
        {
            ng_to =0;
            break;
        }
    if (ng_to)
        printf("\n%d la nguyen to", n);
    else
        printf("\n%d khong la nguyen to", n);
    getch();
    return 0;
}
```

➤ Câu lệnh continue

Cú pháp: **continue;**

Khi gặp lệnh này trong các vòng lặp, chương trình sẽ bỏ qua phần còn lại trong vòng lặp và tiếp tục thực hiện lần lặp tiếp theo.

- Khi gặp lệnh **continue** bên trong thân của vòng lặp **for**, chương trình sẽ bỏ qua các câu lệnh còn lại trong thân **for** (sau **continue**) chuyển sang thực hiện *Biểu thức 3* để khởi đầu cho vòng lặp mới tiếp theo.

- Khi gặp lệnh **continue** bên trong thân của vòng lặp **while** hoặc **do while**, chương trình sẽ bỏ qua các câu lệnh còn lại trong thân vòng lặp (sau **continue**) chuyển tới kiểm tra *Biểu thức* sau từ khóa **while** để khởi động cho vòng lặp tiếp theo (*nếu Biểu thức còn đúng*) hoặc thoát khỏi vòng lặp (*nếu Biểu thức sai*).

☞ **Lưu ý:** Lệnh **continue** chỉ áp dụng cho các vòng lặp (không áp dụng cho toán tử switch)

Ví dụ: Nhập vào 1 dãy số nguyên từ bàn phím đến khi gặp số 0 thì dừng. In ra tổng các số nguyên dương.

```
#include <stdio.h>
#include <conio.h>
int main()
{ int n, tong = 0;
  for( ; ; )
  {   printf("Nhap vao 1 so nguyen: ");
      scanf("%d", &n);
      if (n < 0)
          continue;    //n < 0 quay nguoc len dau vong lap
      if (n == 0)
          break;        //n = 0 thoat vong lap
      tong += n;
  }
  printf("Tong = %d", tong);
  getch();
  return 0;
}
```

BÀI TẬP CHƯƠNG 3

1. Cho biết kết quả khi chạy các đoạn chương trình sau:

a.

```
int main()
{
    int a = 5, b = 6, c = 7, f;
    f = a*b != c*c;
    if (f) a++;
    else b--;
    printf("%d", f) ;
    f = (a>6) && (b<5);
    printf("%d", f) ;
    f = a>100 || b<50;
    printf("%d", f) ;
    return 0;
}
```

b.

```
# include <stdio.h>
int main()
{
    int x = 5, y = x++;
    char op = '*';
    switch (op)
    {
        default : x += 1;
        case '+' : x += y;
        case '-' : x -= y;
    }
    printf("%d%d",x,y);
    return 0 ;
}
```

2. Cho biết kết quả khi chạy các đoạn chương trình sau:

a.

```
#include <stdio.h>
int main ()
{
    int i=1, sum=0, N=6;
    while (i <= N)
    {
        sum += i;
        i++;
    }
    printf("%5d%5d",sum,i);
    return 0;
}
```

b.

```
#include <stdio.h>
int main ()
{
    int i=1, j=10;
    for (i=0; i<=j; i+=2)
    printf("%d", i);
    printf("%d", j);
    return 0;
}
```

c.

```
#include <stdio.h>
int main ()
{
    int x = 3, counter = 0;
    while ((x-1))
    {
        ++counter;
        x--;
    }
    printf("%d", counter);
    return 0;
}
```

d.

```
int main()
{
    int i = 0, j, sum = 0;
    do
    {
        if (i%2 ==0)
            sum += i;
    } while (++i <=6) ;
    printf("%d", sum);
    return 0;
}
```

```
e.
int main()
{   int k, N=8;
    for (k = 2; k <= N; k += 2)
        printf("%d", k);
    k++;
    printf("%d", k);
    return 0;
}
```

```
f.
# define MAX 2
int main()
{   int i=0, j=0 ;
    for (i=0; i<=MAX; j++)
    {       printf("%d", i);
            i = j;
    }
    printf("%d", j);
    return 0;
}
```

3. Xác định kết quả khi chạy đoạn chương trình sau, giải thích?

```
int a = 1;
while (a < 4)
{   printf("This is the outer loop\n") ;
    a++;
    while (a <= 25)
    {       break;
            printf("This prints 25 times\n");
    }
}
```

- Nếu bỏ lệnh *break*; thì kết quả màn hình như thế nào?
- Thay thế lệnh *break* bằng lệnh *continue*, cho biết kết quả?

4. Xác định kết quả của đoạn chương trình sau? Viết chương trình kiểm tra lại kết quả?

	Đoạn chương trình	Kết quả màn hình/Giải thích
a	for (int ctr=10; ctr>=1; ctr-=3) printf("%d\n",ctr);	
b	n =10; i=1; for (i = 0 ; i < n ; i++) printf("%d\n",i);	
c	for (i=1; i<=10; i++); for (j=1; j<=5; j++) { if (i == j) continue; else (i>j) break; else printf("%d%d\n",i,j); }	
d	i=1; start=1;	

	<pre>end=5; step=1; for (; start>=end;) { printf("%d\n",i); start+=step; end--; }</pre>	
--	--	--

CHƯƠNG 4

HÀM

4.1. HÀM

4.1.1. Khái niệm hàm

Trong những chương trình lớn, có thể có những đoạn chương trình viết lặp đi lặp lại nhiều lần, để tránh rườm rà và mất thời gian khi viết chương trình; người ta thường phân chia chương trình thành nhiều module, mỗi module giải quyết một công việc nào đó. Các module như vậy gọi là các chương trình con.

Một tiện lợi khác của việc sử dụng chương trình con là ta có thể dễ dàng kiểm tra xác định tính đúng đắn của nó trước khi ráp nối vào chương trình chính, do đó việc xác định sai sót để tiến hành hiệu chỉnh trong chương trình chính sẽ thuận lợi hơn. Trong C, chương trình con được gọi là hàm. Hàm trong C có thể trả về kết quả thông qua tên hàm hay có thể không trả về kết quả.

Hàm có hai loại: hàm chuẩn và hàm tự định nghĩa.

○ **Hàm chuẩn:** là những hàm đã được định nghĩa sẵn trong một thư viện nào đó, muốn sử dụng các hàm thư viện thì phải khai báo thư viện trước khi sử dụng bằng lệnh:

```
#include <tên_thư_viện.h>
```

Một số thư viện thường dùng:

- **stdio.h:** Thư viện chứa các hàm vào/ ra chuẩn. Gồm các hàm: *printf()*, *scanf()*, *getc()*, *putc()*, *gets()*, *puts()*, *fflush()*, *fopen()*, *fclose()*, *fread()*, *fwrite()*, *getchar()*, *putchar()*, *getw()*, *putw()*...

- **conio.h:** Thư viện chứa các hàm vào ra trong chế độ DOS (DOS console). Gồm các hàm: *clrscr()*, *getch()*, *getche()*, *getpass()*, *cgets()*, *cputs()*, *putch()*, *clreol()*,...

- **math.h:** Thư viện chứa các hàm tính toán. Gồm các hàm: *abs()*, *sqrt()*, *log()*, *log10()*, *sin()*, *cos()*, *tan()*, *acos()*, *asin()*, *atan()*, *pow()*, *exp()*,...

- **alloc.h:** Thư viện chứa các hàm liên quan đến việc quản lý bộ nhớ. Gồm các hàm: *calloc()*, *realloc()*, *malloc()*, *free()*, *farmalloc()*, *farcalloc()*, *farfree()*, ...

- **io.h:** Thư viện chứa các hàm vào ra cấp thấp. Gồm các hàm: *open()*, *_open()*, *read()*, *_read()*, *close()*, *_close()*, *creat()*, *_creat()*, *creatnew()*, *eof()*, *filelength()*, *lock()*,...

○ **Hàm tự định nghĩa:** là những hàm do người lập trình tự tạo ra nhằm đáp ứng nhu cầu xử lý của mình. Một hàm khi được định nghĩa thì có thể sử dụng bất cứ đâu trong chương trình. Trong C, một chương trình bắt đầu thực thi bằng hàm *main*.

Ví dụ: Chương trình nhập vào 2 số nguyên a,b và in ra màn hình số lớn nhất trong 2 số. Chương trình có sử dụng hàm max để tìm số lớn nhất trong hai số.

```
#include <stdio.h>
#include <conio.h>

int max(int a, int b) /*hàm tự định nghĩa*/
{
    return (a>b) ? a:b;
}
```

```
int main()
{
    int a, b, c;
    printf("\n Nhập vào 3 số a, b, c ");
    scanf("%d%d%d", &a, &b, &c);
    printf("\n Số lớn là %d", max(a, max(b,c)));
    getch();
    return 0;
}
```

4.1.2. Cú pháp hàm

Hàm có thể được xem là một đơn vị độc lập của chương trình. Các hàm có vai trò ngang nhau, vì vậy không được phép xây dựng một hàm bên trong các hàm khác.

Cấu trúc của một hàm tự định nghĩa:

```
type Tên hàm ([<kiểu tham số> <tham số>][,<kiểu tham số><tham số>][...])
{
    [Khai báo biến cục bộ]
    Các câu lệnh thực hiện hàm
    [return [<Biểu thức>];]
}
```

Giải thích:

- *type*: là kiểu dữ liệu của kết quả trả về, có thể là: int, byte, char, float, void... Một hàm có thể có hoặc không có kết quả trả về.
- *Kiểu tham số*: là kiểu dữ liệu của tham số.
- *Tham số*: là biến được liệt kê trong danh sách nằm tại phần đầu của định nghĩa chương trình con. Tham số này gọi là **tham số hình thức**, khi gọi hàm chúng ta phải truyền cho nó các **tham số thực**. Nếu có nhiều tham số, thì các tham số này phân cách nhau bằng dấu phẩy (,).
- Bên trong thân hàm (phần giới hạn bởi cặp dấu {}) là các khai báo cùng các câu lệnh xử lý. Các khai báo bên trong hàm được gọi là các khai báo biến cục bộ và các khai báo này chỉ tồn tại bên trong hàm mà thôi. Trong thân hàm chứa các câu lệnh cần thiết để thực hiện một yêu cầu nào đó đã đề ra cho hàm.
- Khi định nghĩa hàm, ta thường sử dụng câu lệnh **return** để trả về kết quả thông qua tên hàm.

4.2. THAM SỐ HÌNH THỨC VÀ THAM SỐ THỰC

Trong lập trình, tham số là biến được thu nhận bởi một chương trình con. Tại thời gian chạy, chương trình con sử dụng các giá trị được gán cho các tham số để thay đổi cách ứng xử của mình. Hầu hết các ngôn ngữ lập trình có thể định nghĩa các chương trình con không có tham số hoặc chấp nhận một vài tham số.

Tham số hình thức là biến được liệt kê trong danh sách tham số (thường nằm tại phần đầu của định nghĩa chương trình con), còn tham số thực là giá trị cụ thể của biến đó tại thời gian chạy.

Để phân biệt rõ hai khái niệm trên, xét ví dụ sau: Viết hàm tính tổng 2 số nguyên.

```
int tong(int x, int y)
{
    return (x + y);
}
```

Hàm *tong* nhận hai tham số hình thức: *x* và *y*. Nó lấy tổng của các giá trị được truyền vào các tham số này và trả về kết quả cho nơi gọi hàm. Đoạn mã gọi hàm *tong* có thể được viết như sau:

```
int s;
int a = 40;
int b = 2;
s = tong(a, b);
```

Các biến *a* và *b* được khởi tạo với các giá trị lần lượt là 40 và 2. Các biến này không phải tham số hình thức hay tham số thực. Tại thời điểm gọi hàm *tong(a,b)*, giá trị đã được gán cho các biến này được truyền vào cho hàm *tong*. Trong hàm *tong*, các tham số hình thức *x* và *y* được thay thế bởi hai tham số thực *a* và *b*, với giá trị cụ thể *a=40* và *b=2*. Giá trị của các tham số thực được cộng lại, kết quả được trả về cho nơi gọi hàm - nơi nó được gán cho biến *s*.

Tham số thực còn được gọi là đối số.

- Đối với các hàm không trả về giá trị thì dùng kiểu *void*.

Ví dụ hàm dùng để hiển thị giá trị lớn nhất của ba số thực có thể viết như sau:

```
void max_3s (float a, float b, float c)
{
    float x;
    x = a>b: a ? b;
    printf("\n Max = %0.2f", x>c:x?c);
}
```

- Hàm không có tham số thì ta cũng dùng *void* để khai báo.

Ví dụ hàm báo thức có thể viết như sau:

```
void bao_thuc (void)
{
    int i;
    for(i=0;i<10;i++) putchar(7);
}
```

- Do tham số hình thức và các biến cục bộ đều có phạm vi hoạt động trong cùng một hàm nên tham số hình thức và biến cục bộ cần có tên khác nhau.
- Lệnh **return** dùng để thoát khỏi một hàm và có thể trả về một giá trị nào đó.
- Khi gặp một toán tử **return** có chứa biểu thức, thì giá trị của biểu thức sẽ được chuyển kiểu cho phù hợp với kiểu của hàm trước khi nó được gán cho hàm.

Cú pháp:

return;	<i>/*không trả về giá trị*/</i>
return <biểu thức>;	<i>/*Trả về giá trị của biểu thức*/</i>

return (<biểu thức>; */*Trả về giá trị của biểu thức*/*

- o Nếu hàm có kết quả trả về, ta phải sử dụng câu lệnh **return** để trả về kết quả cho hàm.

Ví dụ hàm tìm số lớn nhất giữa 2 số nguyên a và b:

```
int max(int a, int b)
{   return (a>b) ? a:b;
}
```

Ví dụ: Viết hàm tìm ước số chung lớn nhất (UCLN) của 2 số tự nhiên a, b.

Cách tìm: đầu tiên ta giả sử UCLN của hai số là số nhỏ nhất trong hai số đó. Nếu điều đó không đúng thì ta giảm đi một đơn vị rồi kiểm tra giá trị đã giảm có thỏa mãn là UCLN hay không. Lặp lại quá trình cho tới khi nào tìm thấy UCLN thì dừng.

```
int ucln(unsigned int a,unsigned int b)
{   unsigned int u;
    if (a<b)    u=a;
    else       u=b;
    while ((a%u !=0) || (b%u!=0))
        u--;
    return u;
}
```

4.3. SỬ DỤNG HÀM

4.3.1. Gọi hàm

Một hàm khi định nghĩa thì chúng vẫn chưa được thực thi trừ khi ta có một lời gọi đến hàm đó.

Cú pháp gọi hàm:

<Tên hàm> ([Danh sách các tham số thực])

Nếu hàm có giá trị trả về thì ta có thể sử dụng hàm này như một biểu thức bình thường với các đối số thích hợp.

Một điều cần nhớ khi gọi hàm là: số tham số thực phải bằng số tham số hình thức và mỗi tham số thực phải có cùng kiểu giá trị như kiểu giá trị của tham số hình thức tương ứng với nó.

Ví dụ: Viết chương trình cho phép tìm ước số chung lớn nhất của hai số tự nhiên.

```
#include<stdio.h>
unsigned int ucln(unsigned int a, unsigned int b)
{   unsigned int u;
    if (a<b)    u=a;
    else       u=b;
    while ((a%u !=0) || (b%u!=0))
        u--;
    return u;
}
int main()
```

```
{    unsigned int a, b, UC;
    printf("Nhap a, b: "); scanf("%d%d", &a, &b);
    UC = ucln(a,b) ;
    printf("Uoc chung lon nhat la: %d", UC);
}
```

4.3.2. Nguyên tắc hoạt động của hàm

Trong chương trình, khi gặp một lời gọi hàm thì hàm bắt đầu thực hiện bằng cách chuyển các lệnh thi hành đến hàm được gọi. Quá trình diễn ra như sau:

- Nếu hàm có tham số, trước tiên các tham số sẽ được gán giá trị thực tương ứng.
- Chương trình sẽ thực hiện tiếp các câu lệnh trong thân hàm bắt đầu từ lệnh đầu tiên đến câu lệnh cuối cùng.
- Khi gặp lệnh *return* hoặc dấu } cuối cùng trong thân hàm, chương trình sẽ thoát khỏi hàm để trở về chương trình gọi nó và thực hiện tiếp tục những câu lệnh của chương trình này.

4.3.3. Truyền tham số cho hàm

Truyền tham số cho hàm là cung cấp các giá trị đầu vào thực sự cho hàm phù hợp với kiểu và trật tự của các tham số đã khai báo.

➤ Truyền theo giá trị (Call by Value)

Mặc nhiên, việc truyền tham số cho hàm trong C là truyền theo giá trị; nghĩa là các giá trị thực (tham số thực) không bị thay đổi giá trị khi truyền cho các tham số hình thức.

Ví dụ:

```
#include <stdio.h>
#include <conio.h>
void hoanvi(int a,int b);
int main()
{    int x = 2;
    int y = 3;
    printf("\nTruoc khi goi ham : x = %d , y = %d ", x, y);
    hoanvi(x,y) ;
    printf("\nSau khi goi ham : x = %d , y = %d ", x, y);
    getch();
    return 0;
}

void hoanvi(int a,int b)
{    int tam;
    printf("\nTruoc khi hoan vi: a = %d , b = %d ", a, b);
    tam = a;
    a = b;
    b = tam;
    printf("\nSau khi hoan vi: a = %d , b = %d ", a, b);
}
```

Kết quả trên màn hình như sau:

```

Truoc khi gọi ham : x = 2 , y = 3
Truoc khi hoan vi : a = 2 , b = 3
Sau khi hoan vi : a = 3 , b = 2
Sau khi gọi ham : x = 2 , y = 3
    
```

Trong hàm *hoanvi* ở trên, hai biến *a* và *b* gọi là tham số hình thức được truyền bằng giá trị (gọi là **tham trị** của hàm). Các tham trị của hàm coi như là một biến cục bộ trong hàm và chúng được sử dụng như là dữ liệu đầu vào của hàm.

Khi chương trình con được gọi để thi hành, tham trị được cấp ô nhớ và nhận giá trị là bản sao giá trị của tham số thực. Do đó, mặc dù tham trị cũng là biến, nhưng việc thay đổi giá trị của chúng không có ý nghĩa gì đối với bên ngoài hàm, không ảnh hưởng đến chương trình chính, nghĩa là không làm ảnh hưởng đến tham số thực tương ứng.

Như vậy hàm đã hoán vị 2 số *x* và *y* (gọi là tham số thực), nhưng khi quay trở lại chương trình chính thì giá trị của *x* và *y* vẫn không thay đổi.

Truyền theo giá trị (tham trị) được sử dụng khi không có nhu cầu thay đổi giá trị của tham số sau khi thực hiện hàm.

Để thay đổi giá trị của tham số sau khi thực hiện hàm ta có thể sử dụng biến toàn cục, nhưng sử dụng biến toàn cục gặp một số vấn đề về bộ nhớ, giá trị của nó có thể thay đổi ở mọi nơi trong chương trình. Trong trường hợp này, chúng ta sử dụng cách truyền theo tham biến. Có hai cách truyền theo tham biến: truyền theo địa chỉ và truyền theo tham chiếu.

➤ Truyền theo địa chỉ (Call by Address)

Trong đoạn chương trình trên, nếu muốn sau khi kết thúc chương trình con giá trị của *x*, *y* thay đổi thì ta phải đặt tham số hình thức là các con trỏ, còn tham số thực sự là địa chỉ của các biến. Lúc này mọi sự thay đổi trên vùng nhớ được quản lý bởi con trỏ là các tham số hình thức của hàm thì sẽ ảnh hưởng đến vùng nhớ đang được quản lý bởi tham số thực tế tương ứng (cần để ý rằng vùng nhớ này chính là các biến ta cần thay đổi giá trị).

Cách truyền tham số theo dạng này được gọi là truyền theo địa chỉ. Người ta thường áp dụng cách này đối với các dữ liệu đầu ra của hàm.

Truyền theo địa chỉ được sử dụng khi có nhu cầu thay đổi giá trị của tham số sau khi thực hiện hàm.

Ví dụ sau minh họa cách truyền địa chỉ:

```

#include <stdio.h>
#include <conio.h>
void hoanvi (int *px, int *py);
int main()
{
    int x = 2;
    int y = 3;
    printf("\nTruoc khi hoan vi: x = %d , y = %d ", x, y);
    hoanvi (&x, &y);
    printf("\nSau khi hoan vi: x = %d , y = %d ", x, y);
    getch();
    return 0;
}
    
```

```
void hoanvi(int *px,int *py)
{
    int tam;
    tam = *px;
    *px = *py;
    *py = tam;
}
```

Kết quả trên màn hình như sau:

```
Truoc khi hoan vi : x = 2 , y = 3
Sau khi hoan vi : 3 = 3 , 2 = 2
```

➤ Truyền theo tham chiếu (Call by Reference)

Cách truyền này được sử dụng trong C++. Theo cách này, các đối số trong hàm được truyền ở dạng địa chỉ (con trỏ) và được bắt đầu bằng dấu & trong khai báo. Cũng như cách truyền theo địa chỉ, ta không được truyền giá trị cho tham số này.

Truyền theo tham chiếu được sử dụng khi có nhu cầu thay đổi giá trị của tham số sau khi thực hiện hàm.

Ví dụ sau minh họa cách truyền tham chiếu:

```
#include <stdio.h>
#include <conio.h>
void hoanvi (int &px, int &py);
int main()
{
    int x = 2;
    int y = 3;
    printf("\nTruoc khi hoan vi: x = %d , y = %d ", x, y);
    hoanvi (x, y);
    printf("\nSau khi hoan vi: x = %d , y = %d ", x, y);
    getch();
    return 0;
}
void hoanvi(int &px,int &py)
{
    int tam;
    tam = px;
    px = py;
    py = tam;
}
```

Kết quả trên màn hình như sau:

```
Truoc khi hoan vi : x = 2 , y = 3
Sau khi hoan vi : 3 = 3 , 2 = 2
```

❖ Một số lưu ý khi truyền tham số cho hàm

- Trong một hàm, các tham số có thể truyền theo nhiều cách.

Ví dụ: void HonHop(int x, int &y)

```
{
    ...
}
```



```

x++;
y++;
}

```

Hàm *HonHop* có 2 tham số: x là tham trị và y là tham biến. Sau khi hàm này được gọi, giá trị của tham số thực được truyền vào cho x không thay đổi, còn giá trị của tham số thực được truyền cho y được tăng lên 1.

- o Sử dụng tham chiếu là một cách để trả về giá trị cho chương trình.

Ví dụ: hai đoạn chương trình sau cho kết quả tương đương:

```

int TinhTong(int x, int y)
{
    return x + y;
}

void TinhTong(int x, int y, int &tong)
{
    tong = x + y;
}

```

4.3.4. Phạm vi của biến

Khi lập trình chúng ta phải nắm rõ phạm vi của biến. Nếu khai báo và sử dụng không đúng, không rõ ràng sẽ dẫn đến sai sót khó kiểm soát được, vì vậy cần phải xác định đúng vị trí, phạm vi sử dụng biến trước khi sử dụng biến.

- Khai báo biến ngoài (biến toàn cục): Vị trí biến đặt bên ngoài tất cả các hàm. Các biến này có ảnh hưởng đến toàn bộ chương trình. Chu kỳ sống của nó là bắt đầu chạy chương trình đến lúc kết thúc chương trình.

- Khai báo biến trong (biến cục bộ): khai báo trong hàm hoặc khối { } và chỉ có tác dụng trong bản thân hàm hoặc khối đó (kể cả khối con nó). Biến cục bộ sẽ bị xóa khỏi bộ nhớ khi kết thúc khối khai báo nó.

Ví dụ sau minh họa phạm vi sử dụng các biến:

```

int a; //biến toàn cục
int Ham1()
{
    int a1; //biến cục bộ trong hàm Ham1
}

int Ham2()
{
    int a2; //biến cục bộ trong hàm Ham2

    {
        int a21; //biến cục bộ trong khối lệnh
    }
}

int main()
{
    int a3; //biến cục bộ trong hàm main
}

```

BÀI TẬP CHƯƠNG 4

1. Cho biết kết quả màn hình khi chạy đoạn chương trình sau:

```
a. int foo(int j)
    {
        int x = 5 ;
        if( j == 0 )
        {
            int x ;
            x = 3;
        }
        return (x) ;
    }
int main()
{
    int x = 0, y;
    y = foo(x);
    printf("%d", y);
    return 0;
}
```

```
c. void swap(int a, int b)
    {
        int t = a;
        a = b;
        b = t;
    }
int main()
{
    int a = 3, b = 2;
    swap(a,b);
    printf("%d%d", a,b);
    return 0;
}
```

```
e. int check_prime(int n)
    {
        int j ;
        if (n<2) return 0;
        for ( j=2 ; j<= n/2 ; j++)
            if (!(n%j)) break;
            if (j>n/2)
                return (1);
            else return (0);
    }
int main()
{
    int i, count = 0;
    for (i=1 ; i<10; i++)
        if (check_prime(i))
            count++;
    printf("%d",count);
    return 0;
}
```

```
b. void foo(int a, int &b)
    {
        a += 2;
        b += 4;
    }
int main ()
{
    int x=4, y=2;
    foo(x, y);
    printf("%d%d", x,y);
    return 0;
}
```

```
d. #include <stdio.h>
void increment( int i )
{
    i++;
}
int main()
{
    int i;
    for( i = 0; i < 10; increment(i))
        printf("%d",i);
    return 0;
}
```

```
f. #include <stdio.h>
void Twist(int a, int & b)
{
    a = a*2;
    b = b + 2;
}
int main ()
{
    int s=2, t=3;
    Twist(s,t);
    Twist(t,s);
    printf("%d%d",s,t);
    return 0;
}
```

2. Cho chương trình sau:

```
#include <stdio.h>
int xyz; // xyz là toàn cục
int add1 (int xyz) // xyz là biến cục bộ trong thân của hàm add1
{
    if (xyz > 0)
    {
        int xyz; // xyz là các biến cục bộ của khối này
        xyz = 5;
        printf("xyz thuộc khối: %d\n",xyz);
    }
    xyz = xyz+1;
    printf("xyz của hàm: %d\n",xyz);
    return (xyz);
}
int main()
{
    xyz=3;
    printf("Biến xyz toàn cục: %d\n",xyz);
    printf("Truyền xyz cho tester: %d\n",add1(xyz));
    printf("Biến xyz toàn cục: %d\n",xyz);
    return 0;
}
```

- Chạy chương trình trên, giải thích kết quả
- Sửa lại dòng `int add1 (int xyz)` thành `int add1 (int & xyz)` chạy lại chương trình, giải thích kết quả.

3. Cho chương trình sau:

```
#include <stdio.h>
void duplicate (int& a, int& b,int& c)
{
    a*=2;
    b*=2;
    c*=2;
}
int main ()
{
    int x=1, y=3, z=7;
    duplicate (x, y, z);
    printf("x=%d\ty=%d\tz=%d",x,y,z);
    return 0;
}
```

- Chạy chương trình trên, giải thích kết quả?
- Nếu bỏ dấu `&` trong khai báo hàm `duplicate` thì kết quả thực hiện như thế nào? Giải thích?

CHƯƠNG 5

LẬP TRÌNH VỚI DỮ LIỆU CÓ CẤU TRÚC

5.1. KIỂU MẢNG

5.1.1. Giới thiệu kiểu mảng

Mỗi biến có kiểu dữ liệu cơ sở chỉ có thể lưu trữ một giá trị đơn lẻ. Trong thực tế có nhiều bài toán cần xử lý một dãy các phần tử có cấu trúc tương tự nhau như một dãy số, một danh sách nhân viên,... Để xử lý hiệu quả các dạng dữ liệu này, C (và các ngôn ngữ lập trình bậc cao khác) cung cấp kiểu dữ liệu mảng.

Mảng là một tập hợp các phần tử có cùng một kiểu dữ liệu, gọi là kiểu phần tử. Kiểu phần tử có thể thuộc kiểu dữ liệu bất kỳ: ký tự, số, chuỗi ký tự...; cũng có khi ta sử dụng kiểu mảng để làm kiểu phần tử cho một mảng (trong trường hợp này ta gọi là mảng của mảng hay mảng nhiều chiều).

Mảng là kiểu dữ liệu được sử dụng rất thường xuyên. Chẳng hạn người ta cần quản lý một danh sách họ và tên của khoảng 100 sinh viên trong một lớp. Nhận thấy rằng mỗi họ và tên để lưu trữ ta cần 1 biến kiểu chuỗi, như vậy 100 họ và tên thì cần khai báo 100 biến kiểu chuỗi. Nếu khai báo như thế này thì đoạn khai báo cũng như các thao tác trên các họ tên sẽ rất dài dòng và rắc rối. Vì thế, kiểu dữ liệu mảng rất hữu ích trong trường hợp này; chỉ cần khai báo 1 biến, biến này có thể coi như là tương đương với 100 biến chuỗi ký tự; đó là 1 mảng mà các phần tử của nó là chuỗi ký tự.

5.1.2. Mảng một chiều

5.1.2.1. Khai báo

a. Khai báo mảng với số phần tử xác định

Cú pháp: **<Kiểu> <Tên biến mảng> [<số phần tử>];**

Ý nghĩa:

- *Tên biến mảng*: tên của mảng được đặt đúng theo quy tắc đặt tên.
- *Số phần tử*: là một hằng số nguyên, cho biết số lượng phần tử tối đa trong mảng.
- *Kiểu*: kiểu dữ liệu của các phần tử trong mảng.
- Ở đây, ta khai báo một biến mảng gồm có *số phần tử* phần tử, phần tử thứ nhất là *tên biến mảng* [0], phần tử cuối cùng là *tên biến mảng*[*số phần tử* - 1].

Ví dụ: `int a[10];`

/ Khai báo biến mảng tên a, phần tử thứ nhất là a[0], phần tử cuối cùng là a[9]. */*

Có thể xem mảng a là một dãy liên tiếp các phần tử trong bộ nhớ như sau:

Vị trí	0	1	2	3	4	5	6	7	8	9
Tên phần tử	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

Chúng ta có thể khai báo mảng 1 chiều theo dạng sau:

```
typedef <Kiểu> <Tên kiểu mảng> <[số phần tử]>;  
<Tên kiểu mảng> <Tên biến mảng>;
```

Ví dụ:

```
typedef int Mang1Chieu[10];  
Mang1Chieu m1, m2, m3;
```

Với cách khai báo này, chương trình sẽ tạo ra 3 mảng m1, m2, m3, và mỗi mảng chứa tối đa 10 phần tử số nguyên.

b. Khai báo mảng với số phần tử không xác định

Cú pháp: **<Kiểu> <Tên mảng> <[]>;**

Kiểu khai báo này thường được áp dụng trong các trường hợp: vừa khai báo vừa gán giá trị, khai báo mảng là tham số hình thức của hàm.

➤ Vừa khai báo vừa gán giá trị

Cú pháp:

```
<Kiểu> <Tên mảng> <[]> = { Các giá trị cách nhau bởi dấu phẩy };
```

Nếu vừa khai báo vừa gán giá trị thì mặc nhiên C sẽ hiểu số phần tử của mảng là số giá trị mà chúng ta gán cho mảng trong cặp dấu {}. Chúng ta có thể sử dụng hàm **sizeof()** để lấy số phần tử của mảng như sau:

Số phần tử = *sizeof(tên mảng) / sizeof(kiểu)*

Ví dụ: `double A[] = {10.5, 69.3, 39.4};`
`int n = sizeof(A) / sizeof(double);`

➤ Khai báo mảng là tham số hình thức của hàm

Vì tham số kiểu mảng truyền cho hàm chính là địa chỉ của phần tử đầu tiên của mảng, nên chúng ta có thể bỏ đi số lượng tối đa của phần tử trong khai báo hàm (Trong trường hợp này ta không cần chỉ định số phần tử của mảng là bao nhiêu), và có thể đưa số lượng phần tử thực sự qua một biến khác.

Ví dụ: Hàm nhập mảng có thể được khai báo như sau:

```
void NhapMang(int a[], int &N);
```

Trong đó a là tên của mảng chứa các phần tử số nguyên, N là số lượng phần tử thực sự của mảng.

5.1.2.2. Truy xuất từng phần tử của mảng

Sau khi mảng được khai báo, mỗi phần tử trong mảng đều có chỉ số để tham chiếu. Chỉ số bắt đầu từ 0 đến n-1 (với n là kích thước mảng). Trong ví dụ trên, ta khai báo mảng 10 phần tử thì chỉ số bắt đầu từ 0 đến 9.

Mỗi phần tử của mảng được truy xuất thông qua *Tên biến mảng* theo sau là *chỉ số* nằm trong **cặp dấu ngoặc vuông []**. Chẳng hạn a[0] là phần tử đầu tiên của mảng a được khai báo ở trên. Chỉ số của phần tử mảng là một biểu thức mà giá trị là kiểu số nguyên.

Với cách truy xuất theo kiểu này, *Tên biến mảng[Chỉ số]* có thể coi như là một biến có kiểu dữ liệu là **kiểu** được chỉ ra trong khai báo biến mảng.

5.1.2.3. Gán dữ liệu kiểu mảng

Khi gán dữ liệu cho các phần tử của mảng, không được sử dụng phép gán thông thường mà phải gán trực tiếp giữa các phần tử tương ứng. Ví dụ, khi lần lượt gán các giá trị của các phần tử trong mảng `a[5]` cho các phần tử trong mảng `b[5]`, chúng ta không thể viết `b=a`; mà phải viết như sau:

```
for(int i=0; i<5;i++)
    b[i]=a[i];
```

Ví dụ 1: Viết chương trình nhập vào 1 mảng có `n` số nguyên. Tính và in ra trung bình cộng của các số trong mảng.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int a[50], i, n, sum = 0;
    printf("Nhập vào số phần tử của mảng: ");
    scanf("%d", &n);
    //Nhập dữ liệu vào mảng
    for(i = 0; i < n; i++)
    {
        printf("Nhập vào phần tử thứ %d: ", i + 1);
        scanf("%d", &a[i]); //Nhập giá trị cho phần tử thứ i
    }
    //Tính tổng giá trị các phần tử
    for(i = 0; i < n; i++)
        sum += a[i]; //cộng dồn từng phần tử vào sum
    printf("Trung bình cộng: %.2f\n", (float) sum/n);
    getch();
    return 0;
}
```

Ví dụ 2: Chương trình vừa khai báo vừa gán trị cho 1 mảng 1 chiều các số nguyên. In mảng số nguyên này lên màn hình.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int n,i,j,tam;
    int dayso[]={66,65,69,68,67,70};
    n=sizeof(dayso)/sizeof(int); /*Lấy số phần tử*/
    printf("\n Nội dung của mảng ");
    for (i=0;i<n;i++)
        printf("%d ", dayso[i]);
    return 0;
}
```

5.1.2.4. Truyền mảng cho hàm

Khi truyền mảng cho hàm, thì tham số kiểu mảng trong khai báo hàm giống như khai báo biến mảng.

Ví dụ, khi khai báo một hàm sắp xếp tăng dần một mảng số nguyên có tối đa 100 phần tử, có thể viết như sau:

```
void SapXep(int a[100]);
```

Như trên đã trình bày, vì tham số kiểu mảng truyền cho hàm chính là địa chỉ của phần tử đầu tiên của mảng, nên chúng ta có thể bỏ đi số lượng tối đa của phần tử trong khai báo hàm, và có thể đưa số lượng phần tử thực sự qua một biến khác.

Ví dụ, hai cách khai báo sau là tương đương:

```
void Sapxep(a[100], int N);    ⇔    void Sapxep(a[], int N);
```

Khi gọi hàm *Sapxep*, chỉ cần viết: *Sapxep(a, N)*;

Một điều cần lưu ý nữa là sau khi thực hiện hàm, mảng có thể sẽ thay đổi nội dung.

Ví dụ: Viết chương trình nhập vào một dãy n số nguyên có tối đa 100 phần tử và sắp xếp các số theo thứ tự tăng dần.

Có rất nhiều giải thuật sắp xếp. Một trong số đó được mô tả như sau: Đầu tiên đưa phần tử thứ nhất so sánh với các phần tử còn lại, nếu nó lớn hơn một phần tử đang so sánh thì đổi chỗ hai phần tử cho nhau. Sau đó tiếp tục so sánh phần tử thứ hai với các phần tử từ thứ ba trở đi ... cứ tiếp tục như vậy cho đến phần tử thứ n-1.

Chương trình sẽ được chia thành các hàm *NhapMang* (Nhập các số), *SapXep* (Sắp xếp) và *InMang* (In các số); các tham số hình thức của các hàm này là 1 mảng không chỉ định rõ số phần tử tối đa, nhưng ta cần có *thêm số phần tử thực tế được sử dụng của mảng là bao nhiêu*, đây là một giá trị nguyên.

```
#include<conio.h>
#include<stdio.h>
#define MAX 100
void NhapMang(int a[], int &N)
{
    int i;
    printf("Nhap so luong phan tu N<=100: ");
    scanf("%d", &N);
    for(i=0; i< N; i++)
    {
        printf("Phan tu thu %d: ", i);
        scanf("%d",&a[i]);
    }
}
void InMang(int a[], int N)
{
    int i;
    for (i=0; i<N; i++)
        printf("%d ", a[i]);
    printf("\n");
}
void SapXep(int a[], int N)
{
    int t,i;
    for(i=0; i<N-1; i++)
        for(int j=i+1; j<N; j++)
            if (a[i] > a[j])
            {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
}
```

```
int main()
{
    int b[MAX], N;
    NhapMang(b, N);
    printf("Mang vua nhap: ");
    InMang(b, N);
    SapXep(b, N);      /* Gọi hàm sắp xếp */
    printf("Mang sau khi sap xep: ");
    InMang(b, N);
    getch();
    return 0;
}
```

5.1.3. Mảng nhiều chiều

Mảng nhiều chiều là mảng có từ 2 chiều trở lên. Điều đó có nghĩa là mỗi phần tử của mảng là một mảng khác. Người ta thường sử dụng mảng nhiều chiều để lưu các ma trận, các tọa độ 2 chiều, 3 chiều...

Phần này tập trung trình bày về mảng 2 chiều.

5.1.3.1. Khai báo

a. Khai báo mảng 2 chiều với số phần tử xác định

Cú pháp:

<Kiểu> <Tên mảng> [<Số phần tử chiều 1>] [<Số phần tử chiều 2>];

Ví dụ: Khai báo một ma trận 3x4 gồm các số thực:

```
float a[3][4];
```

Các phần tử được cấp phát vùng nhớ như sau:

Đòng \ Cột	0	1	2	3
0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Có thể khai báo mảng 2 chiều theo cú pháp sau:

typedef <Kiểu> <Tên kiểu mảng> [<Số phần tử chiều 1>] [<Số phần tử chiều 1>;
<Tên kiểu mảng> <Tên biến mảng>;

Ví dụ:

```
typedef int Mang2Chieu[3][4];
Mang2Chieu m4, m5;
```

b. Khai báo mảng 2 chiều với số phần tử không xác định

Để khai báo mảng 2 chiều không tường minh, ta vẫn phải chỉ ra số phần tử của chiều thứ hai (chiều cuối cùng).

Cú pháp:

<Kiểu> <Tên mảng> [<]> [<Số phần tử chiều 2>;

Cách khai báo này cũng được áp dụng trong trường hợp vừa khai báo, vừa gán trị hay đặt mảng 2 chiều là tham số hình thức của hàm.

5.1.3.2. Truy xuất từng phần tử của mảng hai chiều

Ta có thể truy xuất một phần tử của mảng hai chiều bằng cách viết ra **tên mảng** theo sau là hai chỉ số đặt trong hai cặp dấu ngoặc vuông. Chẳng hạn ta viết `a[2][3]`.

Với cách truy xuất theo cách này, **Tên mảng[Chỉ số 1][Chỉ số 2]** có thể coi là 1 biến có kiểu được chỉ ra trong khai báo biến mảng.

5.1.3.3. Gán dữ liệu cho mảng hai chiều

Giống như mảng một chiều, chúng ta không được sử dụng phép gán thông thường mà phải gán trực tiếp giữa các phần tử tương ứng.

Để minh họa việc gán lần lượt các giá trị của các phần tử trong mảng a cho mảng b thì chúng ta không thể viết `b=a`; mà phải thực hiện như đoạn chương trình sau:

```
int a[5][10], b[5][10];
int i, j;
for (i = 0; i < 5; i++)
    for (j = 0; j < 10; j++)
        b[i][j] = a[i][j];
```

Ví dụ: Viết chương trình nhập vào 1 ma trận số nguyên $n \times n$. In ra ma trận vừa nhập vào.

```
#include <stdio.h>
#include <conio.h>
#define MAX 50
int main()
{
    int a[MAX][MAX], i, j, n;
    printf("Nhập vào cấp ma tran: ");
    scanf("%d", &n);

    //Nhập dữ liệu vào ma trận
    for (i = 0; i < n; i++) //vòng for có giá trị i chạy từ 0 đến n-1 cho hàng
        for (j = 0; j < n; j++) //vòng for có giá trị j chạy từ 0 đến n-1 cho cột
        {
            printf("Nhập vào phần tử a[%d][%d]: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }

    //In ma trận
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            printf("%3d ", a[i][j]);
        printf("\n"); //xuống dòng để in hàng kế tiếp
    }

    getch();
    return 0;
}
```

5.1.3.4. Truyền mảng hai chiều cho hàm

Khi truyền mảng hai chiều cho hàm, thì cách thực hiện giống như truyền mảng một chiều cho hàm.

Ví dụ 2: Viết chương trình nhập vào 1 ma trận số nguyên $m \times n$ ($m, n \leq 10$). Sau đó hãy in ra ma trận vừa nhập và hiển thị giá trị lớn nhất trong ma trận.

Ý tưởng để tìm giá trị lớn nhất trong ma trận, ta giả sử giá trị max hiện tại là giá trị phần tử đầu tiên $a[0][0]$, sau đó lần lượt kiểm tra các phần tử còn lại để cập nhật max.

Chương trình sẽ được chia thành các hàm NhapMatran (Nhập các số), XuatMatran (Hiển thị ma trận) và TimMax (Tìm số lớn nhất trong ma trận). Các tham số hình thức của các hàm này là một mảng 2 chiều có số phần tử tối đa của chiều 1 (hàng) không cần khai báo, nhưng số phần tử của chiều 2 (cột) phải được khai báo, và giá trị thực sự của hàng và cột được đưa vào 2 biến khác.

Chương trình minh họa như sau:

```
#include<conio.h>
#include<stdio.h>
#define MAXD 10
#define MAXC 10
void NhapMaTran(int a[][MAXC], int &m, int &n) //m: so dong, n: so cot
{
    printf("Nhap so dong, so cot cua ma tran: ");
    scanf("%d%d", &m, &n);
    int i, j;
    for (i=0; i<m; i++)
        for (j=0; j<n; j++)
        {
            printf("Nhap a[%d][%d]: ", i, j);
            scanf("%d", &a[i][j]);
        }
}
void XuatMaTran(int a[][MAXC], int m, int n)
{
    int i, j;
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
            printf("%5d", a[i][j]);
        printf("\n");
    }
}
int TimMax(int a[][MAXC], int m, int n)
{
    int i, j, max;
```

```

max = a[0][0];
for (i = 0; i < m; i++)
    for (j = 0; j < n; j++)
        if (a[i][j] > max)
            max = a[i][j];
return max;
}
int main()
{
    int b[MAXD][MAXC], m, n;
    NhapMaTran(b, m, n);
    printf("\nMa tran vua nhap: \n");
    XuatMaTran(b, m, n);
    printf("\nGia tri lon nhat trong ma tran max = %d", TimMax(b, m, n));
    getch();
    return 0;
}

```

5.2. CHUỖI KÝ TỰ

Chuỗi ký tự là một dãy gồm các ký tự hoặc một mảng các ký tự được kết thúc bằng ký tự ‘\0’ (còn được gọi là ký tự NULL trong bảng mã ASCII).

Các hằng chuỗi ký tự được đặt trong cặp dấu nháy kép “”.

Ví dụ: chuỗi "Helloworld" được lưu trữ như sau:

‘H’	‘e’	‘l’	‘l’	‘o’	‘w’	‘o’	‘r’	‘l’	‘d’	‘\0’
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------



Ký tự kết thúc chuỗi

5.2.1. Khai báo

a. Khai báo theo mảng

Cú pháp:

```
char <Biến> [Chiều dài tối đa];
```

Ví dụ: `char a[12];`

Ghi chú:

- Chiều dài tối đa của biến chuỗi là một hằng nguyên nằm trong khoảng từ 1 đến 255 bytes.
- Chiều dài tối đa không nên khai báo thừa để tránh lãng phí bộ nhớ, nhưng cũng không nên khai báo thiếu.

b. Khởi tạo chuỗi

Một chuỗi được khởi tạo theo cách giống như khởi tạo một mảng thông thường.

✓ Khởi tạo chuỗi với độ dài cụ thể:

Ví dụ:

```

char s[10] = {'T', 'H', 'C', 'S', ' ', 'A', '\0'};
<=> char s[10] = "THCS A"; // Tự động thêm '\0'

```

Chuỗi s được xác định như sau:

0	1	2	3	4	5	6	7	8	9
'T'	'H'	'C'	'S'	' '	'A'	'\0'			

✓ Khởi tạo chuỗi và tự xác định độ dài:

Ví dụ:

```
char s[] = {'T', 'H', 'C', 'S', ' ', 'A', '\0'};
<=> char s[] = "THCS A"; // Tự động thêm '\0'
```

Chuỗi s được xác định như sau:

0	1	2	3	4	5	6
'T'	'H'	'C'	'S'	' '	'A'	'\0'

Ghi chú:

- Chuỗi được khai báo là một mảng các ký tự nên các thao tác trên mảng có thể áp dụng đối với chuỗi ký tự.

5.2.2. Nhập/xuất chuỗi ký tự

a. Nhập chuỗi

Hàm *gets()* được sử dụng để nhập một chuỗi ký tự từ bàn phím.

Hàm *gets* nhận các ký tự từ bàn phím đến khi gặp ký tự xuống dòng, và chuỗi nhận được là những gì người dùng nhập (trừ ký tự xuống dòng).

Cú pháp: **gets(<Biến chuỗi>);**

Ví dụ:

```
char a[20];
gets(a);
```

Ta cũng có thể sử dụng hàm *scanf()* với đặc tả "%s" để nhập dữ liệu cho biến chuỗi, tuy nhiên lúc này hàm *scanf()* chỉ nhận các ký tự từ bàn phím cho đến khi gặp ký tự khoảng trắng hoặc ký tự xuống dòng, và chuỗi nhận được không bao gồm ký tự khoảng trắng và ký tự xuống dòng.

Ví dụ: Với đoạn chương trình sau:

```
char monhoc[50];
printf("Nhap mot chuoai: ");
scanf("%s", monhoc);
printf("Chuoai nhan duoc la: %s", monhoc);
```

Nếu nhập vào nội dung *Tin hoc co so* thì chuỗi s chỉ nhận được nội dung là *Tin*.

Ngoài ra, hàm *cgets()* (trong **conio.h**) cũng được sử dụng để nhập chuỗi.

b. Xuất chuỗi

Để xuất một chuỗi (biểu thức chuỗi) lên màn hình, ta sử dụng hàm *puts()*. Sau khi xuất chuỗi, con trỏ sẽ tự động xuống dòng.

Cú pháp: **puts(<Biểu thức chuỗi>)**

Ví dụ: Nhập vào một chuỗi và hiển thị trên màn hình chuỗi vừa nhập.

```
#include<conio.h>
#include<stdio.h>
#include<string.h>
int main()
{   char Ten[12];
    printf("Nhap chuoi: "); gets(Ten);
    printf("Chuoi vua nhap: "); puts(Ten);
    getch();
}
```

Có thể sử dụng hàm *printf()* với đặc tả “%s” để xuất chuỗi, nhưng sau khi xuất chuỗi thì con trỏ không tự động xuống dòng.

Ví dụ, hai cách viết sau là tương đương:

```
puts(Ten); ⇔ printf("%s\n", Ten);
```

5.2.3. Một số hàm thao tác trên chuỗi ký tự

Để sử dụng các hàm xử lý chuỗi, chương trình phải khai báo thư viện **string.h**.

5.2.3.1. Hàm strlen

- Cú pháp: **int strlen(const char *s)**
- Xác định độ dài một chuỗi nhập từ bàn phím.
- Ví dụ:

```
char s[] = "Visual C++ 6.0";
int len = strlen(s); // => 14
```

5.2.3.2. Hàm strcpy

- Cú pháp: **char *strcpy(char *dest, const char *src)**
- Sao chép chuỗi src sang chuỗi dest, dừng khi ký tự kết thúc chuỗi ‘\0’ vừa được chép. Chuỗi dest phải đủ lớn để chứa chuỗi src.
- Ví dụ:

```
char s[100];
s = "Visual C++ 6.0";           // câu lệnh sai
strcpy(s, "Visual C++ 6.0");    // đúng
```

5.2.3.3. Hàm strdup

- Cú pháp: **char *strdup(const char *s)**
- Tạo bản sao của một chuỗi s cho trước. Hàm sẽ tự tạo vùng nhớ dài strlen(s) + 1 (bytes) để chứa chuỗi s. Nếu hàm thực hiện thành công: trả về con trỏ đến vùng nhớ chứa chuỗi bản sao, nếu thất bại: trả về NULL.
- Ví dụ:

```
char *s;
```

```
s = strdup("Visual C++ 6.0");
```

5.2.3.4. Hàm strlwr

- Cú pháp: **char *strlwr(char *s)**
- Chuyển chuỗi s thành chuỗi thường ('A' thành 'a', 'B' thành 'b', ..., 'Z' thành 'z').
- Ví dụ:

```
char s[] = "Visual C++ 6.0";
strlwr(s);
puts(s);           // visual c++ 6.0
```

5.2.3.5. Hàmstrupr

- Cú pháp: **char *strupr(char *s)**
- Chuyển chuỗi s thành chuỗi IN ('a' thành 'A', 'b' thành 'B', ..., 'z' thành 'Z').
- Ví dụ:

```
char s[] = "Visual C++ 6.0";
strupr(s);
puts(s);           // VISUAL C++ 6.0
```

5.2.3.6. Hàm strrev

- Cú pháp: **char *strrev(char *s)**
- Đảo ngược thứ tự các ký tự trong chuỗi s (trừ ký tự kết thúc chuỗi).
- Ví dụ:

```
char s[] = "Visual C++ 6.0";
strrev(s);
puts(s);           // 0.6 ++C lausiv
```

5.2.3.7. Hàm strcmp

- Cú pháp: **int strcmp(const char *s1, const char *s2)**
- So sánh hai chuỗi s1 và s2 (có phân biệt ký tự chữ cái hoa và ký tự chữ cái thường).

Nếu s1=s2: kết quả nhận được là 0

Nếu s1<s2: kết quả <0

Nếu s1>s2: kết quả >0

- Ví dụ:

```
char s1[] = "visual C++ 6.0";
char s2[] = "Visual C++ 6.0";
int kq = strcmp(s1, s2); // => kq > 0
```

5.2.3.8. Hàm stricmp

- Cú pháp: **int stricmp(const char *s1, const char *s2)**
- So sánh hai chuỗi s1 và s2 (không phân biệt ký tự chữ cái hoa và ký tự chữ cái thường).
- Nếu s1=s2: kết quả nhận được là 0

Nếu $s1 < s2$: kết quả < 0

Nếu $s1 > s2$: kết quả > 0

– Ví dụ:

```
char s1[] = "visual C++ 6.0";
char s2[] = "Visual C++ 6.0";
int kq = strcmp(s1, s2);    // => kq = 0
```

5.2.3.9. Hàm strcat

– Cú pháp: **char* strcat(char *dest, const char *src)**

– Nối chuỗi src vào sau chuỗi dest. Độ dài chuỗi dest phải đủ chứa kết quả.

– Ví dụ:

```
char s1[100] = "Visual C++";
char s2[] = "6.0";
strcat(s1, " ");    // => "Visual C++ "
strcat(s1, s2);      // => "Visual C++ 6.0"
```

5.2.3.10. Hàm strstr

– Cú pháp: **char* strstr(const char *s1, const char *s2)**

– Tìm vị trí xuất hiện đầu tiên của s2 trong s1. Nếu hàm thực hiện thành công: trả về con trỏ đến vị trí xuất hiện đầu tiên của s2 trong s1; nếu thất bại: trả về NULL.

– Ví dụ:

```
char s1[] = "Visual C++ 6.0";
char s2[] = "C++";
if (strstr(s1, s2) != null)
    printf("Tim thay s2 trong s1...");
```

5.3. KIỂU STRUCT

Để lưu trữ các giá trị gồm nhiều thành phần dữ liệu giống nhau ta có kiểu biến mảng. Thực tế rất nhiều dữ liệu là tập các kiểu dữ liệu khác nhau tập hợp lại, để quản lý dữ liệu kiểu này C đưa ra kiểu dữ liệu cấu trúc.

Kiểu cấu trúc (struct) là kiểu dữ liệu bao gồm nhiều thành phần có kiểu khác nhau, mỗi thành phần được gọi là một trường (field).

Một ví dụ của dữ liệu kiểu cấu trúc là một bảng lý lịch trong đó mỗi nhân sự được lưu trong một bản ghi gồm nhiều kiểu dữ liệu khác nhau như họ tên, tuổi, giới tính, mức lương ...

5.3.1. Khai báo, xử lý dữ liệu kiểu cấu trúc

5.3.1.1. Định nghĩa kiểu cấu trúc

Cách 1:

```
struct <Tên cấu trúc>
{
    <Kiểu> <Trường 1> ;
```

```
<Kiểu> <Trường 2> ;
.....
<Kiểu> <Trường n> ;
};
```

Cách 2: Sử dụng từ khóa **typedef** để định nghĩa kiểu:

```
typedef struct
{
    <Kiểu> <Trường 1> ;
    <Kiểu> <Trường 2> ;
    .....
    <Kiểu> <Trường n> ;
} <Tên cấu trúc>;
```

Trong đó:

- *<Tên cấu trúc>*: là một tên được đặt theo quy tắc đặt tên của định danh; tên này mang ý nghĩa sẽ là tên kiểu cấu trúc.
- *<Kiểu> <Trường i>* (i=1..n): mỗi trường trong cấu trúc có dữ liệu thuộc kiểu nào đó, tên của trường phải là một tên được đặt theo quy tắc đặt tên của định danh.

Ví dụ 1: Định nghĩa một kiểu dữ liệu cấu trúc ngày:

Định nghĩa cấu trúc theo cách 1:

```
struct Date
{
    int ngay;
    int thang;
    int nam;
};
```

Định nghĩa cấu trúc theo cách 2:

```
typedef struct
{
    int ngay;
    int thang;
    int nam;
} Date;
```

Ví dụ 2: Mỗi nhân viên cần được quản lý bởi các thông tin: mã nhân viên, họ tên, ngày sinh. Lúc này ta có thể khai báo một cấu trúc gồm các thông tin trên.

Định nghĩa cấu trúc theo cách 1:

```
struct NhanVien
{
    int MaNV;
    char HoTen[40];
    Date Ngaysinh;
};
```

Định nghĩa cấu trúc theo cách 2:

```
typedef struct
{
    int MaNV;
    char HoTen[40];
    Date Ngaysinh;
} NhanVien;
```

5.3.1.2. Khai báo biến kiểu cấu trúc

Cú pháp:

- Đối với cấu trúc được định nghĩa theo cách 1:

```
struct <Tên cấu trúc> <Biến 1> [, <Biến 2>...];
```

- Đối với các cấu trúc được định nghĩa theo cách 2:

<Tên cấu trúc> <Biến 1> [, <Biến 2>...];

Ví dụ: Khai báo biến nv có kiểu cấu trúc NhanVien.

Theo cách 1: struct NhanVien nv;

Theo cách 2: NhanVien nv;

5.3.1.3. Khởi tạo cấu trúc

Việc khởi tạo cấu trúc có thể được thực hiện trong lúc khai báo biến cấu trúc. Các trường của cấu trúc được khởi tạo được đặt giữa 2 dấu { và }, chúng được phân cách nhau bởi dấu phẩy (,).

Ví dụ: Khởi tạo biến cấu trúc ngay:

```
struct Date ngay = {15, 11, 1985};
```

5.3.2. Truy xuất đến các thành phần của biến cấu trúc

Chúng ta không thể truy xuất trực tiếp dữ liệu kiểu cấu trúc được, mà phải thông qua toán tử thành phần cấu trúc . hay còn gọi là toán tử chấm (dot operator).

Cú pháp:

<Biến cấu trúc>.<Tên trường>

Ví dụ:

```
typedef struct
{
    int MaNV;
    char HoTen[40];
}NhanVien;
NhanVien nv;
printf("%5d %s\n", nv.MaNV, nv.Hoten);
```

5.3.3. Gán dữ liệu kiểu cấu trúc

Có hai cách gán dữ liệu kiểu cấu trúc:

- Cách 1: **<Biến cấu trúc đích> = <Biến cấu trúc nguồn>;**

- Cách 2: **<Biến cấu trúc đích>.<Tên thành phần> = <Giá trị>;**

Ví dụ 1:

```
typedef struct
{
    int x, y;
}DIEM;
DIEM diem1={29, 15}, diem2;
...
diem2=diem1;
diem2.x=diem1.x;
diem2.y=diem1.y*2;
```

Ví dụ 2: Viết chương trình nhập và in thông tin của một nhân viên. Thông tin gồm: mã nhân viên, họ tên và ngày tháng năm sinh của nhân viên.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

typedef struct
{
    int Ngay;
    int Thang;
    int Nam;
}NgayThang;

typedef struct
{
    int MaNV;
    char HoTen[40];
    NgayThang NgaySinh;
}NhanVien;

//hàm nhập 1 mẫu tin nhân viên
void NhapNV(NhanVien &nv)
{
    printf("Nhap Ma nhan vien: ");scanf("%d",&nv.MaNV);
    fflush(stdin);
    printf("Nhap Ho va ten: ");gets(nv.HoTen);
    printf("Sinh ngay: ");scanf("%d",&nv.NgaySinh.Ngay);
    printf("Thang: ");scanf("%d",&nv.NgaySinh.Thang);
    printf("Nam: ");scanf("%d",&nv.NgaySinh.Nam);
}

//hàm hiển thị 1 mẫu tin nhân viên
void InNV(NhanVien nv)
{
    printf("%5d| %30s | %d-%d-%d \n",nv.MaNV,nv.HoTen, nv.NgaySinh.Ngay,
    nv.NgaySinh.Thang, nv.NgaySinh.Nam);
}

int main()
{
    NhanVien NV;
    printf("Nhap thong tin cua nhan vien:\n");
    NhapNV(NV);
    printf("Thong tin cua nhan vien:\n");
    InNV(NV);
}
```

```
    getch();

    return 0;

}
```

☞ **Lưu ý:**

- Với các biến kiểu cấu trúc ta không thể thực hiện được các thao tác sau đây:
 - o Sử dụng các hàm xuất nhập trên biến cấu trúc.
 - o Các phép toán quan hệ, các phép toán số học và logic.

5.3.4. Mảng cấu trúc

Như đã đề cập ở các chương trước, khi sử dụng một kiểu giá trị (ví dụ như kiểu int) ta có thể khai báo các biến và các mảng kiểu đó. Ví dụ như khai báo:

```
int a,b,c[10];
```

Cho ta hai biến nguyên là a,b và một mảng nguyên c có 10 phần tử.

Hoàn toàn tương tự như vậy: ta có thể sử dụng một kiểu cấu trúc đã mô tả để khai báo các cấu trúc và mảng cấu trúc.

5.3.4.1. Cách khai báo mảng cấu trúc

Cú pháp: **struct tên_kiểu tên_mảng [số_phần_tử];**

Trong đó:

- tên_kiểu: là tên kiểu cấu trúc đã định nghĩa.
- tên_mảng: là tên mảng cấu trúc.
- số_phần_tử: là số phần tử của mảng.

Ví dụ 1: Giả sử kiểu cấu trúc **nhanvien** đã được định nghĩa như mục trên. Khi đó dòng khai báo:

```
struct nhanvien  nv1, nv2, dsnv1[10], dsnv2[7];
```

sẽ cho: Hai biến cấu trúc nv1 và nv2; Hai mảng cấu trúc dsnv1 có 10 phần tử và dsnv2 có 7 phần tử và mỗi phần tử của hai nhóm này có kiểu **nhanvien**.

Ví dụ 2: Đoạn chương trình sau sẽ tính tổng lương cho các phần tử thuộc dsnv1:

```
double tongluong=0;
for (i=0;i<10;++i)
    tongluong+=dsnv1[i].luong;
```

Ví dụ 4: Chương trình sau minh họa nhập và xuất một danh sách nhân viên. Thông tin của nhân viên gồm: mã nhân viên, họ tên và ngày sinh.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
typedef struct
{
    int Ngay;
    int Thang;
    int Nam;
```

```

}NgayThang;
typedef struct
{
    int MaNV;
    char HoTen[40];
    NgayThang NgaySinh;
}NhanVien;

//hàm nhập 1 mẫu tin nhân viên
void NhapNV(NhanVien &nv)
{
    printf("Nhap Ma nhan vien: ");scanf("%d",&nv.MaNV);
    fflush(stdin);
    printf("Nhap Ho va ten: ");gets(nv.HoTen);
    printf("Sinh ngay: ");scanf("%d",&nv.NgaySinh.Ngay);
    printf("Thang: ");scanf("%d",&nv.NgaySinh.Thang);
    printf("Nam: ");scanf("%d",&nv.NgaySinh.Nam);
}

//hàm hiển thị 1 mẫu tin nhân viên
void InNV(NhanVien nv)
{
    printf("%5d| %30s | %d-%d-%d \n",nv.MaNV, nv.HoTen, nv.NgaySinh.Ngay,
        nv.NgaySinh.Thang, nv.NgaySinh.Nam);
}

//hàm nhập 1 danh sách nhân viên
void NhapDSNV(NhanVien nv[],int &n)
{
    printf("So nhan vien: "); scanf("%d",&n);
    for (int i = 0; i < n; i++)
    {
        printf("\nNhap thông tin nhan vien thu %d",i);
        NhapNV(nv[i]);
    }
}

//hàm xuất 1 danh sách nhân viên ra màn hình
void InDSNV(NhanVien nv[], int n)
{
    printf("\nMNV: | Ho va ten | Ngay Sinh \n");
    for (int i = 0; i < n; i++)
    {
        InNV(nv[i]);
        printf("\n");
    }
}

int main()
{
    NhanVien NV, DSNV[20];
    int n;
    NhapDSNV(DSNV,n);
    InDSNV(DSNV,n);
    getch();
}

```

```
    return 0;
}
```

5.3.4.2. Phép gán cấu trúc

Vì bản chất 1 phần tử của mảng kiểu cấu trúc cũng là 1 biến kiểu cấu trúc nên hai biến kiểu cấu trúc có thể gán cho nhau, và mỗi một phép gán tương đương với một dãy phép gán các thành phần tương ứng.

Ví dụ: Chương trình sau minh họa cách dùng phép gán cấu trúc để sắp xếp n thí sinh theo thứ tự giảm của tổng điểm

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
typedef struct
{
    char Hoten[25];
    float td;
} ThiSinh;
//hàm nhập thông tin 1 thí sinh
void NhapTS(ThiSinh &ts)
{
    float diem;
    fflush(stdin);
    printf("\nNhap Ho va ten: "); gets(ts.Hoten);
    printf("\nNhap diem: "); scanf("%f",&ts.td);
}
//hàm xuất thông tin 1 thí sinh
void XuatTS(ThiSinh ts)
{
    printf("%30s | %5f", ts.Hoten,ts.td);
}
//hàm nhập danh sách thí sinh
void NhapDSTS(ThiSinh ts[], int &n)
{
    printf("So thi sinh: "); scanf("%d",&n);
    for (int i = 0; i < n; i++)
    {
        printf("\nNhap thông tin thí sinh thu %d:",i);
        NhapTS(ts[i]);
    }
}
//hàm xuất danh sách thí sinh
void XuatDSTS(ThiSinh ts[], int n)
{
    printf("\nHo va ten          | Diem \n");
    for (int i = 0; i < n; i++)
    {
```

```
        XuatTS(ts[i]);
        printf("\n");
    }
}

void SapXep(ThiSinh ts[], int n)
{
    int i,j;
    ThiSinh tg;
    for (i = 0;i < n-1; i++)
        for (j = i+1;j < n;j++)
            if (ts[i].td < ts[j].td)
            {
                tg = ts[i];
                ts[i] = ts[j];
                ts[j] = tg;
            }
}

int main()
{
    ThiSinh ts, dsts[20];
    int n;
    NhapDSTS(dsts,n);
    printf("\nDanh sach thi sinh truoc khi sap:\n");
    XuatDSTS(dsts,n);
    SapXep(dsts,n);
    printf("\nDanh sach thi sinh sau khi sap:\n");
    XuatDSTS(dsts,n);
    getch();
    return 0;
}
```

BÀI TẬP CHƯƠNG 5

1. Trả lời các câu hỏi sau:

a) Có bao nhiêu phần tử trong khai báo sau? Phần tử thứ nhất và phần tử sau cùng?

```
int a[5];
```

b) Tìm chỗ sai trong đoạn chương trình sau, sửa lại cho đúng:

```
int a[5];
for(i = 1; i <= 5; i = i + 1)
    a[i] = 0;
```

c) Tìm chỗ sai trong đoạn chương trình sau, sửa lại cho đúng:

```
int a[10] = { 0 }, b[10];
b = a;
```

2. Cho biết kết quả màn hình khi chạy đoạn chương trình sau:

```
a. #define MAX 5
int main()
{
    int a[MAX] = {0}, i, j = 2;
    for (i = 0; i < 3; i++)
        a[i] = j+i;
    printf("%d%d", a[0], a[3]);
    return 0;
}

b. #define MAX 5
void bubbleSort(int a[ ], int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[i] > a[j])
            {
                int t = a[i] ;
                a[i] = a[j] ;
                a[j] = t;
            }
}

int main()
{
    int a[MAX] = {3, 7, 4, 6, 9};
    bubbleSort(a, MAX);
    printf("%5d%5d", a[0], a[3]);
    return 0;
}
```

3. Kết quả biểu thức: cho khai báo như sau:

```
int grades[3][5] = {80, 90, 96, 73, 65, 67, 90, 68, 92, 84, 70, 55, 95, 78,
                    100};
```

Xác định biểu thức không hợp lệ (nếu có) và tính giá trị của biểu thức nếu hợp lệ:

a) grades[2][3]

b) grades[2][4]

c) grades[0][1]

4. Cho biết kết quả của các đoạn chương trình sau:

```
a. #include<stdio.h>
#include<conio.h>
int main()
{
    int a[][4]={1,2,3,4,5,6,7,8,9};
    for(int i=0;i<4;i++)
        printf("%d",a[1][i]);
    getch();
}
```

A. 5 7 8 9

B. 1 2 3 4

C. 5 6 7 8

D. 4 5 6 7

```
b. int main()
{
    float a[5] = {5, 4, 3, 2, 1};
    float x;
    int i;
    for (i=0;i<5;i++)
        a[i]+=a[0];
    x=a[1]+a[4];
    printf("%d",x);
}
```

A. 125

B. 25

C. 5

D. 100

TÀI LIỆU THAM KHẢO

- [1] Hoàng Kiếm, “Giải một bài toán trên máy tính như thế nào?, Tập 1”, NXB Giáo Dục, 2001
- [1] Quách Tuấn Ngọc, “Ngôn ngữ lập trình C”, NXB Giáo Dục, 2003
- [2] Phạm Văn Ất, “Kỹ thuật lập trình C cơ bản và nâng cao”, NXB Hồng Đức, 2009
- [3] Nguyễn Thanh Thủy, “Nhập môn lập trình ngôn ngữ C”, NXB KHKT, 2003
- [4] ĐH.KHTN Tp.HCM, “Bài tập minh họa học phần nhập môn lập trình ”
- [5] B. W. Kernighan and D. M., “C Programming Language, 2nd Edition”, Prentice Hall, 1988

PHẦN 2

BÀI TẬP THỰC HÀNH

LAB 01 – BIỂU THỨC, TOÁN TỬ TRONG C

1. MỤC TIÊU

- Sử dụng môi trường soạn thảo và chạy một chương trình C đơn giản.
- Làm quen với các kiểu dữ liệu và toán tử cơ bản của C.
- Làm quen với biến và lệnh gán, các lệnh xuất nhập: printf, scanf.

2. YÊU CẦU

- Đọc tài liệu hướng dẫn sử dụng DEV-CPP (Xem trong **Phần 3 – Một số phụ lục**).
- Ôn lại kiến thức về hằng, biến, kiểu dữ liệu, biểu thức, toán tử trong C.
- Hiểu rõ toán tử: +, -, *, /, %, ++, --, +=, -=, *=, /=, &&, ||, !, !=, >, <, <=, >=, ...
- Độ ưu tiên của toán tử.
- Biến, hằng, lệnh gán, các lệnh xuất nhập: printf, scanf.

☞ **Lưu ý:** Mỗi bài tập thực hành nên bổ sung phần ghi chú ở phần đầu tập tin, theo mẫu sau:

```

/*****
* Lab 01, Bai 01
* SV: Nguyen X, MSSV: 123456789
* Lop: XX, Khoa: Y.
* Created date: dd/mm/yyyy;      Modified date: dd/mm/yyyy
*****/
    
```

3. BÀI TẬP THỰC HÀNH

Bài 1. Viết chương trình hiện ra màn hình hai câu thông báo sau:

Chao ban!

Chuc mung ban den voi mon Nhap mon Lap Trinh.

Bài 2. Xuất ra màn hình nội dung sau:

```

*****
*      TRUONG DAI HOC NHA TRANG      *
*      KHOA CONG NGHE THONG TIN      *
*****
    
```

Bài 3. Viết chương trình hiển thị lên màn hình tam giác sau:

```

*
* *
* * *
* * * *
    
```

Bài 4. Viết chương trình in ra màn hình hình tam giác cân sau:

```

      *
    * * *
  * * * * *
* * * * * *
  
```

Bài 5. Viết chương trình nhập tên sản phẩm, số lượng và đơn giá. Tính tiền và thuế giá trị gia tăng phải trả, biết:

- Tiền = số lượng * đơn giá
- Thuế giá trị gia tăng = 10% tiền

Bài 6. Viết chương trình nhập điểm thi và hệ số 3 môn Toán, Lý, Hóa của một sinh viên. Tính điểm trung bình của sinh viên đó.

Bài 7. Viết chương trình nhập bán kính của đường tròn. Tính chu vi và diện tích của hình tròn đó, yêu cầu chương trình có khai báo hằng số $PI=3.14$.

Bài 8. Viết chương trình nhập vào hai số nguyên dương. Tính tổng, hiệu, tích, thương của 2 số. Hiển thị kết quả ra màn hình.

Bài 9. Viết chương trình tính tổng bình phương của hai số được nhập vào từ bàn phím.

Bài 10. Viết chương trình nhập giờ, phút, giây. In ra tổng số giây.

Bài 11. Viết chương trình nhập một ký tự từ bàn phím, in ra mã ASCII của nó.

Bài 12. Viết chương trình nhập vào một ký tự c. Cho biết ký tự đứng trước, đứng sau ký tự c.

Bài 13. Nhập số tiền nguyên N đồng, đổi ra xem được bao nhiêu tờ 10 đồng, 5 đồng, 2 đồng và 1 đồng.

Ví dụ: $N = 543đ = 54 \text{ tờ } 10đ + 0 \text{ tờ } 5đ + 1 \text{ tờ } 2đ + 1 \text{ tờ } 1đ$

Bài 14. Viết chương trình nhập vào số xe (gồm 4 chữ số) của bạn. Cho biết số xe của bạn được mấy nút?

Bài 15. Nhập vào số nguyên có 3 chữ số, tính tổng 3 chữ số đó.

Ví dụ: Số 543 có tổng 3 chữ số là: $5 + 4 + 3 = 12$

Bài 16. Nhập vào số nguyên có 4 chữ số. Xuất ra số đó theo thứ tự ngược.

Ví dụ: nhập vào 9876 thì in ra 6789

LAB 02 – CÂU LỆNH RỄ NHÁNH

1. MỤC TIÊU

- Sử dụng thành thạo cấu trúc rẽ nhánh: if, if...else, switch

2. YÊU CẦU

- Hiểu rõ cấu trúc lệnh if, if...else, switch.
- Kiến thức toán học cơ bản: số chẵn, số lẻ, bội số, ước số, giải phương trình bậc nhất, phương trình bậc hai.
- Bảng mã ASCII.

3. BÀI TẬP THỰC HÀNH

Bài 1. Viết chương trình giải phương trình bậc nhất $ax+b=0$ với a, b nhập từ bàn phím.

Bài 2. Viết chương trình giải phương trình bậc hai $ax^2+bx + c = 0$ với a, b, c nhập từ bàn phím.

Bài 3. Viết chương trình nhập vào hai số a và b. Kiểm tra số a có là số chẵn và là bội số của b không?

Bài 4. Viết chương trình nhập vào 3 số. Tìm giá trị lớn nhất và nhỏ nhất của 3 số đó.

Bài 5. Giả sử tiền điện sinh hoạt phải trả hàng tháng được tính bậc thang theo số KWh tiêu thụ như sau: (giá chưa bao gồm 10% thuế VAT)

- 100 KWh đầu tiên: 1000đ/KWh
- Từ KWh 101-150: 1500đ/KWh
- Từ KWh 151 trở đi: 1800đ/KWh

Viết chương trình nhập vào chỉ số cũ (CSC) và chỉ số mới (CSM) và cho biết tổng số tiền phải trả.

Bài 6. Viết chương trình nhập vào một ký tự chữ cái. Xuất ra kết quả: Nếu ký tự nhập vào là chữ thường thì đổi sang chữ hoa, ngược lại đổi sang chữ thường.

Bài 7. Viết chương trình nhập vào một ký tự từ bàn phím, kiểm tra nó và:

- Hiện thị *la so* nếu nó là số.
- Hiện thị *la chu hoa* nếu nó là chữ hoa.
- Hiện thị *la chu thuong* nếu nó là chữ thường.
- Ngoài ra, hiện thị *Khong la chu cai hoac chu so*.

Bài 8. Viết chương trình nhập vào 3 số a, b, c và kiểm tra xem a, b, c có tạo thành một tam giác không? Nếu có thì tính chu vi và diện tích của tam giác, nếu không thì in ra câu " Không tạo thành tam giác".

Bài 9. Viết chương trình nhập vào 3 số a, b, c và kiểm tra xem a, b, c có tạo thành một tam giác không và đó là tam giác gì (tam giác thường, cân, vuông, đều) ?

Bài 10. Nhập 4 số nguyên a, b, c và d. Hãy sắp xếp giá trị của 4 số nguyên này theo thứ tự tăng dần.

Bài 11.Viết chương trình nhập từ bàn phím 2 số a, b và một ký tự ch. Nếu:

- ch là “+” thì thực hiện phép tính $a + b$ và in kết quả lên màn hình.
- ch là “-” thì thực hiện phép tính $a - b$ và in kết quả lên màn hình.
- ch là “*” thì thực hiện phép tính $a * b$ và in kết quả lên màn hình.
- ch là “/” thì thực hiện phép tính a / b và in kết quả lên màn hình.

Bài 12.Viết chương trình nhập vào điểm trung bình của một học sinh. In ra kết quả xếp loại học tập của học sinh đó. Biết rằng:

- Nếu $ĐTB \geq 9$: Xếp loại xuất sắc.
- Nếu $8 \leq ĐTB < 9$: Xếp loại giỏi.
- Nếu $7 \leq ĐTB < 8$: Xếp loại khá.
- Nếu $5 \leq ĐTB < 7$: Xếp loại trung bình.
- Nếu $ĐTB < 5$: Xếp loại yếu.

Bài 13.Tính tiền đi taxi từ số km nhập vào. Biết:

- 1 km đầu giá 15000đ
- Từ km thứ 2 đến km thứ 5 giá 13500đ
- Từ km thứ 6 trở đi giá 11000đ
- Nếu trên 120km được giảm 10% tổng tiền.

Bài 14.Viết chương trình nhập vào tháng và năm. Cho biết tháng đó có bao nhiêu ngày?

LAB 03 – CÂU LỆNH LẶP

1. MỤC TIÊU

- Sử dụng thành thạo cấu trúc lặp trong C, bao gồm:
 - o for
 - o while
 - o do...while
- Lệnh break, continue ...

2. YÊU CẦU

- Hoàn thành và hiểu rõ Lab01, Lab02.
- Nắm vững cú pháp lặp: for, while, do-while.
- Kiến thức toán học: số nguyên tố, số chính phương, số hoàn hảo, USCLN, BSCNN...

3. BÀI TẬP THỰC HÀNH

Bài 1. Viết chương trình nhập vào một số nguyên $n > 0$. Tính giá trị các biểu thức:

a. $S = 1 + 2 + 3 + 4 + \dots + n$

b. $S = 1 + 1/2 + 1/3 + \dots + 1/n$

c. $S = 1 - 2 + 3 - 4 + \dots + (-1)^{n-1} n$

d. $S = 1^2 + 2^2 + \dots + n^2$ (Ghi chú: có thể dùng hàm $\text{pow}(x, y)$ để tính lũy thừa x^y)

e. $P = 1 * 2 * 3 * 4 * \dots * n = n!$

f. $P = 1 * 2 * 3 * 4 * 5 * \dots * (2n-1)$

g. $S = 1 + 1 * 2 + 1 * 2 * 3 + \dots + 1 * 2 * 3 * \dots * n$

h. $S = 1! - 2! + 3! - 4! + \dots + n!$

i.
$$S = \frac{1 + (1+2)! + \dots + (1 + \dots + n)!}{n!}$$

j.
$$S = 1 + \frac{1}{1+2} + \frac{1}{1+2+3} + \dots + \frac{1}{1+2+3+\dots+n}$$

k.
$$S = e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \dots + \frac{x^n}{n!}$$

l.
$$S = \frac{e^1}{1!} - \frac{e^2}{2!} + \frac{e^3}{3!} - \frac{e^4}{4!} + \dots + (-1)^{n+1} \frac{e^n}{n!}$$
 (Ghi chú: sử dụng hàm $\text{exp}(x)$ để tính e^x)

m.
$$S = \sqrt{n + \sqrt{(n-1) + \sqrt{(n-2) + \dots + \sqrt{1}}}}$$

n.
$$S = \sqrt{1 + \sqrt{2 + \sqrt{3 + \dots + \sqrt{n}}}}$$

Bài 2. Viết chương trình nhập 3 số nguyên a , b và n với $a, b < n$. Tính tổng các số nguyên dương nhỏ hơn n chia hết cho a nhưng không chia hết cho b .

Bài 3. Viết chương trình nhập số nguyên N . Hỏi N có bao nhiêu chữ số.

Bài 4. Viết chương trình in các ký tự từ 'A' đến 'Z' xuôi và ngược, chữ hoa và chữ thường.

Bài 5. Viết chương trình nhập vào số N . Tìm tổng các số trong nó.

Ví dụ: $N = 1234$. Tổng = 10.

Bài 6. Nhập một số nguyên dương N ($N > 0$). Sau đó:

- Kiểm tra N có phải là số đối xứng? Ví dụ: 121, 12321, ...
- Kiểm tra N có phải là số chính phương? Ví dụ: 4, 9, 16, ...
- Kiểm tra N có phải là số nguyên tố? Ví dụ: 2, 3, 5, 7, ...
- Kiểm tra các chữ số trong N có tăng dần hay giảm dần không?
- Chữ số lớn nhất và nhỏ nhất trong N ?

Bài 7. Viết chương trình nhập số nguyên thỏa điều kiện $0 < n < 50$, sau đó:

- In tất cả những số nguyên tố $\leq n$.
- Tính tổng các số nguyên tố $\leq n$.

Bài 8. Nhập vào số nguyên n , cho biết ký số lớn nhất có mặt trong n .

Ví dụ: nhập 12451, ký số lớn nhất là 5.

Bài 9. Viết chương trình nhập vào 2 số nguyên dương. Tìm USCLN và BSCNN của 2 số đó.

Bài 10. Viết chương trình tìm và in lên màn hình tất cả các số nguyên trong phạm vi từ 10 đến 99 sao cho tích của 2 chữ số bằng 2 lần tổng của 2 chữ số đó.

Bài 11. Viết chương trình nhập vào từ bàn phím lần lượt các số nguyên, dấu hiệu chấm dứt là số 0. Tính tổng và trung bình cộng của các số đã nhập.

Bài 12. Viết chương trình in bảng cửu chương n ($2 \leq n \leq 9$).

Bài 13. Viết chương trình in n số Fibonacci đầu tiên, biết rằng:

$$a_0 = a_1 = 1$$

$$a_n = a_{n-1} + a_{n-2}$$

LAB 04 – HÀM

1. MỤC TIÊU

Sử dụng thành thạo hàm trong C, bao gồm:

- Khai báo và định nghĩa hàm
- Phạm vi biến: Toàn cục, cục bộ ...
- Gọi hàm.

2. YÊU CẦU

- Hoàn tất Lab01, Lab02, Lab03.
- Định nghĩa hàm, gọi hàm.
- Biến toàn cục, biến cục bộ.
- Truyền tham số cho hàm.

3. BÀI TẬP THỰC HÀNH

Bài 1. Viết chương trình nhập vào 2 số nguyên x, y . Sau đó viết các hàm thực hiện các yêu cầu sau:

- a. Kiểm tra xem x có chia hết cho y không?
- b. Tìm số lớn nhất trong 2 số. Áp dụng tìm số lớn nhất trong ba số x, y, z với a, b, c nhập từ bàn phím.
- c. Tìm USCLN của 2 số nguyên. Áp dụng: nhập vào tử và mẫu số của một phân số, kiểm tra xem phân số đó đã tối giản hay chưa.

Bài 2. Viết một hàm tính tổng các chữ số của một số nguyên. Viết chương trình nhập vào một số nguyên, dùng hàm trên kiểm tra xem số đó có chia hết cho 3 không. Một số chia hết cho 3 khi tổng các chữ số của nó chia hết cho 3.

Bài 3. Viết chương trình thực hiện các yêu cầu sau, mỗi yêu cầu là một hàm:

- a. Đổi một ký tự hoa sang ký tự thường.
- b. Tìm giá trị nhỏ nhất của 4 số nguyên.
- c. Sắp xếp 4 số nguyên tăng dần.

Bài 4. Viết chương trình nhập vào một số nguyên dương n và thực hiện (Mỗi yêu cầu là một hàm):

- a. Tính $n!$.
- b. Trả về số đảo của số n .
- c. Kiểm tra n có phải là số đối xứng.
- d. Kiểm tra n có phải là số chính phương.
- e. Kiểm tra n có phải là số nguyên tố.
- f. Tính tổng các chữ số lẻ xuất hiện trong n .

Bài 5. Viết hàm in n số fibonacci đầu tiên.

Bài 6. Viết các hàm tính:

$$T1 = 1+2+3+4+\dots+n$$

$$T2 = 1-2+3-4+\dots+(-1)^n$$

$$T3 = 1 + 1/2 + 1/3 + \dots + 1/n$$

$$T4 = 1 + 1/(1+2) + 1/(1+2+3) + \dots + 1/(1+2+\dots+n)$$

$$T5 = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}$$

Trong đó n là số nguyên dương và x là một số bất kỳ được nhập từ bàn phím khi chạy chương trình.

Bài 7. Viết hàm in tất cả những số nguyên tố nhỏ hơn hoặc bằng n (n là số nguyên dương nhập từ bàn phím).

Bài 8. Viết hàm in tính tổng các chữ số chính phương $\leq n$ (n là số nguyên dương và $n \leq 100$).

Bài 9. Viết hàm in ra bảng cửu chương n (n là một tham số nguyên từ 1 đến 10).

Bài 10. Viết chương trình nhập vào 2 số nguyên a, b. Tạo một menu mà mỗi chức năng trong menu là 1 hàm, gồm:

- (1) Tổng hai số
- (2) Hiệu hai số
- (3) Thương 2 số
- (4) Tích 2 số
- (5) Tìm số lớn hơn
- (7) USCLN của hai số
- (8) BSCNN của hai số

Bài 11. Tam giác Pascal là một bảng số, trong đó hàng thứ 0 bằng 1, mỗi một số hạng của hàng thứ n+1 là một tổ hợp chập k của n ($C_{nk} = \frac{n!}{k!(n-k)!}$)

Tam giác Pascal có dạng sau:

```

1 ( hàng 0 )
1 1 ( hàng 1 )
1 2 1 ( hàng 2 )
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1 (hàng 6)
.....
    
```

Viết chương trình in lên màn hình tam giác Pascal có n hàng (n nhập vào khi chạy chương trình) bằng cách tạo hai hàm tính giai thừa và tính tổ hợp.

Bài 12. Viết chương trình phân tích một số nguyên dương ra thừa số nguyên tố.

LAB 05 – MẢNG & CẤU TRÚC

1. MỤC TIÊU

- Sử dụng thành thạo các thao tác trên mảng trong C, bao gồm:
 - o Khai báo, truy cập mảng 1 chiều và mảng 2 chiều.
 - o Hàm và mảng.
 - o Giải thuật cơ bản về mảng: sắp xếp, tìm kiếm, chèn phần tử vào mảng, xóa phần tử khỏi mảng, kiểm tra mảng tăng/giảm dần...
 - o Tách/gộp mảng
- Sử dụng thành thạo một số thao tác trên kiểu cấu trúc trong C, bao gồm:
 - o Khai báo cấu trúc.
 - o Truy cập các thành phần trong cấu trúc.
 - o Thao tác trên mảng cấu trúc.

2. YÊU CẦU

- Hoàn tất Lab01, Lab02, Lab03, Lab04
- Nắm vững kiến thức về mảng, cấu trúc và hàm.

3. BÀI TẬP THỰC HÀNH

Bài 1. Viết chương trình nhập 1 mảng số nguyên có n phần tử, rồi tìm phần tử nhỏ nhất, phần tử lớn nhất trong mảng.

Bài 2. Viết chương trình nhập 1 mảng số nguyên có n phần tử:

- Đếm số phần tử chẵn trong mảng.
- Tính tổng của các phần tử chẵn.
- Đếm số phần tử lẻ trong mảng.
- Tính tổng các phần tử lẻ trong mảng.
- Trung bình cộng của mảng.

Bài 3. Viết chương trình nhập 1 mảng số nguyên có n phần tử, tính tổng các số dương, tổng các số âm trong mảng.

Bài 4. Viết chương trình nhập 1 mảng số nguyên có n phần tử, in ra màn hình:

- Số dương lớn nhất của mảng và vị trí của nó.
- Số dương nhỏ nhất của mảng và vị trí của nó.

Bài 5. Viết chương trình nhập vào một ma trận vuông có $n \times n$ phần tử:

- Xuất ra màn hình ma trận đó
- Tính tổng các phần tử nằm trên đường chéo chính.
- Tính tổng các phần tử chẵn trong ma trận.
- Tổng các phần tử thuộc dòng chẵn và cột lẻ.

Bài 6. Viết chương trình thao tác trên một mảng số nguyên có n phần tử, mỗi thao tác là một hàm:

- Hàm nhập mảng: `Nhap_mang()`
- Hàm xuất mảng: `Xuat_mang()`
- Hàm sắp xếp mảng: `Sap_xep()`
- Hàm tính tổng các phần tử của mảng: `Tong_mang()`
- Hàm đếm các phần tử có giá trị $= k$: `Dem_pt()`
- Hàm tìm vị trí của phần tử có giá trị là x : `Tim_x()`

Bài 7. Viết chương trình chèn một phần tử x (kiểu `int`) tại vị trí i trong mảng A .

Bài 8. Viết chương trình xóa phần tử tại vị trí i trong mảng A .

Bài 9. Viết chương trình xóa các phần tử có giá trị x trong mảng A .

Bài 10. Viết chương trình nhập vào một chuỗi:

- In ra các ký tự lên màn hình, mỗi ký tự nằm trên một dòng.
- Đếm số ký tự trong chuỗi.
- Đếm số ký tự nguyên âm trong chuỗi.

Bài 11. Viết chương trình nhập vào một chuỗi (hai từ chỉ cách một khoảng trắng):

- Đếm số từ có trong chuỗi.
- In ra từ đầu tiên.
- In ra từ cuối cùng.
- In ra các từ trong chuỗi, mỗi từ nằm trên một dòng.

Bài 12. Cho cấu trúc `NHANVIEN` như sau:

`MaNV`: kiểu số nguyên có giá trị trong khoảng $0 \dots 65535$

`Họ tên`: kiểu chuỗi.

`Địa chỉ`: kiểu chuỗi.

`CBQL`: có giá trị 1 nếu nhân viên này là cán bộ quản lý.

Viết chương trình thực hiện:

- Nhập vào thông tin của một nhân viên.
- Nhập vào thông tin của n nhân viên và n được nhập từ bàn phím. In ra họ tên của các nhân viên là cán bộ quản lý.

Bài 13. Cho cấu trúc `NHANVIEN` như sau:

`MaNV`: kiểu số nguyên có giá trị trong khoảng $0 \dots 65535$

`Họ tên`: kiểu chuỗi.

`Địa chỉ`: kiểu chuỗi.

`CBQL`: có giá trị 1 nếu nhân viên này là cán bộ quản lý.

Viết chương trình thực hiện:

- Nhập vào thông tin của n nhân viên và n được nhập từ bàn phím.
- Nhập thêm thông tin của một nhân viên và nhập một số nguyên k. Thực hiện việc chen nhân viên mới vào danh sách tại vị trí k.
- Xóa các nhân viên không là cán bộ quản lý ra khỏi danh sách.

Bài 14.Viết chương trình quản lý sinh viên gồm các thông tin: Mã SV, họ và tên, ngày sinh, nơi sinh, lớp học, điểm thi toán, điểm thi triết, điểm thi anh văn, điểm trung bình là điểm bình quân của các môn học, xếp loại dựa vào điểm trung bình. Chương trình cho phép thực hiện các chức năng sau:

- Nhập thông tin của sinh viên.
- Xem thông tin của một sinh viên (dựa vào mã SV).
- Xem thông tin trích ngang của một lớp bất kỳ.
- Xem danh sách SV theo xếp loại.

Bài 15.Viết chương trình quản lý bán hàng gồm các thông tin sau: Mã hàng, Tên hàng, Đơn vị tính, số lượng, đơn giá, thành tiền= số lượng*đơn giá, chiết khấu =5% *thành tiền, doanh thu=thành tiền – chiết khấu. Chương trình cho phép thực hiện các công việc sau:

- Nhập thông tin bán hàng (gồm các thông tin trên).
- In liệt kê chi tiết bán hàng
- In ra 2 mặt hàng có số lượng bán lớn nhất.
- In ra 2 mặt hàng có doanh thu bán nhỏ nhất.
- Sắp xếp thứ tự giảm dần theo doanh thu bán hàng.
- Hãy lập bảng thống kê bán hàng theo bảng sau:

Tên hàng	Doanh thu
Mặt hàng A	
Mặt hàng B	
Tổng cộng	

Dữ liệu giả định nhập vào:

Mã hàng	Tên hàng	Đơn vị	Số lượng	Đơn giá	Thành tiền	Doanh thu
01	Gạo	Kg	11			
02	Mì	Bao	3			
03	Nếp	Kg	9			
01	Gạo	Kg	9			
01	Gạo	Kg	5			
02	Mì	Bao	12			
03	Nếp	Kg	10			

PHẦN 3

PHỤ LỤC

A - MỘT SỐ HÀM CHUẨN THÔNG DỤNG

Ngoài một số hàm được giới thiệu trong phần lý thuyết, khi lập trình trên C/C++ có thể sử dụng một số hàm sau:

1. Các hàm chuyển đổi dữ liệu

✓ **atof**

double atof(const char *s);

☞ Phải khai báo **math.h** hoặc **stdlib.h**

Chuyển đổi 1 chuỗi sang giá trị double.

Ví dụ: float f;

char *str = "12345.67";

f = atof(str);

Kết quả f = 12345.67;

✓ **atoi**

int atoi(const char *s);

☞ Phải khai báo **stdlib.h**

Chuyển đổi 1 chuỗi sang giá trị int.

Ví dụ: int i;

char *str = "12345.67";

i = atoi(str);

Kết quả i = 12345

✓ **itoa**

char *itoa(int value, char *string, int radix); ☞ Phải khai báo **stdlib.h**

Chuyển đổi số nguyên value sang chuỗi string theo cơ số radix.

Ví dụ: int number = 12345;

char string[25];

itoa(number, string, 10); //chuyển đổi number sang chuỗi theo cơ số 10

Kết quả string = "12345";

itoa(number, string, 2); //chuyển đổi number sang chuỗi theo cơ số 2

Kết quả string = "11000000111001";

✓ **tolower**

int tolower(int ch);

☞ Phải khai báo **ctype.h**

Đổi chữ hoa sang chữ thường.

Ví dụ: int len, i;

char *string = "THIS IS A STRING";

len = strlen(string);

for (i = 0; i < len; i++)

```
string[i] = tolower(string[i]); //đổi từ kí tự trong string thành chữ thường
```

✓ **toupper**

```
int toupper(int ch);
```

☞ Phải khai báo **ctype.h**

Đổi chữ thường sang chữ hoa.

Ví dụ: int len, i;

```
char *string = "this is a string";
```

```
len = strlen(string);
```

```
for (i = 0; i < len; i++)
```

```
string[i] = toupper(string[i]); //đổi từ kí tự trong string thành chữ thường
```

2. Các hàm toán học

✓ **abs**

```
int abs(int x);
```

☞ Phải khai báo **stdlib.h**

Cho giá trị tuyệt đối của số nguyên x.

Ví dụ: int num = - 123;

```
num = abs(num); //kết quả num = 123
```

✓ **labs**

```
long int labs(long int x);
```

☞ Phải khai báo **stdlib.h**

Cho giá trị tuyệt đối của số nguyên dài x.

Ví dụ: int num = - 12345678L;

```
num = labs(num); //kết quả num = 12345678
```

Ghi chú: Tương tự cho số thực, ta có hàm *fabs(double x)* để tính giá trị tuyệt đối của số thực x.

✓ **randomize**

```
randomize();
```

Khởi đầu bộ tạo số ngẫu nhiên, ta nên chạy hàm này trước khi dùng hàm *rand* và *random*.

✓ **rand**

```
int rand(void);
```

☞ Phải khai báo **stdlib.h**

Cho 1 giá trị ngẫu nhiên từ 0 đến 32767

Ví dụ: int num;

```
randomize(); //dùng hàm này để khởi đầu bộ số ngẫu nhiên
```

```
num = rand(); //kết quả num = 1 con số trong khoảng 0..32767
```

✓ **random**

```
int random(int num);
```

☞ Phải khai báo **stdlib.h**

Cho 1 giá trị ngẫu nhiên từ 0 đến 32767

Ví dụ: `int n;`
 `randomize();`
 `n = random(100);` //kết quả `n = 1` con số trong khoảng 0..99

✓ **exp**

exp(double x) ;

Tính e^x .

✓ **log**

log(double x) ;

Tính logarit tự nhiên của x

✓ **pow**

double pow(double x, double y); ☞ Phải khai báo **math.h**

Tính x mũ y . Chú ý là y mũ x có thể tính theo công thức: y mũ $x = \exp(y \cdot \log(x))$.

Ví dụ: `double x = 2.0, y = 3.0, z;`
 `z = pow(x, y);` //kết quả `z = 8.0`

✓ **sqrt**

double sqrt(double x); ☞ Phải khai báo **math.h**

Tính căn bậc 2 của x.

Ví dụ: `double x = 4.0, y;`
 `y = sqrt(x);` //kết quả `y = 2.0`

✓ **cos**

double cos(double x);

Tính cos của x (đơn vị của x là radian, 1 radian bằng khoảng 57 độ, cụ thể là bằng 180/3.14). Tương tự, hàm *sin(double x)* và *tan(double x)* được sử dụng để tính sin và tang của x.

✓ **floor**

floor(double x) ;

Cho số nguyên lớn nhất dưới x.

Ví dụ: floor(8.6) là 8

✓ **ceil**

ceil(double x);

Cho số nguyên bé nhất trên x.

Ví dụ: ceil(8.6) là 9

B - HƯỚNG DẪN SỬ DỤNG PHẦN MỀM DEV-C++

(Phiên bản Dev C++ 4.9)

1. Giới thiệu

Dev-C++ (Dev-Cpp, phiên bản hiện đang sử dụng là 5.11) là một bộ công cụ phát triển tích hợp (IDE Integrated Development Environment) các ứng dụng C/C++ thuộc dạng mã nguồn mở. DevCpp dựa trên trình biên dịch mã nguồn mở MinGW (Minimalist GNU* for Windows, <http://www.mingw.org/>). MinGW sử dụng GCC (the GNU G++ compiler collection) sử dụng cho cả hệ thống Windows và Linux. Hiện nay DevCpp là công cụ phát triển các ứng dụng C/C++ được sử dụng rộng rãi để dạy về lập trình cũng như để phát triển các ứng dụng mã nguồn mở.

Mặc dù không có nhiều tính năng cao cấp như các công cụ IDE thương mại khác (Visual Studio của Microsoft) nhưng DevCpp vẫn là một công cụ rất thích hợp trong môi trường giáo dục ở bậc đại học khi dạy và học các ngôn ngữ C/C++. Lý do là Dev-C++ hỗ trợ C/C++ chuẩn, mã nguồn viết trên Dev-C++ có thể biên dịch trên các IDE khác, nhưng điều ngược lại có thể không đúng.

2. Cài đặt

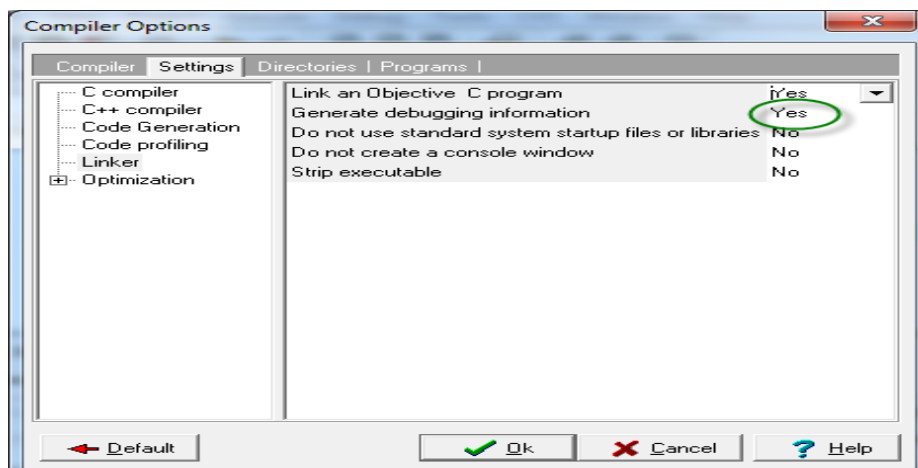
Có 2 phiên bản của Dev-C++. Bản cài đặt và bản portable.

- Bản cài đặt: Tải file cài đặt tại địa chỉ: <http://www.bloodshed.net/devcpp.html>, sau đó thực hiện cài đặt theo hướng dẫn trên màn hình.
- Bản portable: Phiên bản portable không cần cài đặt có thể tải về từ địa chỉ <http://sourceforge.net/projects/devcpp-portable/>

3. Thao tác với Dev-C++

3.1. Cấu hình Dev-C++ (Chỉ thực hiện lần đầu)

- Chạy file cài đặt DEV-C++ (file devcpp.exe).
- Vào menu **Tools** chọn **Compiler Options**.
- Vào tab **Settings**, nhấn vào **Linker** ở khung bên trái và thay đổi thông số **Generate debugging information** sang **Yes**.
- Nhấn OK.



3.2. Tạo chương trình mới hoặc dự án mới

Các bước sử dụng Dev-CPP để tạo ra chương trình C/C++ đơn giản bao gồm các bước chính như sau:

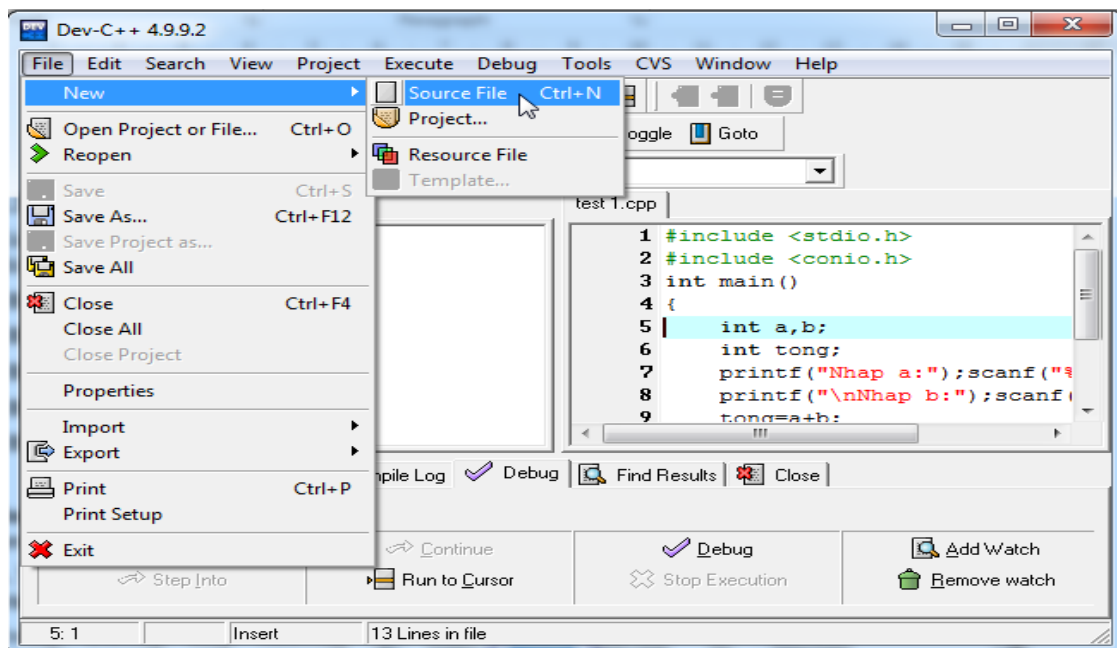
- **Bước 1.** Tạo file/project
- **Bước 2.** Soạn thảo mã nguồn
- **Bước 3.** Biên dịch, sửa lỗi (nếu có)
- **Bước 4.** Chạy thử và kiểm tra kết quả
- **Bước 5.** Debug (nếu cần)

➤ **Bước 1: Tạo file hoặc project (dự án) mới**

a. Trường hợp chương trình đơn giản

Trong trường hợp chương trình đơn giản, chúng ta chỉ cần 1 file để lưu mã nguồn, cách thực hiện như sau:

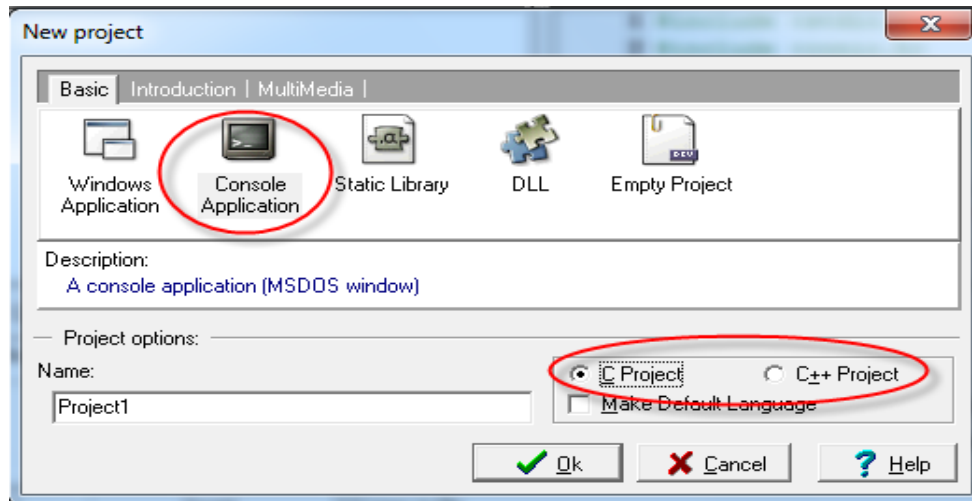
- Chọn menu **File/New/ Source File (Ctrl-N)**.
- Nhập mã nguồn trực tiếp vào file.



b. Trường hợp chương trình phức tạp

Đối với chương trình C/C++ phức tạp, 1 chương trình cần nhiều file để thực hiện thì DevCPP có khái niệm dự án (project). Project là nơi cất giữ tất cả những thông tin liên quan đến mã nguồn, thư viện, tài nguyên... để biên dịch thành chương trình.

- Để tạo Project mới, vào menu **File/New/Project**.
- Chọn **Console Application** và chọn **C Project** nếu là lập trình C, Chọn **C++ Project** nếu là C++. Nhập tên Project, tên Project cũng là tên của file thực thi (.exe) sẽ tạo ra khi biên dịch.
- Nhấn **OK**. Dev-C++ sẽ hỏi nơi lưu trữ Project. Chọn nơi lưu trữ Project và nhấn **OK**.



➤ Bước 2: Soạn thảo mã nguồn

Nếu là **bước 1.a** thì chúng ta tiến hành nhập mã nguồn chương trình.

Nếu là **bước 1.b**, chúng ta có 2 cách thêm mã nguồn vào dự án.

- Vào menu **File** chọn **New Source File** (hoặc tổ hợp phím **CTRL+N**).
- Vào menu **Project** chọn **New File**.

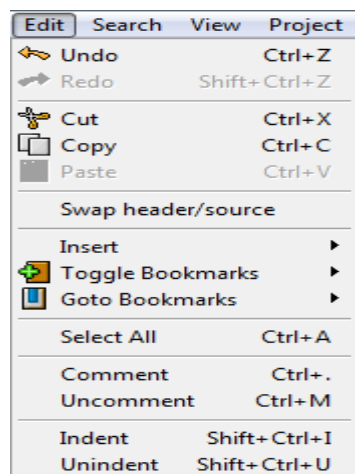
Dev-C++ sẽ không hỏi tên file cho đến khi chúng ta thực hiện một trong các bước sau:

- Compile
- Save the project
- Save the source file
- Exit Dev-C++

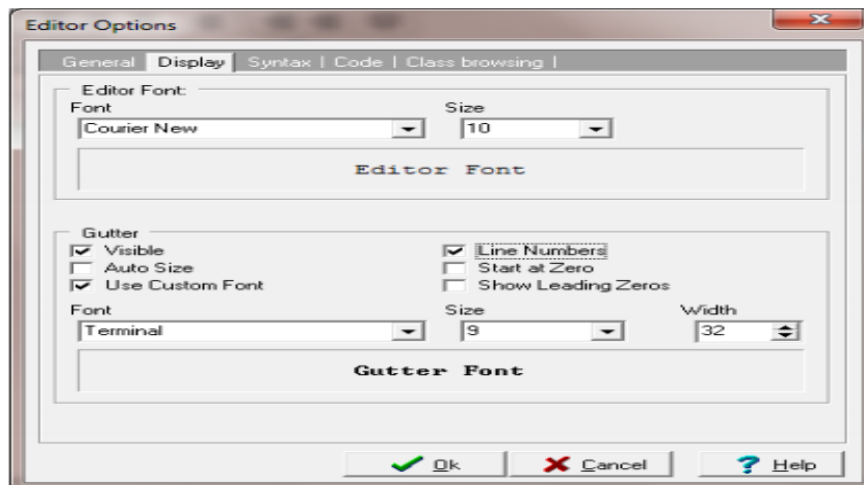
Chúng ta cũng có thể thêm file nguồn có sẵn bằng 1 trong 2 phương pháp sau:

- Vào menu **Project** chọn **Add to Project**.
- Nhấn phím phải chuột vào project name ở khung bên trái và chọn **Add to Project**.

Các quy tắc soạn thảo gần giống với chương trình soạn thảo văn bản chuẩn (MS Word). Tham khảo menu Edit để biết chi tiết và các phím tắt trong khi soạn thảo, một vài tính năng như: Undo, Redo, Copy, Paste, Cut, Select All...



Để thuận tiện cho việc nhìn và xem code (chữ lớn hơn và có hiển thị số dòng): vào menu **Tool / Editor Option** → Chọn Tab **Display** → Click chọn **Line Numbers** và chọn **Size**.



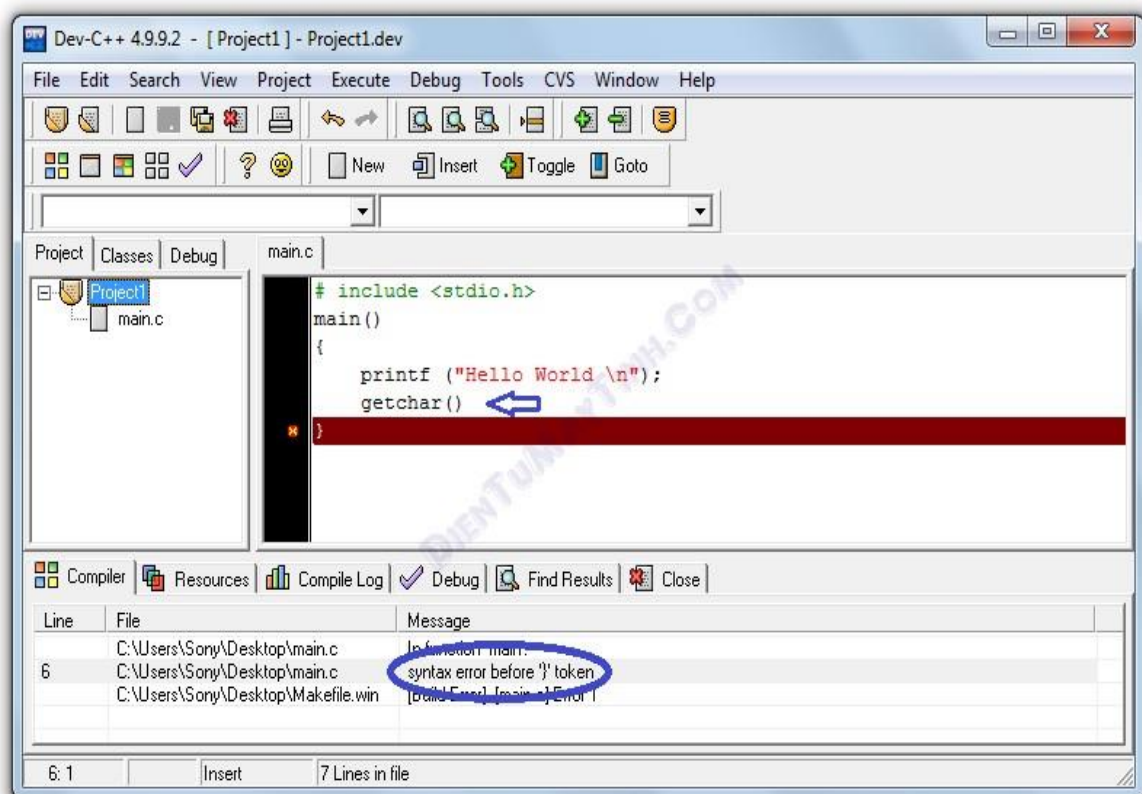
➤ Bước 3: Biên dịch (Compile)

Sau khi nhập file nguồn xong, để biên dịch chúng ta thực hiện:

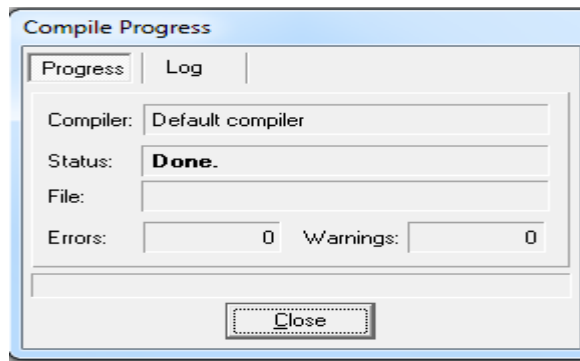
- Vào menu **Execute** chọn **Compile** (hoặc nhấn tổ hợp phím CTRL+F9).

Thông thường, khi biên dịch lần đầu sẽ dễ có lỗi về Syntax. Lỗi Syntax sẽ hiển thị ở tab biên dịch ở phía dưới màn hình. Để biết lỗi xảy ra ở vị trí nào của source code, double-click vào lỗi đó và control màn hình sẽ trở về vị trí có lỗi khi biên dịch.

Hình dưới minh họa lỗi syntax (thiếu dấu ;) khi biên dịch.



Trong trường hợp biên dịch thành công, hộp thoại thông báo có status là **Done**.



➤ **Bước 4: Thực thi chương trình**

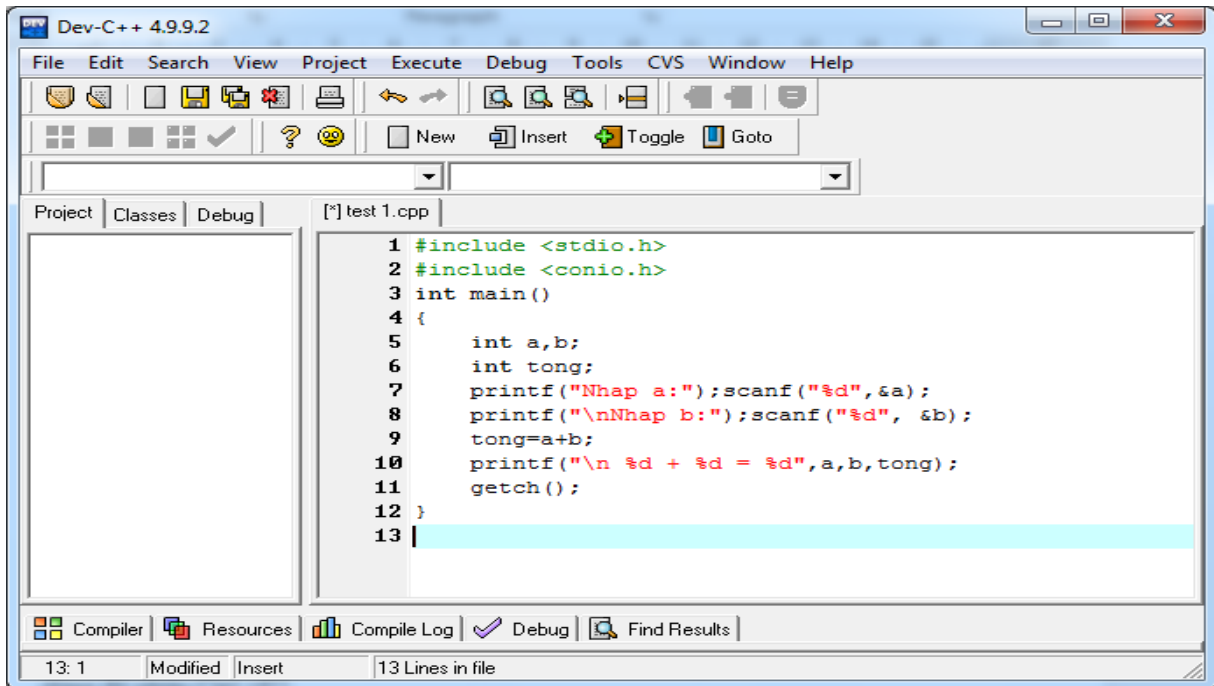
- Vào menu **Execute** chọn **Run** (hoặc nhấn Ctrl + F10).

☞ **Lưu ý:**

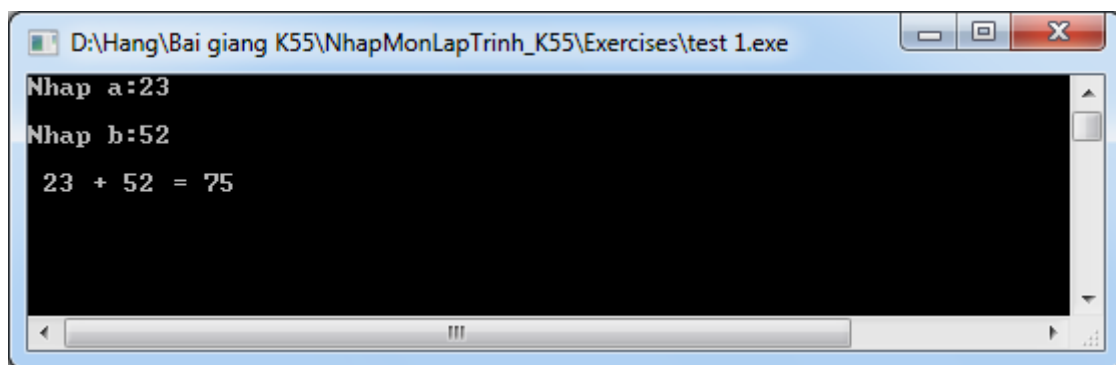
- ☞ Bước 4 và bước 5 có thể thực hiện đồng thời bằng menu **Execute**, chọn **Compile & Run** (hoặc nhấn F9).
- ☞ Màn hình kết quả không xuất hiện:
 - Nếu khi chạy chương trình, bạn thấy màn hình console DOS màu đen xuất hiện, chớp sáng 1 hoặc vài lần và trở lại màn hình soạn thảo thì tức là chương trình của bạn đã chạy xong.
 - Điều này có nghĩa là chương trình thực thi xong và tự đóng cửa sổ console sau khi chương trình thoát. Để giải quyết trường hợp này chúng ta có thể sử dụng lệnh **getch();** trước dấu “**}**” đóng hàm main để tạm dừng màn hình và có thể quan sát kết quả chương trình. Và để lệnh đó sử dụng được ta thêm thư viện: **#include<conio.h>** ở đầu chương trình.

Để kiểm tra hoạt động của Dev-Cpp, sau đây là chương trình “Tính tổng 2 số nguyên” sử dụng ngôn ngữ C. Bạn hãy thực hiện theo các bước trên, nhập mã nguồn và chạy thử để kiểm tra kết quả.

- Nhấn Ctrl + F9 để biên dịch (Compile), F9 để biên dịch và chạy (Compile + Run), Ctrl + F10 để chạy (Run).
- Nhập giá trị của a và b (Nhấn Enter để kết thúc nhập).



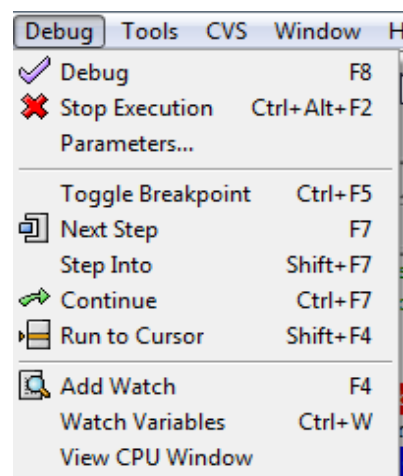
Màn hình kết quả như sau:



➤ Bước 5: Debug

Trong trường hợp chương trình chạy không như ý của bạn, sử dụng công cụ Debug để xác định cụ thể. Chức năng Debug cơ bản có tại tab Debug ở cuối màn hình, nhiều chức năng khác nhau có tại menu Debug. Một vài tính năng chính của Debug như:

- Chạy tới con trỏ màn hình.
- Thêm cửa sổ theo dõi.
- Theo dõi giá trị của biến...



C - MỘT SỐ QUY ƯỚC KHI VIẾT MÃ NGUỒN C/C++

Khi lập trình, việc sửa lỗi, hay dùng lại mã nguồn là điều rất cần thiết. Để sử dụng mã nguồn một cách hiệu quả và làm cho người đọc dễ hiểu thì việc trình bày mã nguồn là điều rất quan trọng. Nếu không có một quy tắc nào trong viết mã nguồn, thì chính người viết ra nó cũng khó hiểu được mã nguồn đó sau một thời gian dài. Do đó chúng ta cần có các **Quy tắc** viết mã nguồn.

1. Các quy tắc đặt tên trong chương trình

- **Quy tắc 1:** Tên các định danh (hằng, biến, hàm, mảng, cấu trúc, lớp...) trong chương trình phải thể hiện được ý nghĩa của nó.

- **Quy tắc 2:** Tên hằng được viết hoa toàn bộ, các từ cách nhau bằng ký tự '_'.

```
const int NUMBER_OF_ELEMENTS 100;
```

- **Quy Tắc 3:** Tên biến phải thể hiện được ý nghĩa của nó, viết bằng chữ thường, các từ cách nhau bằng ký tự '_'.

```
int tu_so, mau_so;
```

- **Quy tắc 4:** Tên hàm phải là động từ thể hiện hành động cần thực hiện.

```
int kiem_tra_so_nguyen_to (int n);
```

2. Quy tắc viết câu lệnh

- **Quy tắc 5:** Mỗi câu lệnh nên được viết riêng trên một dòng.

```
if (a>b)
```

```
    printf("%d > %d", a, b);
```

```
else
```

```
    printf("%d <= %d",a, b);
```

- **Quy tắc 6:** Các câu lệnh if, switch, while, for, do được viết riêng trên một đoạn.

```
if (a > b)
```

```
    printf( "a lon hon b");
```

```
for (int i = 0; i < n; i++)
```

```
    x = x + 5;
```

```
    k = k * x;
```

3. Quy tắc về khoảng trắng (space)

- **Quy tắc 7:** Các câu lệnh nằm giữa dấu '{', '}' được viết cách vào một khoảng tab. Các câu lệnh con của if, else, while, for, switch được viết cách vào một khoảng tab.

```
i=1;
```

```
s=0;
```

```
while(i<=10)
```

```
{    s=s+i;
```

```
    i=i++;
```

```
}
```


- **Quy tắc 8:** Xung quanh các toán tử 2 ngôi (+, -, *, >=,...) viết cách một khoảng trắng.

$x = x + 5 * a - c;$

$if (a >= b)$

4. Quy tắc viết chú thích

- **Quy tắc 9:** Chú thích phải rõ ràng, dễ hiểu và diễn giải được ý nghĩa của đoạn lệnh. Đầu mỗi hàm, struct, class, hàm phải có chú thích diễn giải ý nghĩa của nó.

- **Quy Tắc 10:** Sử dụng // cho chú thích 1 dòng; sử dụng /* */ cho các chú thích từ 2 dòng trở lên. Dùng /* */ để tạm thời chú thích các đoạn chương trình để **test** và **debug**.

Mỗi file nguồn, cả **.CPP** và **.H**, đều phải bắt đầu bằng một khối chú thích đủ để người đọc có thể hiểu mục đích của file.

Mẫu như sau:

```

/*****
* Tên/Name:   Họ tên
* Project: mô tả/tên dự án (một dòng, giống nhau tại mọi file)
* Purpose: Mục đích sử dụng của mã chương trình hoặc các khai báo trong
* file này (một vài dòng)
*****/

```

Mỗi **lớp**, **hàm**, **phương thức** phải có một khối chú thích mô tả ngắn gọn lớp, hàm, phương thức đó làm gì; đối với hàm/phương thức: liệt kê tất cả các **tham số**, nêu rõ **ý nghĩa** của tham số; và mô tả điều kiện trước và sau của hàm/phương thức đó.

Có thể chú thích các đoạn code bên trong hàm, tuy nhiên chỉ nên chú thích đủ hiểu. Quá nhiều chú thích và chú thích thừa làm code trông rối. Tất cả các chú thích phải được lùi đầu dòng cùng đoạn code quanh nó.

5. Quy tắc thiết kế

- **Quy tắc 11:** Mỗi phương thức/hàm (kể cả hàm main()) không nên viết quá dài (vượt quá 30 dòng) tính từ dấu mở hàm '{' tới ngoặc kết thúc hàm '}'.
- **Quy Tắc 12:** Hạn chế việc dùng biến toàn cục.

D - CÁC LỖI THƯỜNG GẶP KHI VIẾT CHƯƠNG TRÌNH C/C++¹

Việc học lập trình không phải là chuyện đơn giản như nói là có thể làm được mà nó là một quá trình tích lũy kinh nghiệm trên cơ sở thực hành thường xuyên. Những người mới bắt đầu với ngôn ngữ lập trình C/C++ thường gặp phải những lỗi làm họ sorn nản lòng, nhưng thực sự những lỗi đó có quá khó không? Phần này đưa ra một số lỗi cơ bản thường gặp phải đối với những ai mới bắt đầu học lập trình C/C++.

1. Quên khai báo các biến sử dụng trong chương trình (Undeclared Variables)

Ví dụ: `int main()`

```
{    printf(x);
    scanf("%d", &x);
}
```

Chương trình trên bị lỗi vì trình biên dịch không biết `x` là cái gì. Chúng ta cần phải khai báo biến `x`.

2. Không khởi tạo giá trị cho biến (Uninitialized Variable)

Ví dụ: `int count;`

```
while(count<100)
{
    printf("%d", count);
}
```

Khi thi hành, chương trình không in ra gì cả. Lý do là ở đoạn chương trình trên, biến `count` chưa được khởi tạo giá trị. Vì vậy khi chương trình chạy nó có thể là giá trị bất kỳ nào trong dãy các số nguyên (`int`). Có thể nó sẽ có giá trị là 586 chẳng hạn, vậy thì nó sẽ không vào trong vòng lặp `while` khiến cho kết quả chương trình bị sai, có thể chương trình sẽ in ra kết quả là các giá trị rác từ -1000 đến 99.

3. Thiết lập một biến với một giá trị chưa khởi tạo (Setting a variable to an uninitialized value)

Ví dụ: `int a, b;`

```
int sum=a+b;
printf("Enter two numbers to add: ");
scanf("%d", &b);
printf("The sum is: %d", sum);
```

Khi chạy (RUN):

Enter two numbers to add:

1

3

The sum is: -1393

Câu hỏi đặt ra: Có gì sai với chương trình trên?

¹ <http://www.cprogramming.com/tutorial/common.html>

Khi chúng ta gán giá trị cho một biến, nó có giá trị đó cho đến khi nó được gán một giá trị khác. Trong chương trình ví dụ trên, bởi vì a, b không được khởi tạo giá trị nên tổng của chúng sẽ là một giá trị rác ngẫu nhiên. Có thể sửa chương trình trên như sau:

```
int a, b;

int sum;

printf("Enter two numbers to add: ");

scanf("%d%d",&a, &b);

sum=a+b;
printf("The sum is: %d", sum);
```

4. Sử dụng một dấu bằng trong phép so sánh (Using a single equal sign to check equality)

Ví dụ: `char x='Y';`

```
while(x='Y')
{
    //...
    printf("Continue? (Y/N)");
    x=getche();
}
```

Câu hỏi đặt ra: Tại sao vòng lặp này không bao giờ kết thúc?

Nếu sử dụng một dấu bằng trong phép so sánh, chương trình thay vì so sánh giá trị bên trái có đúng với giá trị bên phải hay không thì nó sẽ thực hiện phép gán giá trị bên phải cho bên trái, và phép gán này tất nhiên trả về giá trị TRUE. Do đó vòng lặp không bao giờ kết thúc. Do đó chúng ta nên nhớ sử dụng `==` cho phép so sánh của mình. Và lời khuyên trong trường hợp này là nên đặt biến phía bên phải giá trị để cho chương trình báo lỗi thay vì thực hiện một vòng lặp vô hạn. Chương trình có thể viết lại như sau:

```
char x='Y';
while('Y'==x)
{
    //...
    printf("Continue? (Y/N)");
    x=getche();
}
```

5. Không khai báo hàm (Undeclared function)

Ví dụ: `int main()`

```
{
    menu();
}

void menu(){ //...}
```

Câu hỏi đặt ra: Chương trình trên đã xảy ra lỗi gì?

Trình biên dịch không biết hàm `menu()` được khai báo trong hàm `main()` là gì. Lý do là hàm `menu()` được định nghĩa phía dưới hàm `main()`. Do đó hàm `menu()` nên được đặt phía trên hàm `main()`, hoặc ta có thể sử dụng một khai báo nguyên mẫu (prototype) cho hàm `menu()` nếu muốn định nghĩa nó bên dưới hàm `main()`, như ví dụ dưới đây:

```
void menu();
int main()
{
    menu();
}
void menu(){ ...}
```

6. Thừa dấu chấm phẩy (Extra semicolons)

Ví dụ:

```
int x;
for(x=0; x<100; x++);
    printf("%d5",x);
```

Tất nhiên kết quả chương trình đưa ra không phải là các số từ 1 đến 100. Bởi vì trong chương trình trên thừa một dấu chấm phẩy (;) sau vòng lặp for. Nếu chúng ta đặt thừa một dấu chấm phẩy ở bất kỳ nơi nào trong chương trình thì rất có thể chương trình sẽ báo lỗi.

7. Vượt quá giới hạn của mảng (Overstepping array boundaries)

Ví dụ:

```
int array[10];
//...
for(int x=1; x<=10; x++)
    printf("%d",array[x]);
```

Câu hỏi đặt ra: Tại sao chương trình chạy ra có kết quả không đúng?

Trong C/C++, một mảng khi khai báo sẽ bắt đầu từ phần tử số 0 chứ không phải là số 1, và kết thúc sẽ là độ dài của mảng khai báo trừ đi 1. Chương trình trên sửa lại như sau:

```
int array[10];
//...
for(int x=0; x<10; x++)
    printf("%d",array[x]);
```

8. Lạm dụng các toán tử && và || (Misusing the && and || operator)

Ví dụ:

```
int value;
do
{
    //...
    value=10;
} while(!(value==10) || !(value==20))
```

Câu hỏi đặt ra: Tại sao giá trị value bằng 10 mà vòng lặp vẫn chạy?

Nên nhớ công thức của hàm Boolean là: $!(A \parallel B)$ tương đương với $!A \&\& !B$. Do đó chương trình đúng sẽ là:

```
int value;
do
{
    //...
    value=10;
}while(!(value==10) && !(value==20))
```